

Wisteria/BDEC-01 システム 利用手引書

Version 1.1.24
2024 年 5 月 24 日
富士通株式会社

改版履歴

版	発行年月日	章	改版理由
1.1.0	2021.08.02	- 4.1 5. 5.8.4	初版リリース Environment Modules の設定変更を反映 Environment Modules の設定変更を反映 pjstat -rsc の表示形式を統一化する変更を反映
1.1.1	2021.08.26	2.8	デスクトップ環境の設定変更を反映
1.1.2	2021.09.22	2.7 2.8	pysub のライン番号指定方法の変更を反映 vncserver のパスワード設定・入力の追記
1.1.3	2021.10.29	2.7 2.8	pysub 実行時の注意事項を追加 vncserver のポート確認方法の修正、本文表現の修正 環境変数 DISPLAY の利用を追加
1.1.4	2021.12.17	2.6.6 5. 6.3	章を 3.1.1 に移動、文面を修正 Aquarius にて gcc 環境の自動設定解除に伴い、ジョブスクリプト例及び文面の修正
1.1.5	2022.01.28	1.6.1 2.6.5 3.1.1.1 5.3.2	トークンの計算式を修正 表 2-6 の show_module コマンドのオプションなしの説明を修正 JupyterNotebook のホームディレクトリの変更方法を追記 Odyssey 用ジョブ資源オプションに strict 指定の説明を追記
1.1.6	2022.03.31	5.2.1	share-1 の割り当てノード数を 6 に変更
1.1.7	2022.05.27	5.7.1.2	ジョブ連携用リソースグループと WaitIO ライブラリを使用した運用を記載
1.1.8	2022.06.20	8	表 8-2 RIST から提供されたアプリケーション一覧を追加
1.1.9	2022.06.24	3.1	lpomoea-01 システムへのアクセスパスを記載
1.1.10	2022.07.29	8	表 8-2 RIST から提供されたアプリケーション一覧に公開元サイトの URL を追加
1.1.11	2022.08.02	5.2.2	interactive-o_n1 のジョブあたりの実行時間制限を2時間に変更
1.1.12	2022.08.19	5.4.1	ジョブスクリプトのシェバングに関する注意事項を修正
1.1.13	2022.09.16	6.3.1.3	VTune プロファイラの利用方法について注意事項を追加
1.1.14	2022.10.28	2.3.1.1.2 2.4.1.2 2.5.1.2 3.1	ssh-keygen コマンドを用いた ssh 秘密鍵/公開鍵ペアの作成方法を追加 Windows PowerShell を用いた SSH 接続方法を追加 rsync コマンドの使用方法を追加 chhome コマンドはログインノードで実行するよう説明を追記
1.1.15	2022.11.25	5.13	計算ノードから外部リソースへのアクセス方法を追記
1.1.16	2022.12.23	3.1.1 5.3.1	chhome コマンドで入力するパスワードについて説明を追記 ファイル名の表記方法を追記
1.1.17	2023.01.13	2.8.2	Apache Guacamole の URL を修正
1.1.18	2023.01.27	1.6	利用区分の説明を修正
1.1.19	2023.02.22	2.7.2, 2.8.2 2.8.1.2.4 2.8.3 2.9	各 Web サービスにアクセスできない場合の注意事項を追記 vncserver の起動ホスト確認方法を追記 vncserver の起動ホストへのアクセス方法を追記 SSH ポートフォワードによる Web サービスへのアクセス方法を追記

1.1.20	2023.04.05	5.2.1 5.7.1.2 5.3.4	表 5-4、表 5-5、表 5-28 のリソースグループの割り当てノードを修正 Odyssey 用 FEFS クライアントパラメータの変更オプションを追加
1.1.21	2023.06.23	5.2.1	表 5-4 ジョブあたりのバルクサブジョブ数を追加
1.1.22	2023.09.22	4.7.11	KSP の tutorials/ex1.c のコンパイル例を修正
1.1.23	2024.04.03	2.8	vncserver のデスクトップ環境の表示を変更
1.1.24	2024.05.24	5.3.2	スタックサイズの仕様を追記

目次

はじめに	1
1. Wisteria/BDEC-01 システムの概要.....	2
1.1. システム構成	2
1.2. ハードウェア概要.....	3
1.3. ソフトウェア構成	5
1.4. サービス時間	7
1.5. システム利用概要	7
1.6. 利用区分	8
2. Wisteria/BDEC-01 システムへのアクセス	10
2.1. アカウントと認証方式	10
2.2. 接続先アドレス	10
2.3. システム初回ログイン時の設定	11
2.4. SSH ログイン	20
2.5. ファイル転送	23
2.6. ログイン環境	26
2.7. JupyterHub	30
2.8. Apache Guacamole	35
2.9. SSH ポートフォワードを使用した Web サービスへのアクセス方法	42
3. ファイルシステム環境.....	43
3.1. ファイルシステムの概要	43
3.2. 共有ファイルシステム	48
3.3. ディスク使用状況と上限値の表示.....	52
4. コンパイル/リンク.....	53
4.1. 環境設定	53
4.2. Fujitsu Development Studio のコンパイル/リンク概要	55
4.3. Intel OneAPI のコンパイル/リンク概要	62
4.4. NVIDIA HPC SDK のコンパイル/リンク概要	65
4.5. NVIDIA CUDA SDK のコンパイル/リンク概要	69
4.6. GCC のコンパイル/リンク概要	71
4.7. ライブラリ	73
5. バッチジョブ・インタラクティブジョブ	92
5.1. ジョブ管理システム概要	92
5.2. ジョブ実行リソース	93
5.3. ジョブ投入オプション.....	96
5.4. バッチジョブ実行	104

5.5.	インタラクティブジョブ実行	132
5.6.	MPI プログラムの実行	135
5.7.	高度なジョブ実行	153
5.8.	ジョブ状態表示	159
5.9.	ジョブキャンセル	168
5.10.	ジョブ保留	169
5.11.	ジョブ保留の解除	169
5.12.	ジョブ履歴の参照	170
5.13.	計算ノードから外部リソースへのアクセス	171
6.	開発ツール	172
6.1.	開発ツールの概要	172
6.2.	Odyssey 向け開発ツール	172
6.3.	Aquarius 向け開発ツール	186
7.	マニュアル参照	192
8.	アプリケーション	193
9.	機械学習ライブラリ	195
9.1.	PyTorch の利用方法	195
9.2.	TensorFlow の利用方法	197
10.	コンテナ環境	202
10.1.	Singularity の利用方法	202

はじめに

本利用手引書は、東京大学情報基盤センターが運用するスーパーコンピュータシステム Wisteria/BDEC-01 システムの利用方法について説明した資料です。システムを利用する方は、必ずお読みください。

本利用手引書の内容は、不定期に更新いたします。最新の利用手引書は、利用支援ポータルより入手することが可能です。

<https://wisteria-www.cc.u-tokyo.ac.jp/>

本利用手引書に記載しているシェルスクリプト等は、Wisteria/BDEC-01 システム ログインノードの以下のディレクトリに格納していますので、併せてご利用ください。

/usr/local/example 配下

Wisteria/BDEC-01 システムの運用状況は、東京大学情報基盤センターの下記 URL や利用支援ポータル等でアナウンスします。

<https://www.cc.u-tokyo.ac.jp/>
<https://www.cc.u-tokyo.ac.jp/supercomputer/schedule.php>

Wisteria/BDEC-01 システム利用に関する質問は Web ページまたはメールで受け付けています。質問の種類によって異なるメールアドレスを用意しておりますので下記 URL を参照ください。

<https://www.cc.u-tokyo.ac.jp/supports/contact/>

なお、よくあるご質問とその回答については、「FAQ」にまとめてあります。下記 URL を参照ください。

<https://www.cc.u-tokyo.ac.jp/faq/wisteria.php>

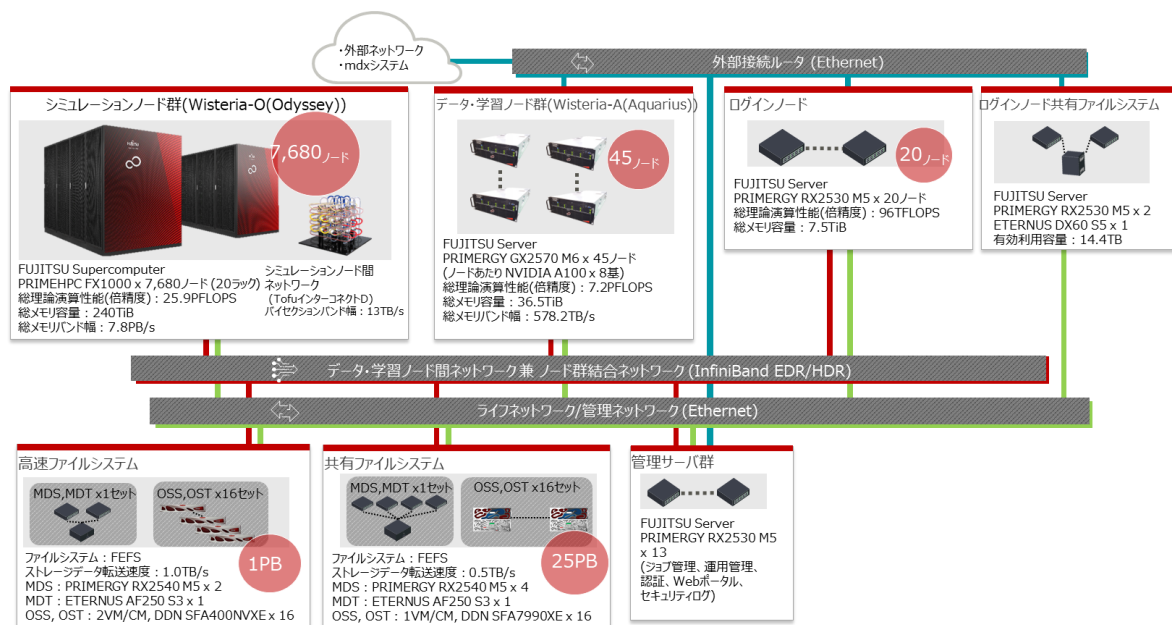
本書の一部、または全部を無断で複製、転載、再配布することを禁じます。

1. Wisteria/BDEC-01 システムの概要

1.1. システム構成

Wisteria/BDEC-01 システムは、シミュレーションノード群（Odyssey）とデータ・学習ノード群（Aquarius）の 2 つの計算ノード群を有したスーパーコンピュータシステムです。シミュレーションノード群（Odyssey）は、「富岳」と同型の CPU を搭載した大規模分散並列計算機「FUJITSU Supercomputer PRIMEHPC FX1000」で構成されます。データ・学習ノード群（Aquarius）は Intel 社の CPU と NVIDIA 社の GPU を搭載した「FUJITSU Server PRIMERGY GX2570 M6」で構成されます。

Wisteria/BDEC-01 システムの全体構成を図 1-1 に示します。



Wisteria-O(Odyssey) は 7,680 台の計算ノード「FUJITSU Supercomputer PRIMEHPC FX1000」で構成され、総理論演算性能(倍精度浮動小数点演算性能)は 25.9 PFLOPS です。FX1000 は高性能プロセッサ(A64FX)と高速メモリを採用し、1 ノードあたり 48 コア/CPU で 3.3TFLOPS(倍精度)の理論演算性能と 1,024GB/s の高いメモリバンド幅を有します。

Wisteria-A(Aquarius)は 45 台の計算ノード「FUJITSU Server PRIMERGY GX2570 M6」で構成され、総理論演算性能(倍精度浮動小数点演算性能)は 7.2 PFLOPS です。各ノードに NVIDIA A100 を 8 枚搭載し、複数 GPU 並列実行にも対応します。各ノードは InfiniBand HDR で接続され、ノード間通信とファイル共有を行ないます。

ストレージ環境は、高速ファイルシステムと共有ファイルシステムで構成します。高速ファイルシステムの有効利用容量は 1 PB、共有ファイルシステムの有効利用容量は 25PB です。

Wisteria/BDEC-01 システムへのアクセスは、ssh/scp による SSH アクセスとブラウザによる HTTPS アクセス(利用支援ポータル)が可能です。ユーザーはログインノード上にて、プログラムの編集、コンパイル/リンク、バッチ

ジョブの操作、インタラクティブジョブの実行等の作業を行うことが可能です。インタラクティブジョブでは、デバッグやチューニングが可能です。

Wisteria/BDEC-01 システムは、「大規模 HPC チャレンジ」サービスも提供しており、Odyssey の 6,144 ノード(294,912 コア)を最大 23 時間・1 研究グループで占有して利用できます。

1.2. ハードウェア概要

1.2.1. FUJITSU Supercomputer PRIMEHPC FX1000

Odyssey は、FUJITSU Supercomputer PRIMEHPC FX1000 で構成されます。「富岳」と同じアーキテクチャーを持つ、超大規模システム向けのスーパーコンピュータです。



CPU である「A64FX」は、ARM v8.2+SVE(Scalable Vector Extension:Arm の AArch64 の HPC 拡張命令セット)に準拠しノード性能は 3.3792 TFLOPS です。

計算ノード間ネットワークの「Tofu インターコネクト D」はトーラス構成による直接結合網です。リンク当たりのバンド幅は 6.8GB/s(双方向)であり、ノード同時データ性能は 40.8GB/s(双方向)です。

表 1-1 FUJITSU Supercomputer PRIMEHPC FX1000 の主な仕様

1 ノードあたりの構成		
CPU	プロセッサ	A64FX (Arm v8.2-A + SVE)
	プロセッサ数	1 プロセッサ
	演算コア数	48
	理論演算性能	3.3792 TFLOPS
主記憶	種別	HBM2
	容量	32GiB
	メモリバンド幅	1,024GB/s
インタコネクト	種別	Tofu インターコネクト D
	理論性能	最大 40.8GB/s x 双方向

1.2.2. FUJITSU Server PRIMERGY GX2570 M6

Aquariusは、FUJITSU Server PRIMERGY GX2570 M6で構成されます。8枚のNVIDIA社製GPUを搭載し、大規模な解析業務や人工知能/ディープラーニング用途に最適な 2WAY ラック型(4U)サーバです。



表 1-2 FUJITSU Server PRIMERGY GX2570 M6 の主な仕様

項目	説明
CPU	Intel® Xeon Platinum 8360Y (開発コード名 : Ice Lake) x 2
メモリ	512 GiB
GPU	NVIDIA A100 x 8
計算ノード間ネットワーク	InfiniBand HDR(200Gbps) x 4

表 1-3 Intel Xeon Platinum 8360Y の主な仕様

項目	説明
物理コア数	36
論理コア数	36 (ハイパースレッディングは無効化)
プロセッサ・ベース動作周波数	2.40 GHz
ターボ・ブースト利用時の最大周波数	3.50 GHz
キャッシュ	54 MB
命令セット拡張	Intel AVX-512
理論性能	2.40GHz x 32 演算 x 36 コア = 2.7648TFLOPS

1.3. ソフトウェア構成

Wisteria/BDEC-01 システムで利用可能なソフトウェア一覧を以下に示します。

表 1-4 ソフトウェア一覧

項目	Wisteria-O(Odyssey)	Wisteria-A(Aquarius)
OS	Red Hat Enterprise Linux 8	
コンパイラ	Fujitsu Development Studio (富士通社製コンパイラ) GCC	Intel OneAPI(インテルコンパイラ) NVIDIA HPC SDK NVIDIA CUDA SDK GCC
ライブラリ	富士通社製 MPI ライブラリ 富士通社製数値計算ライブラリ (BLAS, CBLAS, LAPACK, ScaLAPACK)	インテル MPI ライブラリ インテル MKL ライブラリ (BLAS, CBLAS, LAPACK, ScaLAPACK) NVIDIA 社製ライブラリ (cuBLAS, cuSPARSE, cuFFT, MAGMA, cuDNN, NCCL)
	HDF5, FFTW, METIS, MT_METIS, ParMETIS, NetCDF, Parallel netCDF, PETSc, SuperLU, SuperLU_MT, SuperLU_DIST, Scotch, PT-Scotch, Xabclib, GSL, ppOpen-HPC, Trilinos	
	Archiconda	Miniconda
デバッガ	富士通社製並列実行デバッガ	Arm DDT
ツール	富士通社製プロファイラ	インテル VTune プロファイラー インテル Advisor インテル Inspector インテル Trace Analyzer & Collector Arm MAP
アプリケーション	OpenFOAM, ABINIT-MP, PHASE, FrontFlow/blue, FrontISTR, REVOCAP, OpenMX, MODYLAS, GROMACS, BLAST, R Packages, Bioconductor, BioPerl, BioRuby, BWA, GATK, SAMtools, Quantum ESPRESSO, Xcrypt, ROOT, Geant4, LAMMPS, CP2K, NWChem, DeepVariant, ParaView, VisIt, POV-Ray, TensorFlow, Chainer, PyTorch, Keras, Horovod, MXNet	
ファイルシステム	FEFS	
ジョブ管理システム	Technical Computing Suite	
フリーソフトウェア	bash, tcsh, zsh, emacs, autoconf, automake, bzip2, cmake, cvs, gawk, gmake, gzip, perl, python, ruby, sed, tar, vim など	

Wisteria/BDEC-01 システムのオープンソースソフトウェアをご利用の際は、自己責任の範囲内でご活用ください。可能な範囲で動作確認は実施していますが、オープンソースソフトウェアのご利用方法等についてはお答えいたしかねますのでご了承ください。



Anaconda 社がサービス提供するリポジトリの利用規約が改定され、Anaconda の商用利用は有償になりました。

ただし、教育機関の学生または職員による教育活動での利用、非営利の研究機関による非商業的活動での利用等は継続して無償利用可能です。詳細については以下をご参照ください。

<https://www.anaconda.com/terms-of-ser>

可能であれば無償版である miniconda + conda forge の利用を推奨します。

1.4. サービス時間

Wisteria/BDEC-01 システムは原則 24 時間サービスを行っていますが、システム定期保守作業や設備保守作業（空調機メンテナンス・電源設備点検）などで、定期的にサービスを休止（システムを停止）します。また必要に応じて緊急保守を実施します。Wisteria/BDEC-01 システムの運用状況は、下記 URL でアナウンスします。

<https://www.cc.u-tokyo.ac.jp/supports/contact/>
<https://www.cc.u-tokyo.ac.jp/supercomputer/schedule.php>

1.5. システム利用概要

Wisteria/BDEC-01 システムでは、全ての計算ノードとログインノード、共有ファイルシステム(FEFS)を共有します。

ログインノードでは、SSH によるログイン後、コマンドの対話的実行が可能であり、プログラムの作成・編集、コンパイル/リンク、ジョブ投入・表示などを行います。なお、ログインノードの CPU やメモリなどの資源は多くのユーザーで共有しますので、高負荷な処理は行わないようにしてください。高負荷な前処理、後処理を行う場合は、プリポストノードを利用してください。

計算ノード向けのプログラムを実行する場合、バッチジョブもしくはインタラクティブジョブとしてジョブ管理システムに処理を依頼します。インタラクティブジョブは、プログラムのデバッグや会話的なアプリケーション、可視化ソフトウェアの実行に使用します。



ログインノードで使用できるメモリ量は、ユーザーあたり 24GB です。使用メモリ量が合計で 24GB を越えた場合、使用メモリ量の大きいプロセスから順に強制終了されます。

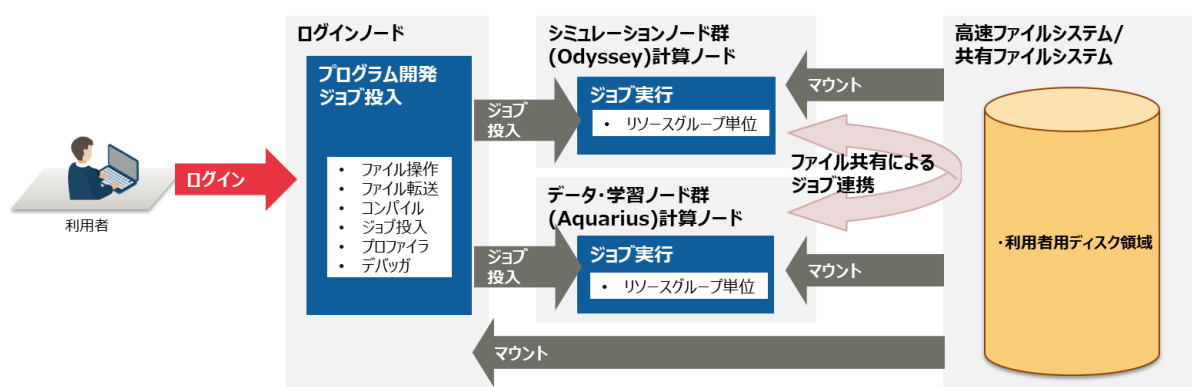


図 1-2 システム利用方法

1.6. 利用区分

Wisteria/BDEC-01 システムでは、以下に示す利用区分を設けています。ユーザーはいずれかの区分で Wisteria/BDEC-01 システム利用申請を行います。申請が許可された後、Wisteria/BDEC-01 システムが利用可能になります。

- 通常利用（一般・トライアル）
- 企業利用（一般・トライアル）
- 教育利用
- 若手・女性利用
- 萌芽共同研究公募課題
- 大規模 HPC チャレンジ
- お試しスパコン利用（無料体験）

各利用区分の詳細や申請方法は、下記 URL を参照してください。

<https://www.cc.u-tokyo.ac.jp/guide/>

1.6.1. トークン

利用するコースと申込ノード数に応じて、計算ノードのノード時間積であるトークンが割り当てられます。トークンを消費するリソースグループでジョブを実行すると、ジョブ毎(ステップジョブ、バルクジョブの場合はサブジョブ毎)に以下の計算式でトークンを消費します。

- Odyssey
トークン = 要求ノード数 × 経過時間 × トークン消費係数
- Aquarius
トークン = 要求 GPU 数 × 経過時間 × トークン消費係数

表 1-5 トークン消費係数

システム名	消費係数	備考
Odyssey	1.0	
	1.5	優先利用リソースグループを使用する場合
Aquarius	3.0	

割り当てトークン量や消費済みトークン量を確認するには、`show_token` コマンドを利用します。

トークン消費率が 100%を超えた場合、トークンを消費するリソースグループへの新規ジョブ投入が行えなくなります。

また、定期的にトークン消費率の状況についてチェックが行われ、100%を超えた場合、投入済みジョブの実行が保留されます（実行中ジョブは影響を受けません）。

ジョブの保留処理の実行にはタイムラグがあるため、トークン消費率が 100%を超えた後もジョブの実行が開始される場合があります。

```
[username@wisteria01 ~]$ show_token
```

Project	User	Limit(hours)	Used(hours)	Used%	Max-node(O)	Max-node(A)	Max-gpu	Expiry-date
projcode	-	1,388,888	27,196.0	1%	7,680	45	-	2025/03/31
	username	2,400,000	0.0	0%	1,024	4	-	2025/03/31

表 1-6 show_token 表示項目

項目	説明
Project	プロジェクトコード
User	ユーザー名
Limit(hours)	トークン上限値（単位：時間）
Used(hours)	トークン消費値（単位：時間）
Used%	トークン消費率
Max-node(O)	1 ジョブあたりの最大利用可能ノード数（Odyssey）
Max-node(A)	1 ジョブあたりの最大利用可能ノード数（Aquarius）
Max-GPU	1 ジョブあたりの最大利用可能 GPU 数（Aquarius）
Expiry-date	トークン有効期限

2. Wisteria/BDEC-01 システムへのアクセス

2.1. アカウントと認証方式

Wisteria/BDEC-01 システムへのアクセスに使用するユーザー名は、申込時に通知されるユーザー名です。通知されるパスワードは利用支援ポータルにログインするためのパスワードになります。

Wisteria/BDEC-01 システムを初めて利用する場合、ユーザー名とパスワードを用いて利用支援ポータルにログインし、SSH 公開鍵を登録してください。Wisteria/BDEC-01 への SSH アクセス(ssh/scp)は、SSH version 2 の公開鍵認証および GSI 認証のみ許可しています。

表 2-1 Wisteria/BDEC-01 システムへのアクセス方法と認証

サービス	アクセス	認証
仮想端末	ssh	公開鍵認証(ユーザーが設定したパスフレーズ) GSI 認証(HPCI 利用)
ファイル転送	scp, sftp	
利用支援ポータル	https	LDAP 認証

2.2. 接続先アドレス

Wisteria/BDEC-01 システムがサービスを提供するノードで、ユーザーが外部から直接アクセスできるノードは、ログインノードと利用支援 WEB ノードのみです。接続先アドレスを表 2-2 に示します。

Wisteria/BDEC-01 システムのログインノードは 6 台のノードで構成されており、wisteria.cc.u-tokyo.ac.jp にアクセスすると、DNS ラウンドロビンにより、いずれかのログインノードにアクセスすることができます。利用支援ポータル Web ノードは、利用支援ポータルサービスを提供しており、SSH 公開鍵登録やマニュアル参照等が可能です。

表 2-2 Wisteria/BDEC-01 接続先一覧

ホスト名	プロトコル	用途
wisteria.cc.u-tokyo.ac.jp	ssh	スーパーコンピュータシステム ● 仮想端末 ● ファイル転送
wisteria01.cc.u-tokyo.ac.jp		
wisteria02.cc.u-tokyo.ac.jp		
wisteria03.cc.u-tokyo.ac.jp		
wisteria04.cc.u-tokyo.ac.jp		
wisteria05.cc.u-tokyo.ac.jp		
wisteria06.cc.u-tokyo.ac.jp		
wisteria-www.cc.u-tokyo.ac.jp	https	利用支援ポータル ● ドキュメント閲覧 ● パスワード変更 ● SSH 公開鍵登録

2.3. システム初回ログイン時の設定

Wisteria/BDEC-01 システムへの初回ログインまでの手順は以下のとおりです。

1. ssh 秘密鍵/公開鍵ペアを作成する (2.3.1)
2. 利用支援ポータルにログインし、初期パスワードを変更する (2.3.2)
3. 利用支援ポータル上から公開鍵を登録する (2.3.3)
4. 利用支援ポータル上からメール転送を設定する。(2.3.4)
5. ssh クライアントを使用して Wisteria/BDEC-01 システムへログインする (2.4)

2.3.1. ssh 秘密鍵/公開鍵ペアの作成

ssh 秘密鍵/公開鍵ペアは、作成済みの鍵を使用することも可能です。

Windows 環境で公開鍵認証方式用の公開鍵/秘密鍵のペアを作成する場合は「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」を、UNIX/Mac 環境で作成する場合は「2.3.1.2 ssh 秘密鍵/公開鍵ペアの作成(UNIX/Mac 環境)」を参照してください。

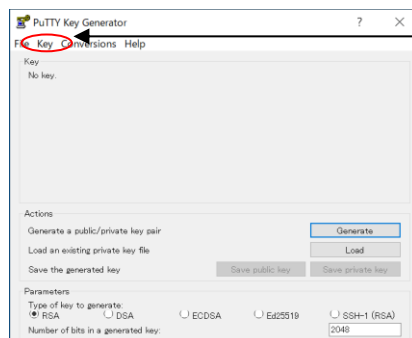
2.3.1.1. ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)

PC にて、ssh 秘密鍵/公開鍵ペアを作成します。鍵の作成は PuTTY をインストールした際に付属する PuTTYgen を用いる方法と OpenSSH の `ssh-keygen` コマンドを用いる方法があります。すでに鍵を作成済みの場合、作業を行う必要はありません。

2.3.1.1.1. PuTTYgen を用いた ssh 秘密鍵/公開鍵ペアの作成方法

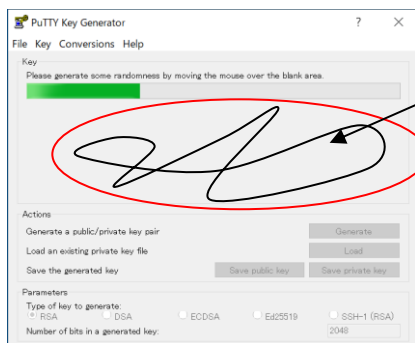
PuTTY は以下のサイトからダウンロードすることができます。

PuTTY: <http://www.chiark.greenend.org.uk/~sgtatham/putty/>



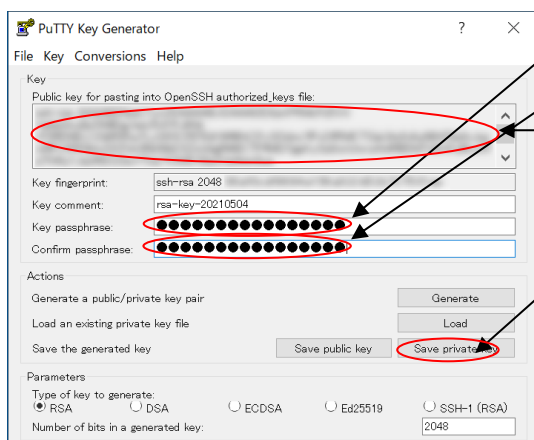
1. [Key]をクリック
2. 表示されるメニュー中の[Generate key pair]をクリック

図 2-1 PuTTY での秘密鍵/公開鍵作成画面 1



3. 鍵作成のための乱数を生成するため、マウスを動かす

図 2-2 PuTTY での秘密鍵/公開鍵作成画面 2



4. パスフレーズを入力
5. 再度パスフレーズを入力
6. 表示された公開鍵内容をコピーし、「2.3.3.2 公開鍵登録（直接入力）」で登録
もしくは、テキストエディタ等でファイル
7. [Save private key]をクリック

図 2-3 PuTTY での秘密鍵/公開鍵作成画面 3



セキュリティ向上のため、パスフレーズは必ず入力してください。

2.3.1.1.2. ssh-keygen コマンドを用いた ssh 秘密鍵/公開鍵ペアの作成方法

Windows PowerShell を起動して *ssh-keygen* コマンドを実行します。

```
PS C:¥Users¥username> ssh-keygen ← ssh-keygen コマンドを入力
Generating public/private rsa key pair.
Enter file in which to save the key (C:¥Users¥username/.ssh/id_rsa): ← リターンを入力
Created directory 'C:¥Users¥username/.ssh'. ← パスフレーズを入力
Enter passphrase (empty for no passphrase): ← 再度パスフレーズを入力
Enter same passphrase again:
Your identification has been saved in C:¥Users¥username/.ssh/id_rsa.
Your public key has been saved in C:¥Users¥username/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:XXXXXXXXXX username@machinename
The key's randomart image is:
+----[RSA 3072]-----+
|
|X X          X |
|X X          X |
|X X X      X X |
|X X X      X XX|
|  X X XX    XX |
|XXXX X  X  XX |
|XXX  X  X  X |
|XXXXXXXX  XX |
+----[SHA256]-----+
PS C:¥Users¥username>
```



セキュリティ向上のため、パスフレーズは必ず入力してください。

2.3.1.2. ssh 秘密鍵/公開鍵ペアの作成(UNIX/Mac 環境)

アクセス元端末にて、ssh 秘密鍵/公開鍵ペアを作成します。鍵の作成は `ssh-keygen` コマンドを実行します。すでに鍵を作成済みの場合、作業を行う必要はありません。

■ UNIX/Linux

端末エミュレータを起動して、`ssh-keygen` コマンドを実行します。

■ Mac

ターミナルを起動して、`ssh-keygen` コマンドを実行します。

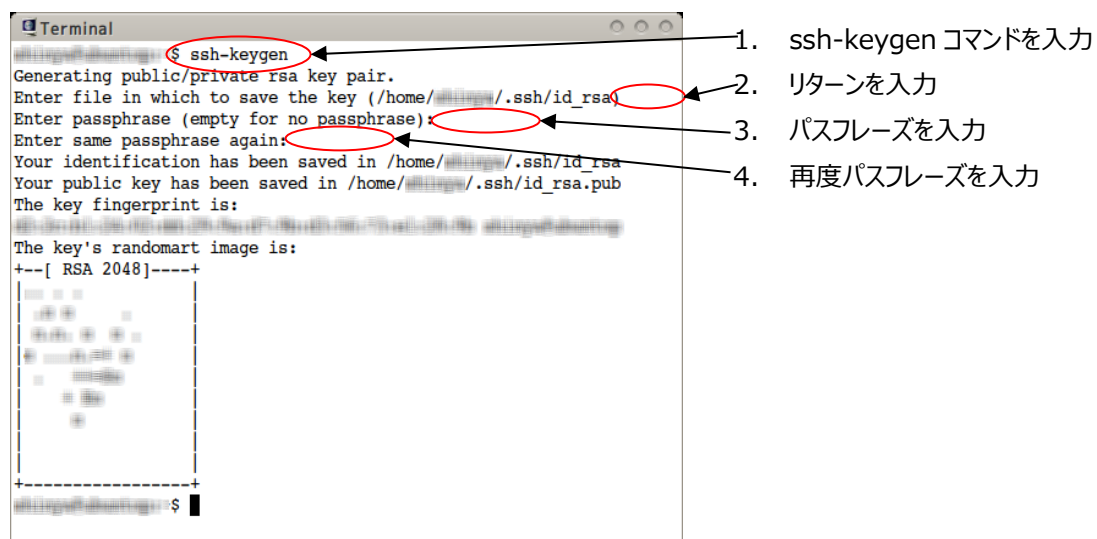


図 2-4 鍵ペアの作成



セキュリティ向上のため、パスフレーズは必ず入力してください。

2.3.2. 利用支援ポータルパスワード変更



利用支援ポータルの初期パスワードは必ず変更してください。初期パスワードを変更していない場合、利用支援ポータルにログインすると、パスワード変更画面に遷移します。

(1) ブラウザで下記 URL にアクセスします。

<https://wisteria-www.cc.u-tokyo.ac.jp/>

(2) 利用支援ポータルにログインします。※初回のパスワードは別途通知されます。

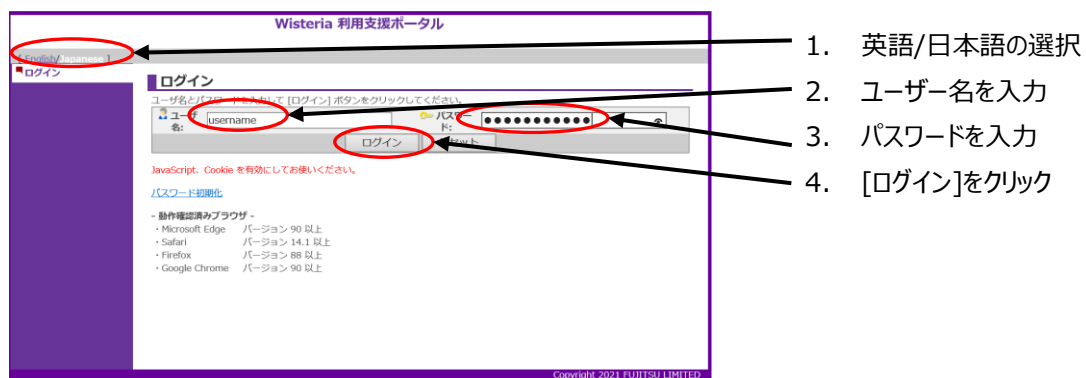


図 2-5 利用支援ポータルログイン画面

(3) [パスワード変更] メニューから、パスワードを変更します。

1. [パスワード変更]を選択

2. 現在のパスワードを入力

3. 新しいパスワードを入力

4. 確認のため再度入力

5. [変更]をクリック

新しいパスワードの条件：

- 8文字以上であること
- (a)-(d)を1文字以上含むこと
- (a) 英字小文字
- (b) 英字大文字
- (c) 数字
- (d) 特殊文字
- 現在のパスワードと3文字以上異なること
- Linux 辞書に無い単語であること

図 2-6 パスワード変更画面

(4) ポータル画面上に、エラーメッセージが表示されないことを確認します。

6.

利用支援ポータルログイン用パスワードを変更しました。

図 2-7 パスワード変更の確認

2.3.3. SSH 公開鍵登録

Wisteria/BDEC-01 システムでは、ssh 接続を行う前に利用支援ポータルで公開鍵登録を行う必要があります。公開鍵を登録するには、[公開鍵登録]メニューを選択します。

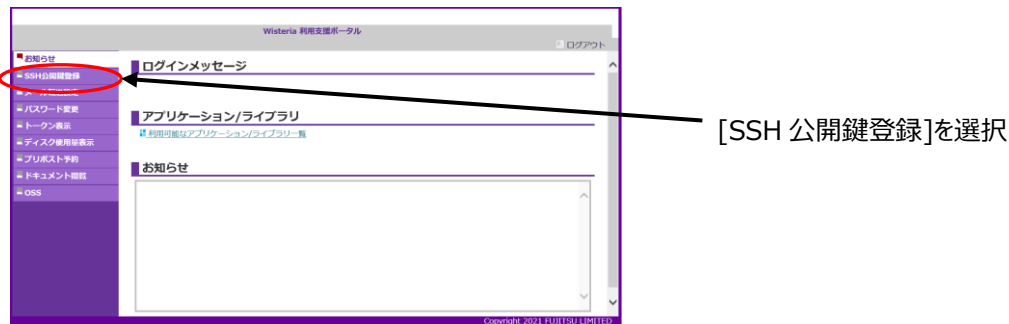


図 2-8 SSH 公開鍵登録画面への移動

公開鍵の登録は、作成した公開鍵ファイルをアップロードする方法（2.3.3.1 公開鍵登録（ファイルアップロード））と、公開鍵の内容をブラウザ内のテキストボックスに貼り付ける方法（2.3.3.2 公開鍵登録（直接入力））があります。公開鍵は複数登録可能です。

2.3.3.1. 公開鍵登録（ファイルアップロード）

(1) 公開鍵ファイルを指定します。

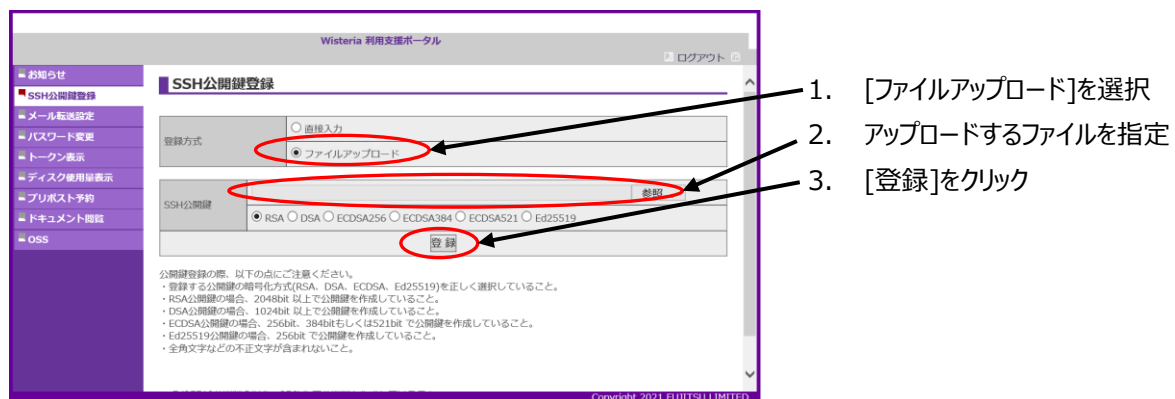


図 2-9 公開鍵ファイルのアップロード

- (2) ポータル画面上にエラーメッセージが表示されないことを確認します。



図 2-10 公開鍵登録の確認

2.3.3.2. 公開鍵登録（直接入力）

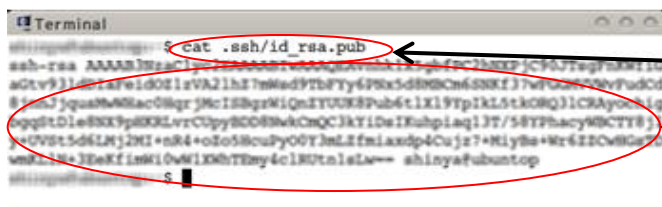
- (1) 公開鍵の内容を表示し、コピーします。

Mac: Terminal を起動して、cat コマンドで公開鍵ファイルの内容を表示します。

UNIX/Linux: 端末エミュレータを起動し、cat コマンドで公開鍵ファイルの内容を表示します。



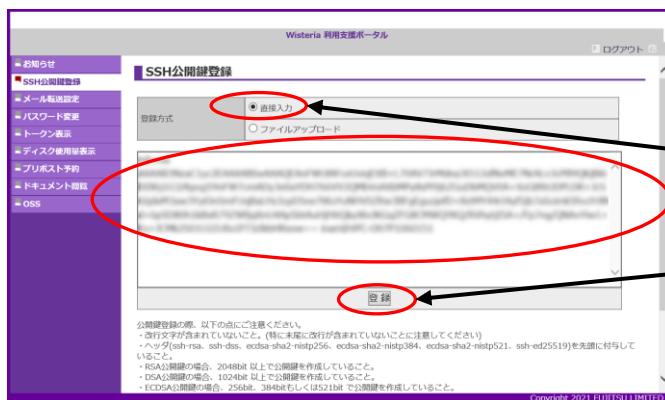
公開鍵ファイルは、デフォルトでは ~/.ssh/id_rsa.pub です。



1. 公開鍵の内容を表示
\$cat “公開鍵ファイル”
2. 内容をコピー

図 2-11 公開鍵の内容をコピー

- (2) 公開鍵の内容を貼り付けます。



1. [直接入力]を選択
2. 公開鍵を貼り付け
3. [登録]をクリック

図 2-12 公開鍵の登録

(3) ポータル画面上にエラーメッセージが表示されないことを確認します。



図 2-13 公開鍵登録の確認

2.3.3.3. 公開鍵削除

(1) 削除する公開鍵を指定します。

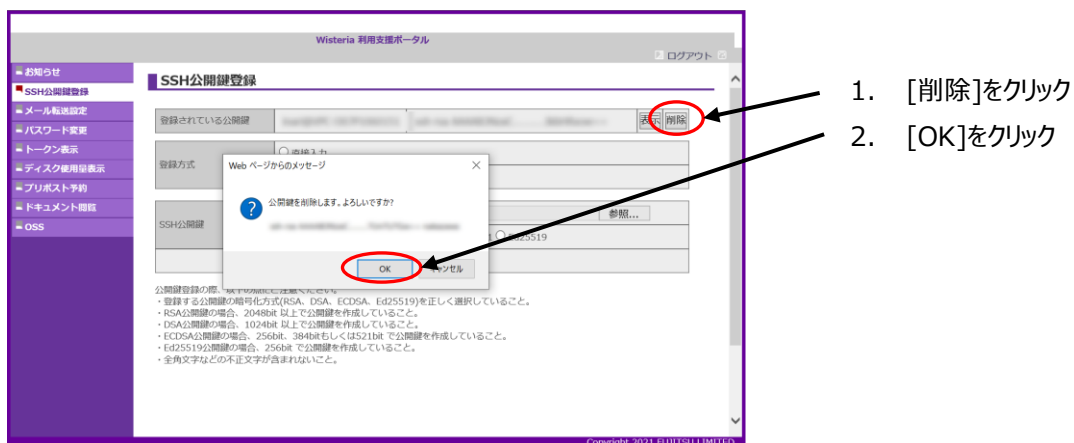


図 2-14 公開鍵の削除

2.3.4. メール転送設定

ジョブの開始時や終了時にメールを通知することができます。通知するメールアドレスを設定/変更するには、[メール転送設定]メニューを選択します。

(1) メールアドレスを設定します。

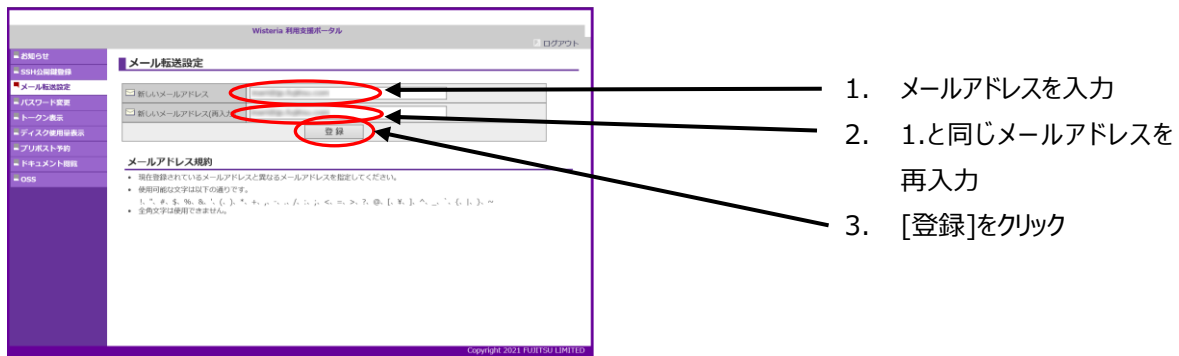


図 2-15 メール転送設定

(2) ポータル画面上にエラーメッセージが表示されないことを確認します。



図 2-16 メール転送設定の確認

2.4. SSH ログイン

2.4.1. SSH ログイン(Windows 環境)

Windows では様々なターミナルソフトが利用できますが、ここでは PuTTY を用いた SSH 接続方法と Windows PowerShell を用いた SSH 接続方法を説明します。Cygwin を使用される方は「2.4.2 SSH ログイン(UNIX/Mac 環境)」を参照してください。

2.4.1.1. PuTTY を用いた SSH 接続方法

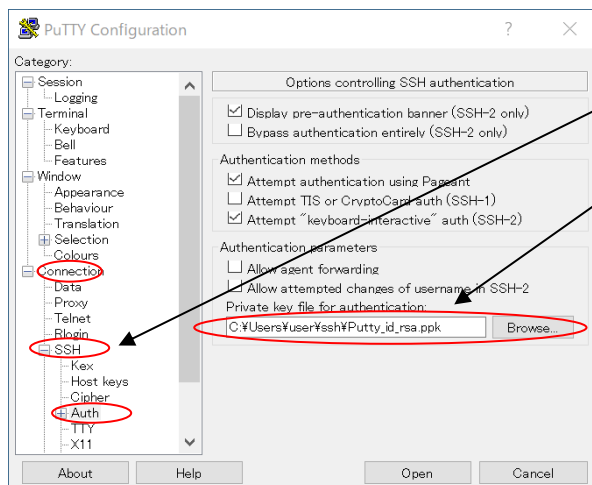
PuTTY は以下のサイトからダウンロードすることができます。

PuTTY: <http://www.chiark.greenend.org.uk/~sgtatham/putty/>



PuTTY のバージョンが古い場合、セキュリティ上の理由でログインに失敗することがあります。その場合は、最新版にアップデートしてください。

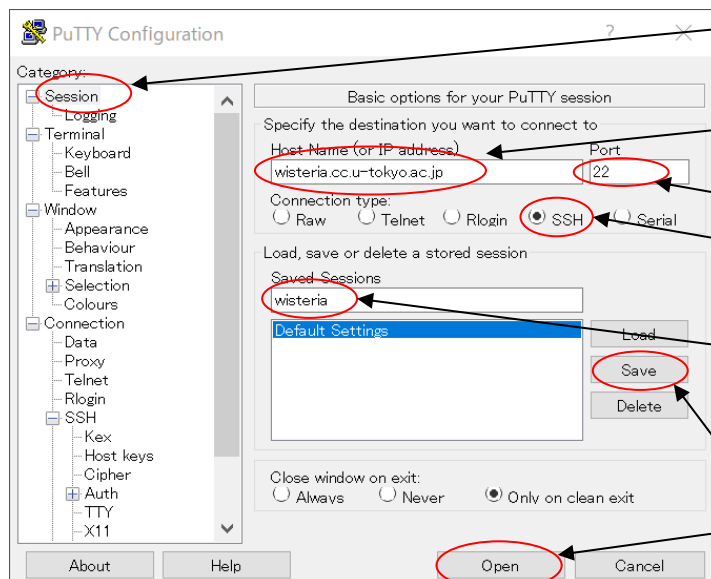
- (1) PuTTY を起動し、「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」で作成した秘密鍵ファイル(.ppk ファイル)を指定します。



1. [接続(Connection)] - [SSH] - [認証(Auth)] メニューを選択
2. [参照(Browse)] をクリックし、登録した公開鍵と対となる秘密鍵ファイルを指定

図 2-17 PuTTY での秘密鍵指定画面

- (2) PuTTY から Wisteria/BDEC-01 システムにアクセスします。



1. セッション画面を開く
2. 下記情報を入力
[ホスト名(Host Name)]
wisteria.cc.u-tokyo.ac.jp
[ポート(Port)] 22
[接続タイプ(Connection type)]
SSH
3. [セッション一覧 (Saved Sessions)] に session 名 (例: wisteria)を入力
4. [保存(Save)] をクリック
5. [開く(Open)] をクリック

図 2-18 PuTTY セッション画面

ユーザー名と「2.3.1.1 ssh 秘密鍵/公開鍵ペアの作成(Windows 環境)」にて入力したパスフレーズを入力します。

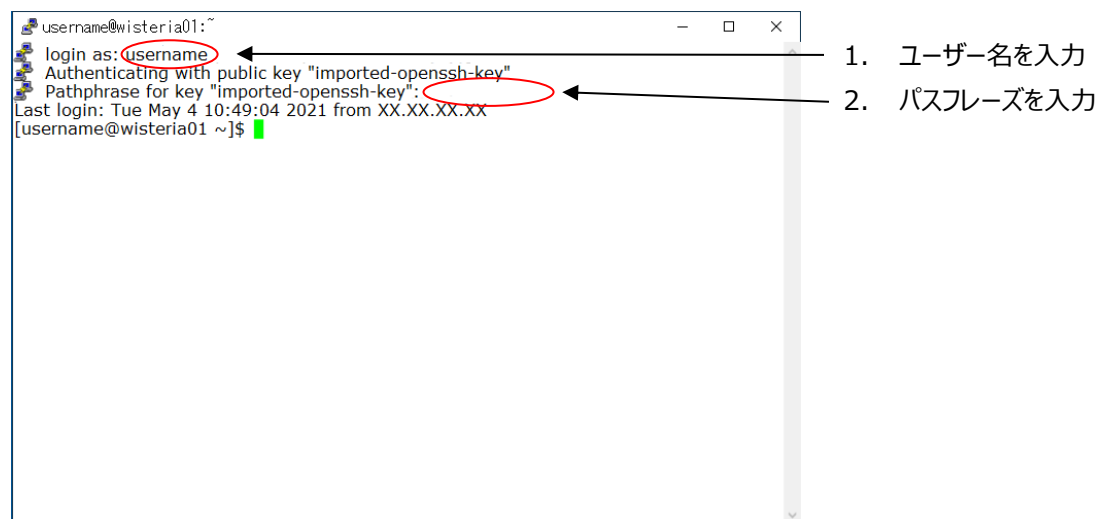


図 2-19 PuTTY ログイン完了画面

2.4.1.2. Windows PowerShell を用いた SSH 接続方法

Windows PowerShell から Wisteria/BDEC-01 システムへログインする場合は、ssh コマンドを利用します。

```
PS C:\Users\username> ssh -l username wisteria.cc.u-tokyo.ac.jp ← ssh コマンドを入力
Enter passphrase for key 'C:\Users\username\.ssh\id_rsa': ← パスフレーズを入力
Last login: Thu Oct 6 11:51:58 2022 from XXX.XXX.XXX.XX
[username@wisteria01 ~]$
```

2.4.2. SSH ログイン(UNIX/Mac 環境)

アクセス元端末から Wisteria/BDEC-01 システムへログインする場合は、ssh コマンドを利用します。

```
$ ssh -l username wisteria.cc.u-tokyo.ac.jp
The authenticity of host '
wisteria.cc.u-tokyo.ac.jp (XX.XX.XX.XX)' can't be established. ← 初回ログイン時のみ表示される。
RSA key fingerprint is XX:XX:XX:XX:XX:XX:XX:XX:XX:XX
Are you sure you want to continue connecting (yes/no)? ← yes を入力
Warning: Permanently added 'XX.XX.XX.XX' (RSA) to the list of known hosts.
Enter passphrase for key '/home/username/.ssh/id_rsa': ← パスフレーズ入力
[username@wisteria01 ~]$
```

2.5. ファイル転送

2.5.1. ファイル転送(Windows 環境)

2.5.1.1. WinSCP を利用する場合

PCと Wisteria/BDEC-01 システム間のファイル転送について、WinSCP によるファイル転送方法を説明します。WinSCP は以下のサイトから入手できます。

WinSCP: <http://winscp.net/eng/docs/lang:jp>

WinSCP は転送するファイルをドラッグ & ドロップすることでファイル転送することが可能です。

(1) WinSCP を起動して、ログインします。

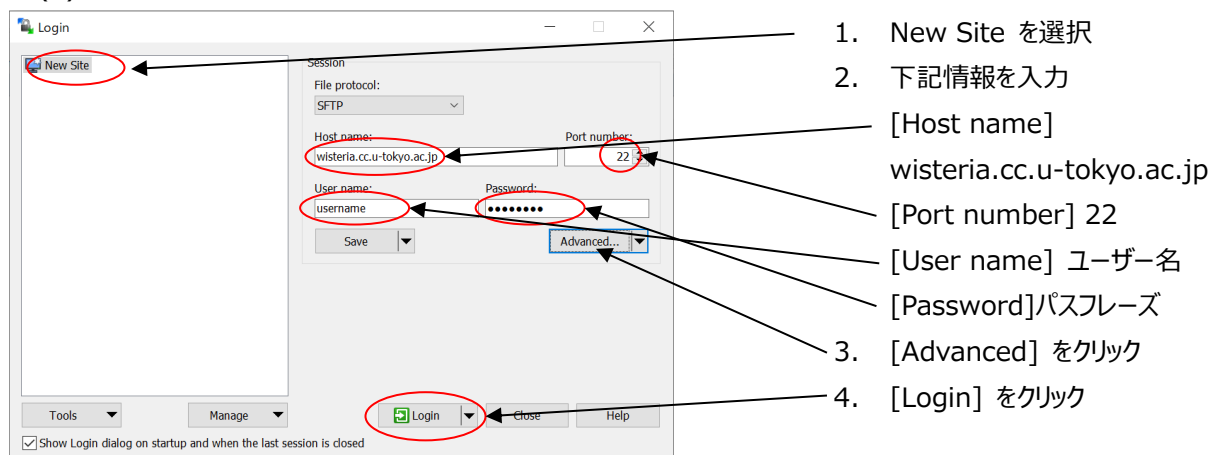


図 2-20 WinSCP Login 画面

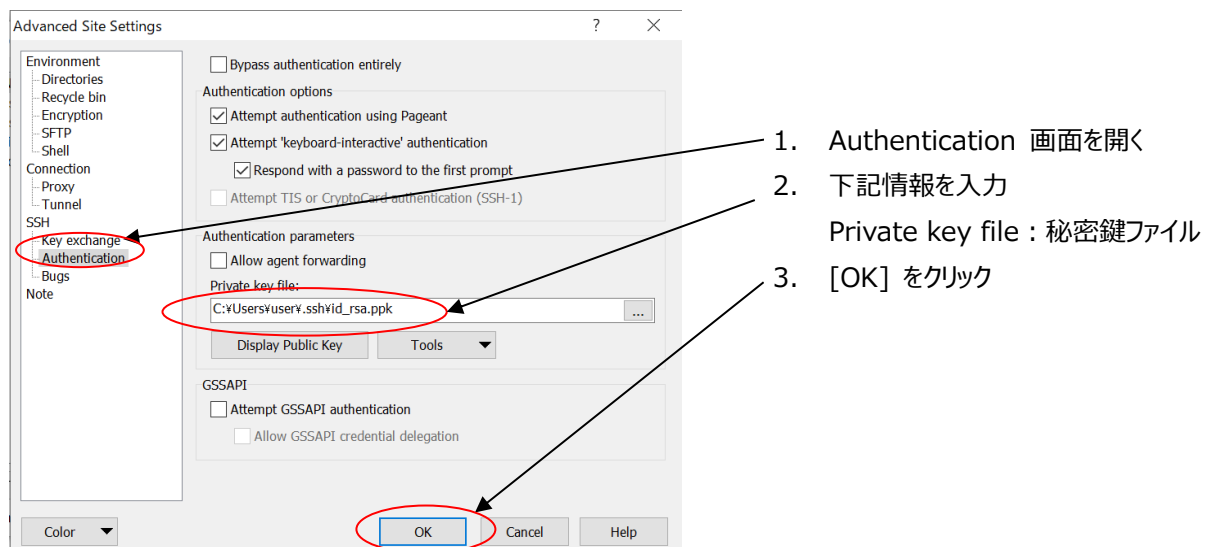


図 2-21 WinSCP Settings 画面

(2) ドラッグ＆ドロップでファイル転送を行います。

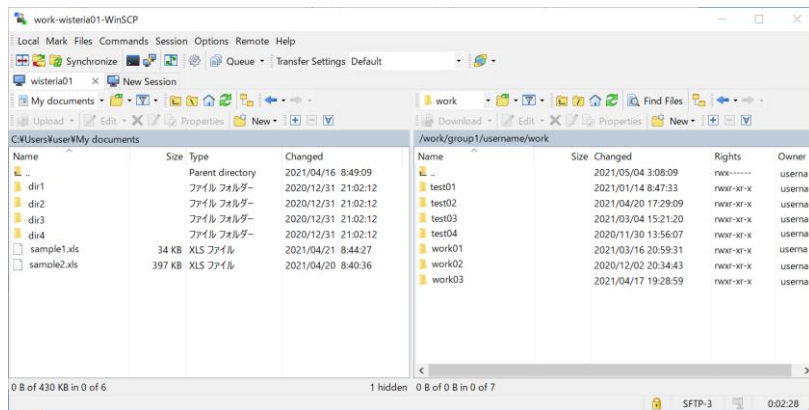


図 2-22 WinSCP ログイン後の画面

2.5.1.2. rsync コマンドを利用する場合

WSL(Windows Subsystem for Linux)を使用して Linux 環境を構築し、*rsync* コマンドによりファイル転送を行う方法を説明します。以下では Linux のディストリビューションとして Ubuntu を使用しています。

(1) Windows PowerShell を起動して、*wsl --install* コマンドにより Linux 環境をインストールします。

※ *wsl --list --online* と入力すると、利用可能な Linux ディストリビューションを一覧表示できます。

```
PS C:\Users\username> wsl --install -d Ubuntu
```

ダウンロード中: Ubuntu

インストール中: Ubuntu

Ubuntu はインストールされました。

Ubuntu を起動しています...

(2) インストール後、Windows PowerShell のターミナルとは別に新規に WSL のターミナルが起動されます。起動された WSL ターミナルにて Linux 環境で利用するユーザ名/パスワードを入力します（ここで入力したユーザ名/パスワードは Linux 環境でのみ利用されます）。

Installing, this may take a few minutes...

Please create a default UNIX user account. The username does not need to match your Windows username.

For more information visit: <https://aka.ms/wslusers>

Enter new UNIX username: XXXX ← ユーザ名を入力

New password: ++++++++ ← パスワードを入力

Retype new UNIX password: ++++++++ ← パスワードを再入力

passwd: password updated successfully

Installation successful!

...

(3) *exit* コマンドで WSL ターミナルを閉じます（WSL ターミナル上でも以降の操作は可能です）。

(4) Windows PowerShell 上で *ws/* コマンドを実行し、構築した Linux 環境にログインします。

```
PS C:¥Users¥username> wsl -d Ubuntu
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
$
```

(5) *rsync* コマンドを実行します。以下は 2.3.1.1.2 項で PC に保存した秘密鍵を使用する場合の例です。

```
$ rsync -e "ssh -i /mnt/c/Users/username/.ssh/id_rsa" local-file ¥
username@wisteria.cc.u-tokyo.ac.jp:remote-dir
sending incremental file list

sent XX bytes  received XX bytes  XX.XX bytes/sec
total size is XX  speedup is X.XX
```

WSL の Linux 環境から PC のファイルにアクセスする場合は、*wslpath* コマンドを使用して変換されたパスを指定する必要があります。上記 *rsync* コマンドの実行例では、以下のように表示されたパスを指定しています。

```
$ wslpath "C:¥Users¥username¥.ssh¥id_rsa"
/mnt/c/Users/username/.ssh/id_rsa
```

(6) WSL の Linux 環境からログアウトする場合は *exit* コマンドを実行します。

2.5.2. ファイル転送(UNIX/Mac 環境)

UNIX 系/Mac 系の PC と Wisteria/BDEC-01 システム間でファイル転送をする場合、*scp(sftp)* コマンドを利用します。

```
$ scp local-file username@wisteria.cc.u-tokyo.ac.jp:remote-dir
Enter passphrase for key : ++++++++ ← パスフレーズを入力

local-file 100% |*****| file-size transfer-time
```

2.6. ログイン環境

2.6.1. ログインシェル

Wisteria/BDEC-01 システムのデフォルトログインシェルは、bash が設定されています。ログインシェルは `chsh` コマンドを使用して `tcsh` もしくは `zsh` に変更することができます。ログインシェルの変更は次回ログインから有効となります。また、ログインシェルの反映には 10 分程度時間がかかります。

```
$ chsh [option] <new_shell>
```

表 2-3 chsh コマンドオプション一覧

オプション	説明
-l	利用可能なログインシェル一覧を表示する [/bin/bash, /bin/tcsh, /bin/zsh]
-s	ログインシェルを変更する

例) ログインシェルを tcsh に変更する。

```
[username@wisteria01 ~]$ chsh -s /bin/tcsh
Enter LDAP Password:          ← 利用支援ポータルパスワードを入力します
modifying                    entry
"uid=username,ou=People,dc=wisteria-bdec01,dc=cc,dc=u-tokyo,dc=ac,dc=jp"
```

Wisteria/BDEC-01 システムへログインすると、Wisteria/BDEC-01 システムを利用するための環境設定が自動で設定されます。環境変数 `PATH` や環境変数 `LD_LIBRARY_PATH` 等をカスタマイズする場合は、表 2-4 に示す個人用設定ファイルに設定してください。

表 2-4 個人用設定ファイル一覧

ログインシェル	個人用設定ファイル
bash	<code>\$HOME/.bash_profile</code>
tcsh	<code>\$HOME/.cshrc</code>
zsh	<code>\$HOME/.zshrc</code>



設定ファイルに環境変数 `PATH` にパスを追加する際は、環境変数 `PATH` の最後に追加してください。先頭に追加した場合、システムを正常に使用できなくなる恐れがあります。

個人用設定ファイルのオリジナル（雛形）は `/etc/skel` 配下に格納しています。

2.6.2. メール転送設定

ジョブの開始時や終了時にメールを通知することができます。通知するメールアドレスを設定/変更する場合は、利用支援ポータル「メール転送設定」を使用してください。



ログインノードからはメールを送信することはできません。

2.6.3. マスク値

新たに作成されるファイルやディレクトリのパーミッションはマスク値で設定することが可能です。マスク値のデフォルトは 0027 です。マスク値を変更する場合は、*umask* コマンドを使用してください。

例) 同じプロジェクトに所属するユーザーの読み込み・書き込み・実行を許可する場合

```
[username@wisteria01 ~]$ umask 0007
```



ホームディレクトリおよび、ssh ディレクトリ、公開鍵ファイル(*authorized_keys*)のパーミッションに他ユーザーに対する書き込み権があるとログインできなくなります。ご注意ください。

2.6.4. 実行パス/ライブラリ設定

ユーザー環境は、Environment Modules (<http://modules.sourceforge.net/>)を用いて設定されます。ユーザーは *module* コマンドを用いることで容易に環境を変更することが可能です。*module* コマンドは、アプリケーション実行やライブラリの利用時に必要となる環境変数 *PATH* や環境変数 *LD_LIBRARY_PATH* などを設定(load)・解除(unload)します。

```
$ module <sub-command>
```

module コマンドのサブコマンドを表 2-5 に示します。

表 2-5 module コマンドのサブコマンド

サブコマンド	説明
list	設定中のアプリケーション、ライブラリの一覧を表示します。
avail	設定可能なアプリケーション、ライブラリの一覧を表示します。
load <module>	アプリケーション、ライブラリ向けの環境変数を設定します。
unload <module>	アプリケーション、ライブラリ向けの環境変数を解除します。
switch <moduleA> <moduleB>	アプリケーション、ライブラリ向けの環境変数を moduleA から moduleB へ入れ替えます。
purge	設定中のすべてのアプリケーション、ライブラリを解除します。
help <module>	アプリケーション、ライブラリの使用方法を表示します。また、構築に使用したコンパイラのバージョンを表示します。

同一アプリケーションの複数バージョンの利用や、排他関係にあるアプリケーション、ライブラリは同時に利用することはできません。排他関係にあるアプリケーション、ライブラリを同時に利用しようとした場合、以下のメッセージが表示されます。

例) 排他関係にある言語処理系を利用した場合

```
[username@wisteria01 ~]$ module load gcc/8.3.1
WARNING: gcc/8.3.1 cannot be loaded due to a conflict.
HINT: Might try "module unload gcc" first.
```

利用中のアプリケーション、ライブラリを入れ替えるには、以下を実行してください。

例) gcc/8.3.1 を利用する場合(バージョン間で排他関係にあります)

```
[username@wisteria01 ~]$ module switch gcc/7.5.0 gcc/8.3.1
```

アプリケーション、ライブラリの構築されたバージョンを確認するには、以下を実行してください。

例) fftw/3.3.9 の構築バージョンを確認する場合

```
[username@wisteria01 ~]$ module help fftw/3.3.9
-----
Module Specific Help for /work/opt/local/modulefiles/LN/x86_64/compiler/intel/2021.2.0/fftw/3.3.9:

    'fftw/3.3.9' is built by intel/2021.2.0
-----
(以下略)
```

Wisteria/BDEC-01 システムで利用可能なアプリケーション/ライブラリの最新の情報については 2.6.5 利用可能なアプリケーション/ライブラリの一覧表示をご参照ください。

2.6.5. 利用可能なアプリケーション/ライブラリの一覧表示

Wisteria/BDEC-01 システムで利用可能なアプリケーション/ライブラリの一覧を表示するには、`show_module` コマンドを実行してください。

```
$ show_module [option]
```

`show_module` コマンドのオプションを表 2-6 に示します。

表 2-6 `show_module` コマンドのオプション一覧

オプション名	説明
なし	コンパイラ(Fujitsu, Intel, GCC)で利用可能なアプリケーション/ライブラリ、およびコンパイラに依存しないソフトウェアの一覧を表示 Odyssey の場合、デフォルトコンパイラで利用可能なソフトウェアの一覧を表示
-a	利用可能なすべてのアプリケーション/ライブラリの一覧を表示

例)すべてのアプリケーション/ライブラリの一覧を表示

```
[username@wisteria01 bin]$ show_module
```

ApplicationName	ModuleName	Node
BaseCompiler/MPI		
Archiconda	archiconda3/0.2.3	odyssey -
Arm Forge	forge/21.0.2	aquarius -
BLAST	blast/2.11.0	login,aquarius intel/2021.2.0
BLAST	blast/2.11.0	odyssey gcc/8.3.1
BWA	bwa/0.7.17	login,aquarius intel/2021.2.0
BioPerl	bioperl/1.7.8	login,aquarius -
BioPerl	bioperl/1.7.8	odyssey -
BioRuby	bioruby/2.0.2	login,aquarius -
BioRuby	bioruby/2.0.2	odyssey -

(snip)

`show_module` コマンドの表示項目を表 2-7 に示します。

表 2-7 `show_module` コマンドの表示項目

項目	説明
ApplicationName	アプリケーション・ライブラリ名、および説明
ModuleName	module avail で表示するアプリケーション/ライブラリ名
Node	アプリケーション・ライブラリを利用可能なノード (login : ログインノード、compute : 計算ノード)
BaseCompiler/MPI	アプリケーション・ライブラリを利用するためにロードが必要なコンパイラとバージョン、および依存ライブラリとバージョン

2.7. JupyterHub

Wisteria/BDEC-01 システムでは、PythonなどをWebブラウザ上で記述し、計算ノードで実行するためのJupyterHub環境を提供しています。

2.7.1. JupyterHub への接続

(1) ブラウザで下記 URL にアクセスします。

<https://wisteria08.cc.u-tokyo.ac.jp:8000/jupyterhub/>



Internet Explorer では正常にアクセスできない場合がありますので、それ以外のブラウザをお使いください。



利用されるネットワーク環境によっては、JupyterHub の URL にアクセスできない場合があります。アクセスできない場合は、「2.9 SSH ポートフォワードを使用した Web サービスへのアクセス方法」を参照してください。

(2) ログイン画面が表示されますので、Wisteria/BDEC-01 システムのユーザーID でログインします。

図 2-23 JupyterHub ログイン画面

ログインが正常終了すると初期画面が表示されます。

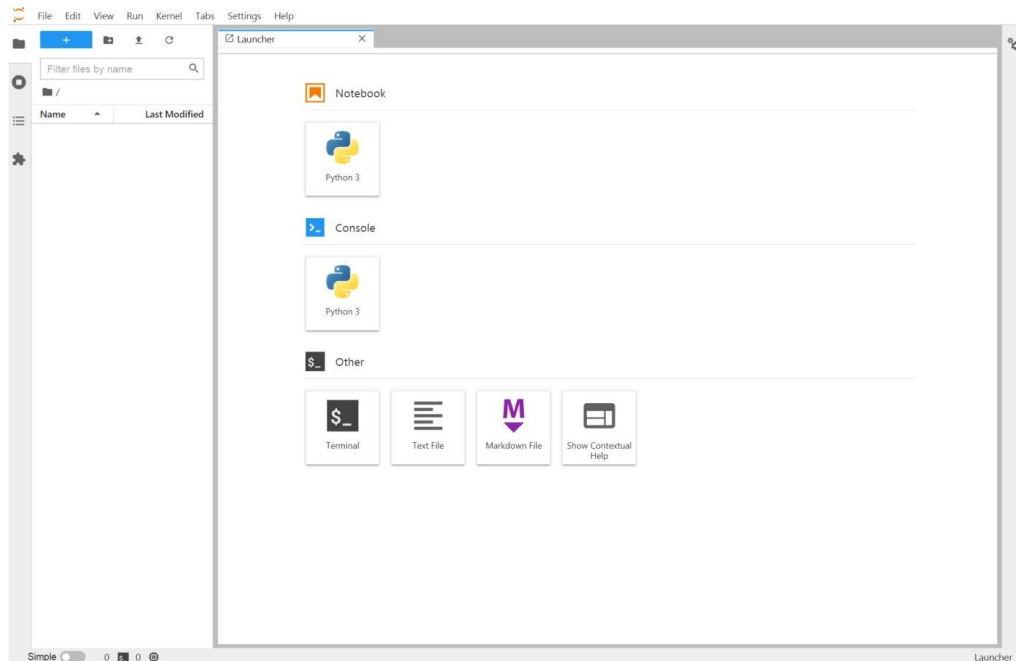


図 2-24 JupyterHub ログイン後の画面



Terminal を起動した場合、Safari のバージョンによってはコマンドが入力できない場合があります。

(3) Jupyter Notebook を起動します。

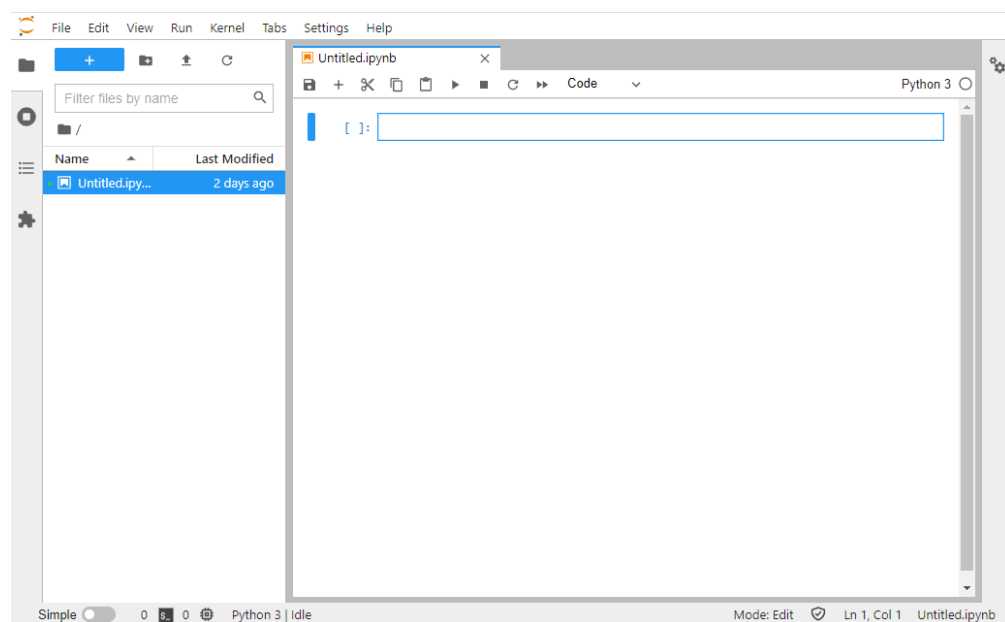


図 2-25 Jupyter Notebook 起動後の画面

2.7.2. Jupyter Notebook からのジョブ投入

Jupyter Notebook(拡張子.ipynb)上から psub コマンドを実行することで Jupyter Notebook の特定のライン番号内に記載されている Python コードをジョブスクリプトに記載してジョブ投入できます。

\$ psub [option] LINENO [,LINENO...][-LINENO]

表 2-8 psub コマンドオプション一覧

オプション名	説明
-f	Pythonコード取得対象となる Jupyter Notebook ファイル名、もしくは、Notebook ディレクトリ(\$HOME/.notebook)からの相対パスを含むファイル名を指定する。 本指定が省略された場合、環境変数\$PJM_NOTEBOOK_FILE(*1)で指定されたファイル名を有効とする。
--nosub	本オプションが指定された場合、ジョブスクリプト生成のみでジョブ投入しない。
-N	ジョブ名を指定する。
-g --gname	ジョブ実行時にトークンを消費するプロジェクトを指定する。
ライン番号	Python コード取得対象となるライン番号を指定する。 ライン番号の指定は、カンマ(,)による複数指定とハイフン(-)による範囲指定が可能。

(*1) 環境変数\$PJM_NOTEBOOK_FILE を使用する場合は、Jupyter Notebook で適宜設定ください。

Jupyter Notebook 上から psub コマンドを実行しジョブを投入する例を以下に示します。

psub コマンドは、システムで設定しているジョブ投入情報をもとに、ジョブスクリプトを作成しジョブを投入します。デフォルト設定では、リソースグループ regular-a に 1 ノードジョブを投入するようにジョブ投入情報が設定されています。ジョブ投入情報は、ジョブスクリプトのテンプレートファイル (\$HOME/.notebook/template_job.sh)を新規に作成することで変更可能です。以下は、テンプレートファイルを作成せずに、リソースグループ regular-a に 1 ノードを使用するジョブを投入する場合の例です。

(1) Jupyter Notebook 上でカレントディレクトリを/work 配下に移動します。

```
[1] !pwd
    /home/user
[2] %cd /work/group/user/job_dir
    /work/group/user/job_dir
```

(2) ジョブで実行する Python コードが記述された Jupyter Notebook ファイルとライン番号を指定して psub コマンドを実行します。

```
[3] import os
[4] print(os.getcwd())
[5] f = open('./tmp.txt', 'a')
    for i in range(0,100):
        print(i, file=f)
    f.close()
[6] !psub -f Notebook.ipynb 3,4,5
    [INFO] PJM 0000 psub Create script Notebook.sh.
    [INFO] PJM 0000 psub Job 12345 submitted.
```



Jupyter Notebook 上での編集内容は"\$HOME/.notebook/Notebook 名"ファイルに反映されるまでに一定期間を要する場合があります。psub で指定した python コードがジョブスクリプトに記載されない場合、Notebook のセルにライン番号が採番された後に 1 度 Notebook を保存してから psub コマンドを実行してください。

2.7.2.1. ジョブスクリプトのテンプレートファイル(\$HOME/.notebook/template_job.sh)の作成方法

ジョブを投入するリソースグループやノード数などジョブ投入情報をデフォルト設定から変更する場合は、ジョブスクリプトテンプレートファイルを新規に Console 等から作成します。本ファイルが存在しない場合、リソースグループ regular-a に 1 ノードジョブを投入します。

ジョブスクリプトテンプレートファイル：\$HOME/.notebook/template_job.sh

```
$ vi $HOME/.notebook/template_job.sh
#!/bin/sh
#PJM -L rscgrp=regular-a
#PJM -L node=1
```

2.7.3. JupyterHub からのログアウト

JupyterHub にログイン中に Web ブラウザを閉じてても状態は保持されますので、再度アクセスすると作業を再開させることができます。JupyterHub からログアウトする場合は、File メニューからログアウトを選択します。

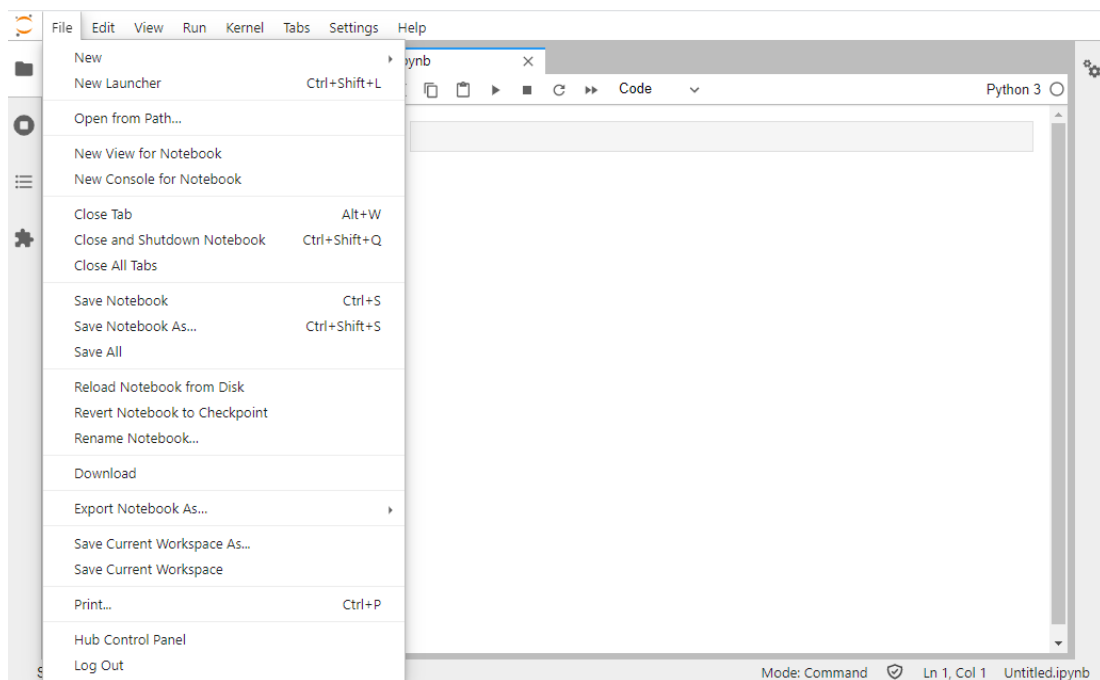


図 2-26 JupyterHub ログアウト画面

2.8. Apache Guacamole

Wisteria/BDEC-01 システムでは、Web ブラウザからログインノードへのリモートアクセスを可能とする Apache Guacamole 環境を提供しています。

2.8.1. vncserver の起動と設定

2.8.1.1. vncserver の起動

Apache Guacamole を使用する前に、ログインノード wisteria01～06 のいずれかにログインして、vncserver を起動します。起動手順を以下に示します。

(1) ログインノード上で以下のコマンドを実行し、vncserver を起動します。

初回起動時は、以下のように vncserver のパスワードを新規に設定します。

下記赤丸で表示されるディスプレイ番号に 5900 を加えた値が「起動ポート番号」となります。

起動ポート番号は(2)の set_vncserver コマンドで必要となります。

```
[username@wisteria01 ~]$ vncserver

You will require a password to access your desktops.

Password: ← パスワードを入力
Verify:    ← 再度パスワードを入力
Would you like to enter a view-only password (y/n)? n ← 画面閲覧モードのパスワード入力の有無
A view-only password is not used

New 'wisteria01:10 (username)' desktop is wisteria01:10
```

(2) 以下のコマンドを実行します。

※入力するパスワードは vncserver のパスワードとは異なります。

```
[username@wisteria01 ~]$ set_vncserver -h 起動ホスト名 -p 起動ポート番号
Enter LDAP Password: ← Wisteria/BDEC-01 システム(利用支援ポータル)のパスワードを入力
```

例) 起動ノード=wisteria01、起動ポート番号=5910(5900+ディスプレイ番号 10)で登録する

```
[username@wisteria01 ~]$ set_vncserver -h wisteria01 -p 5910
Enter LDAP Password: ← Wisteria/BDEC-01 システム(利用支援ポータル)のパスワードを入力
```

2.8.1.2. vncserver の設定確認と設定変更

必要に応じて vncserver の設定確認及び設定変更を実施ください。

2.8.1.2.1. vncserver のパスワード変更

vncserver のパスワードの変更は以下のように行います。

```
[username@wisteria01 ~]$ vncpasswd
Password: ← パスワードを入力
Verify:    ← 再度パスワードを入力
Would you like to enter a view-only password (y/n)? n ← 画面閲覧モードのパスワード入力の有無
```

2.8.1.2.2. vncserver のディスプレイ番号の確認

(1) vncserver のポート番号を確認します。

以下のコマンドを実行します。下記赤丸がディスプレイ番号となります。

```
[username@wisteria01 ~]$ vncserver -list

TigerVNC server sessions:

X DISPLAY #      PROCESS ID
:10          760774
```

2.8.1.2.3. デスクトップ環境の解像度、背景色の変更

デスクトップ環境の解像度、背景色の設定を変更する設定方法を以下に示します。設定変更の対象ファイルは vncserver コマンドの初回実行時に作成されます。設定変更後、再度 vncserver を起動することで変更した設定が読み込まれます。

(1) デスクトップ環境の解像度を変更します。

```
[username@wisteria01 ~]$ vi $HOME/.vnc/config

## Supported server options to pass to vncserver upon invocation can be listed
## in this file. See the following manpages for more: vncserver(1) Xvnc(1).
## Several common ones are shown below. Uncomment and modify to your liking.
##
# securitytypes=vncauth,tlsvnc
# desktop=sandbox
# geometry=2000x1200
# localhost
# alwaysshared
```

- (2) デスクトップ環境の背景色を変更します。

デフォルトの背景色は黒色になります。

```
[username@wisteria01 ~]$ vi $HOME/.config/pcmanfm/default/desktop-items-0.conf
[*]
wallpaper_mode=color
wallpaper_common=1
desktop_bg=#0000ff
desktop_fg=#ffffff
desktop_shadow=#000000
desktop_font=Sans 12
show_wm_menu=0
sort=mtime;ascending;
show_documents=0
show_trash=1
show_mounts=0
```

2.8.1.2.4. vncserver の起動ホスト確認

show_vncserver コマンドで vncserver を起動したホスト名とポート番号を確認できます。

```
[username@wisteria01 ~]$ show_vncserver
server: wisteria01    ← 起動ホスト
port  : 5910         ← ポート番号
```

2.8.1.3. 環境変数 DISPLAY の利用

ログインノードで、環境変数 DISPLAY に vncserver を起動しているログインノードのホスト名及びディスプレイ番号を指定することで、既に起動している vncserver にアプリケーションを起動することが可能です。

ただし、vncserver の起動オプションの変更が必要となります。

- (1) vncserver 起動時のオプション変更

以下のようにオプションを指定して vncserver を起動してください。

```
[username@wisteria01 ~]$ vncserver -listen TCP
```

- (2) 環境変数 DISPLAY の設定とアプリケーションの起動

例えば、wisteria03 から wisteria01 の vncserver(ディスプレイ番号:3)にアプリケーションを起動する場合、以下のように環境変数 DISPLAY を設定し、アプリケーションを起動してください。

```
[username@wisteria03 ~]$ export DISPLAY=wisteria01:3
```

2.8.2. Apache Guacamole への接続

(1) ブラウザで下記 URL にアクセスします。

<https://wisteria09.cc.u-tokyo.ac.jp:8443/guacamole/>



Internet Explorer では正常にアクセスできない場合がありますので、それ以外のブラウザをお使いください。



利用されるネットワーク環境によっては、Apache Guacamole の URL にアクセスできない場合があります。アクセスできない場合は、「2.9 SSH ポートフォワードを使用した Web サービスへのアクセス方法」を参照してください。

(2) ログイン画面が表示されますので、Wisteria/BDEC-01 システムのユーザー名とパスワードでログインします。
※Wisteria/BDEC-01 システムのユーザー名とパスワードは利用支援ポータルにログインするためのユーザー名とパスワードです。



図 2-27 Apache Guacamole ログイン画面

(3) パスワード入力画面が表示されますので、vncserver のパスワードを入力します。



図 2-28 Apache Guacamole ログイン画面

ログインが正常終了するとログインノードのデスクトップ環境が表示されますので各種操作を行うことが可能です。

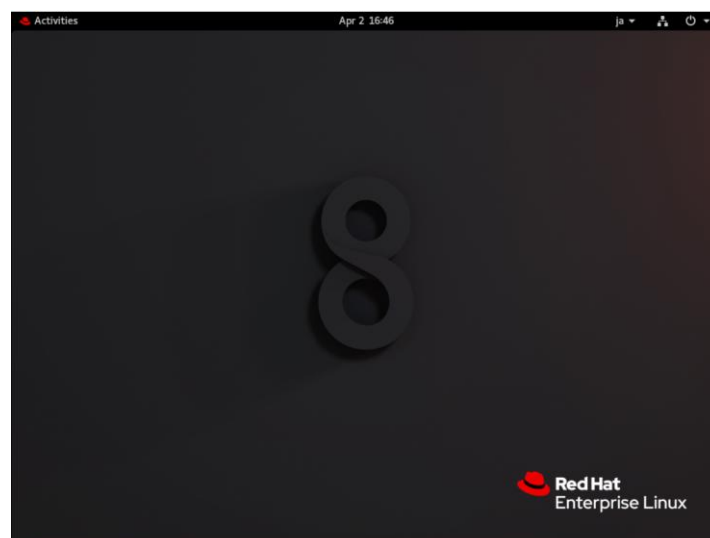


図 2-29 Apache Guacamole ログイン後の画面

2.8.3. Apache Guacamole のログアウト

Apache Guacamole でログイン中に Web ブラウザを閉じてても状態は保持されますので、再度アクセスすると作業を再開させることができます。Apache Guacamole からログアウトする場合の処理は以下の通りです。

(1) Windows 端末の場合は [Ctrl]+[Alt]+[Shift] を入力し、画面左側に表示されるメニューよりログアウトします。また、Mac 端末の場合は [Ctrl]+[Command]+[Shift] を入力することで表示されるメニューよりログアウトします。

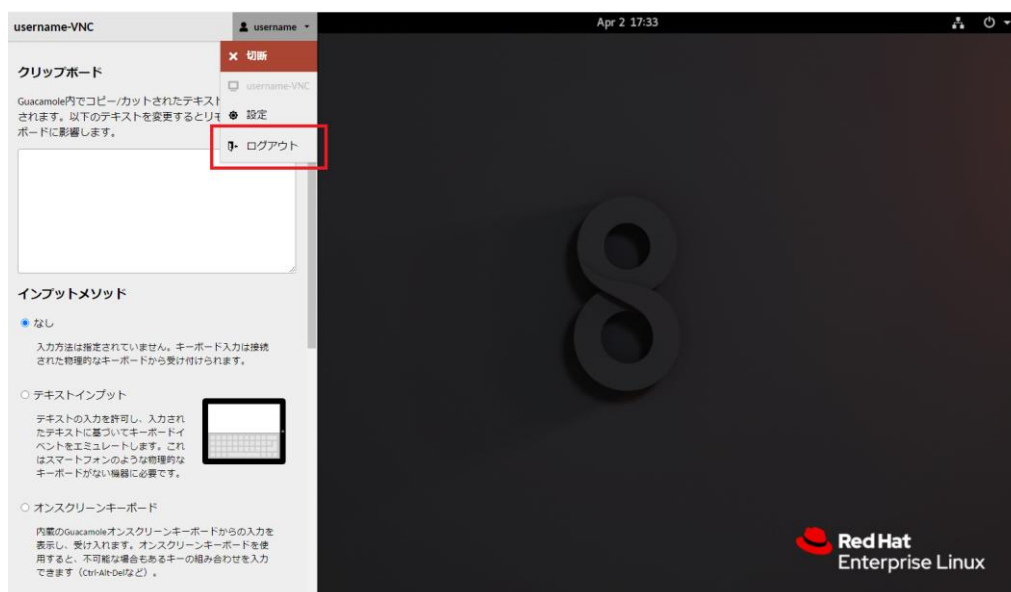


図 2-30 Apache Guacamole ログアウト画面（Windows の場合）

(2) vncserver を起動したログインノードで以下のコマンドを実行し、起動中のセッション番号を確認します。

```
[username@wisteria01 ~]$ vncserver -list
```

例) vncserver のセッション番号を確認する(下記の場合、セッション番号=2)

```
[username@wisteria01 ~]$ vncserver -list
```

TigerVNC server sessions:

```
X DISPLAY #      PROCESS ID
:2          3113735
[username@wisteria01 ~]$
```

(3) vncserver を起動したログインノードで以下のコマンドを実行し、vncserver を終了します。

```
[username@wisteria01 ~]$ vncserver -kill :セッション番号
```

vncserver の起動ホストは、show_vncserver コマンドを使用して確認ください。show_vncserver コマンドの使用法は「2.8.1.2.4 vncserver の起動ホスト確認」を参照ください。

起動ホストへのログインは show_vncserver コマンドで表示されたホスト名にドメイン名 "cc.u-tokyo.ac.jp" を付与してアクセスください。

例) wisteria01 にログインする場合 (Windows PowerShell および Unix 端末の場合)

```
$ ssh -l username wisteria01.cc.u-tokyo.ac.jp
```

以下に、任意のログインノード(wisteria02)にて vncserver の起動ホスト(wisteria01)を確認し、
起動ホストに SSH ログインし vncserver を終了する場合の例を記載します。
セッション番号=2 の vncserver を終了する場合の例を記載しています。

- 1) vncserver の起動ホストを確認する。

```
[username@wisteria02 ~]$ show_vncserver  
server: wisteria01  
port : 5902  
[username@wisteria02 ~]$
```

- 2) クライアント端末から vncserver 起動ホストの wisteria01 にドメイン名(cc.u-tokyo.ac.jp)を付与
して SSH ログインする(Windows PowerShell および Unix 端末の場合)。

```
$ ssh -l username wisteria01.cc.u-tokyo.ac.jp
```

- 3) vncserver を終了する(セッション番号=2)。

```
[username@wisteria01 ~]$ vncserver -kill :2  
Killing Xvnc process ID 3113735  
[username@wisteria01 ~]$ vncserver -list  
TigerVNC server sessions:  
X DISPLAY # PROCESS ID  
[username@wisteria01 ~]$
```

2.9. SSH ポートフォワードを使用した Web サービスへのアクセス方法

利用者のネットワーク環境によっては、本システムの JupyterHub や Apache Guacamole の URL に直接アクセスできない場合があります。その場合は、SSH ポートフォワードを使用したアクセスをお試ください。以下に、クライアント PC が Windows 環境の場合を例にしたアクセス方法を記載します。Unix 環境でも同様の SSH コマンド操作でアクセス可能です。

2.9.1. SSH ポートフォワードを設定したログインノードへの SSH アクセス

- (1) Windows PowerShell を起動して、SSH ポートフォワード設定を行いログインノードに SSH アクセスします。-L オプションで指定するポート番号は、クライアント PC から Web アクセスするための任意のポート番号を指定ください。

・JupyterHub の場合

```
PS C:¥> ssh -l username -i 秘密鍵のパス -L ポート番号:wisteria08.cc.u-tokyo.ac.jp:8000 wisteria.cc.u-tokyo.ac.jp
```

・Apache Guacamole の場合

```
PS C:¥> ssh -l username -i 秘密鍵のパス -L ポート番号:wisteria09.cc.u-tokyo.ac.jp:8443 wisteria.cc.u-tokyo.ac.jp
```

2.9.2. Web サービスへのアクセス

- (1) 2.9.1 で指定した“ポート番号”を設定し Web ブラウザからアクセスします。

・JupyterHub の場合

```
https://localhost:ポート番号/jupyterhub/
```

・Apache Guacamole の場合

```
https://localhost:ポート番号/guacamole/
```

3. ファイルシステム環境

3.1. ファイルシステムの概要

Wisteria/BDEC-01 システムでは、以下のファイルシステム領域が利用可能です。

表 3-1 ファイルシステム領域一覧

ファイルシステム領域	ファイルシステム	サイズ	利用目的
ログインノード共有ファイルシステム	XFS + NFS version 4	約 13 TB	ホーム領域
共有ファイルシステム	FEFS	約 23 PB	作業領域
高速ファイルシステム	FEFS	約 1PB	作業領域(一時/恒久)
Ipomoea-01 システムのストレージ	Lustre + NFS version 3	約 23PB	作業領域

ログインノード用共有ファイルシステムは、ホーム領域として割り当てられ、ユーザーあたり 50 GB のディスクが割り当てられます。

共有ファイルシステムは利用申請の内容に応じたディスクがグループ単位で割り当てられます。パーソナルコース以外のコースに申し込んだ場合、プロジェクト内でファイルを共有する目的のために「/work/グループ名/share」が利用可能です。

高速ファイルシステムはユーザー領域として一時領域と恒久領域の 2 つを用意します。一時領域は 1 週間更新がないファイルが削除されます。また恒久領域は申請制とし、許可された場合に利用可能となります。

表 3-2 割り当てディレクトリと割り当てサイズ

ファイルシステム領域	ディレクトリ	割り当てサイズ
ログインノード用共有ファイルシステム	/home/ユーザー名	50 GB
共有ファイルシステム	/work/グループ名/ユーザー名 /work/グループ名/share	2 TB (1 セットあたり)
高速ファイルシステム (一時領域)	/data/scratch/グループ名/ユーザー名	利用申請内容に応じて設定
高速ファイルシステム (恒久領域)	/data/perm/グループ名/ユーザー名	利用申請内容に応じて設定

ログインノード、および計算ノードから各ファイルシステムへのアクセス可否を以下に示します。

表 3-3 ファイルシステム領域へのアクセス可否一覧

ファイルシステム領域	ログインノード プリポストノード	Odyssey 計算ノード	Aquarius 計算ノード
ログインノード用共有ファイルシステム	○	×	×
共有ファイルシステム	○	○	○
高速ファイルシステム (一時, 恒久)	○	○	○
Ipomoea-01 システムのストレージ	○ (ログインノードのみ)	×	×

○ : 利用可能 × : 利用不能

Ipomoea-01 システムのストレージへのアクセスパスを表 3-4 に示します。

表 3-4 Ipomoea-01 システムのストレージへのアクセスパス

Ipomoea-01	Wisteria/BDEC-01
/home/ユーザー名	/common1/home/ユーザー名
/work/グループ名	/common1/work/グループ名

Ipomoea-01 システムについての詳細は下記を参照ください。

<https://www.cc.u-tokyo.ac.jp/supercomputer/ipomoea01/service/>

3.1.1. ホームディレクトリの変更

OdysseyとAquariusの計算ノードからは、ユーザー登録時に設定されたホームディレクトリ(/home/ユーザー名)にアクセスできません。利用するアプリケーションによっては、ホームディレクトリにアクセスが必須のアプリケーションもあります。そのような場合は、chhome コマンドを実行しホームディレクトリを計算ノードからアクセス可能な共有ファイルシステム(/work/グループ名/ユーザー名)に変更できます。chhome コマンドはログインノード上で実行してください。

\$ chhome [option] <homedir>

chhome コマンドのオプションを表 3-5に示します。

表 3-5 chhome コマンドのオプション一覧

オプション名	説明
なし	ホームディレクトリ(\$HOME)を homedir に変更します。 homedir に指定可能なのは以下の 2 種類です。 ● /home/ユーザー名 ● /work/グループ名/ユーザー名 .ssh などドット付きファイルも移動しますが、移動先に同名ファイルが存在する場合は上書きされません。
-f	移動先に .ssh などドット付きファイルがあっても上書きします。

例) ホームディレクトリを/work/groupname/username に変更する

```
[username@wisteria01 ~]$ chhome /work/groupname/username
Enter LDAP Password: ← Wisteria/BDEC-01 システム(利用支援ポータル)のパスワードを入力
modifying entry "uid=username,ou=People,dc=wisteria-bdec01,dc=cc,dc=u-tokyo,dc=ac,dc=jp"

[username@wisteria01 ~]$
```



ホームディレクトリの変更は次回ログイン時から有効になりますので、利用端末から一度ログアウトして再ログインしてください。

3.1.1.1. JupyterNotebook のホームディレクトリ変更

chhome コマンドで変更したホームディレクトリを JupyterHub 環境に反映するためには、起動済みの JupyterNotebook プロセスを再起動する必要があります。

JupyterNotebook プロセスの再起動は以下の手順で実施してください。

- (1) 起動している JupyterNotebook を保存して閉じます。
- (2) JupyterHub 画面上部の[File]メニューから[Hub Control Panel]を選択します。

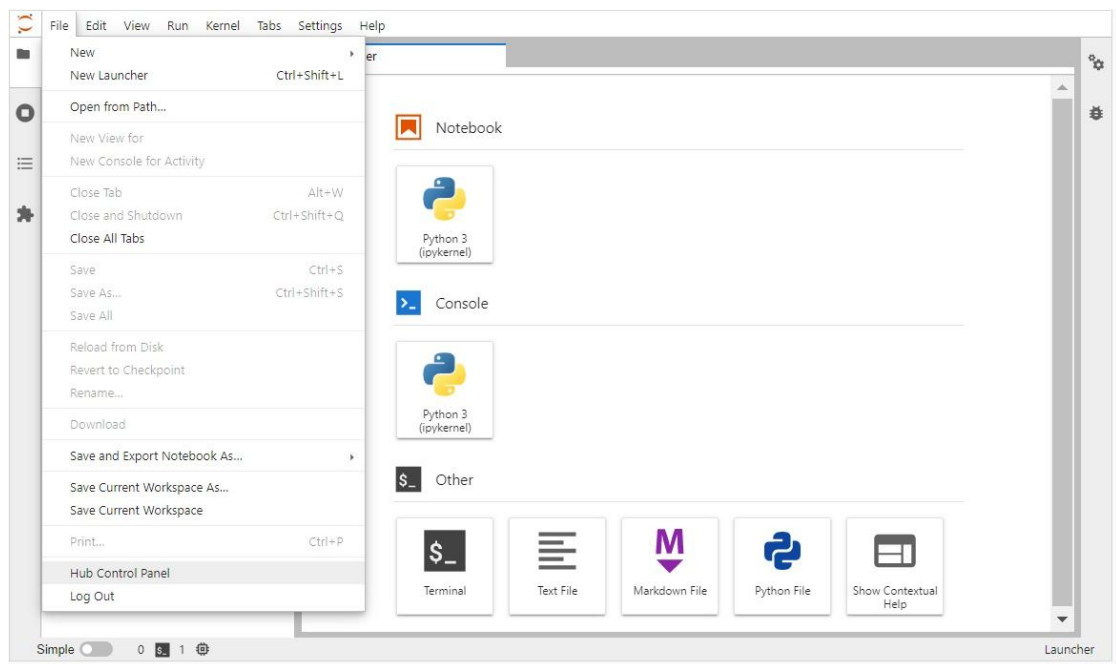


図 3-1 JupyterHub メニュー画面

- (3) JupyterHub の WebApp 画面が新規に起動するため、[Stop My Server]を押下し JupyterNotebook プロセスを停止します。

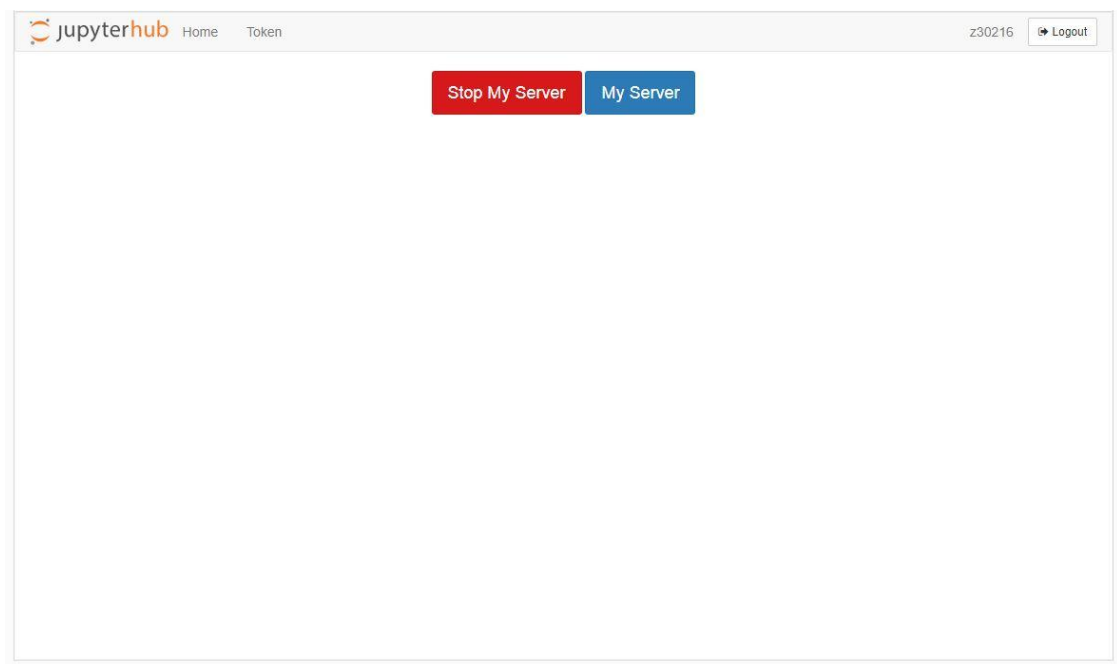


図 3-2 JupyterHub の WebApp 画面

- (4) [Start My Server]を押下し JupyterNotebook プロセスを起動します。
- ホームディレクトリの変更が反映されるまでに一定時間を要する場合があります。chhomeコマンドを実行してから 10 分程度経過してから JupyterNotebook プロセスを起動してください。

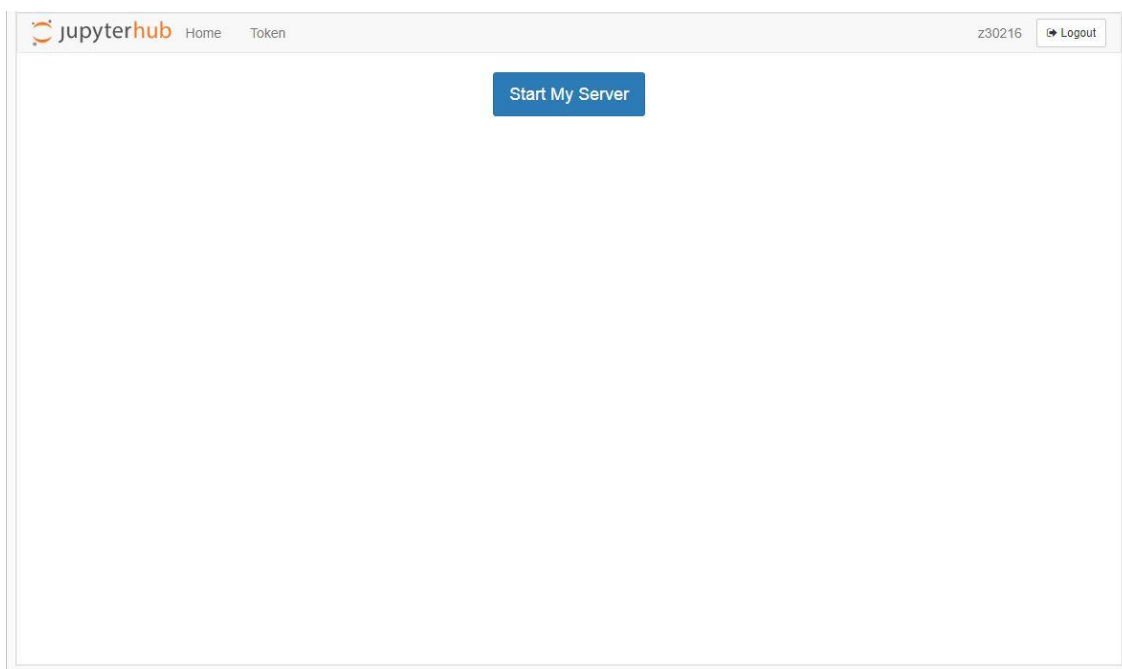


図 3-3 JupyterHub の WebApp 画面

- (5) Terminal を起動し printenv コマンドなどでホームディレクトリが変更されていることを確認します。

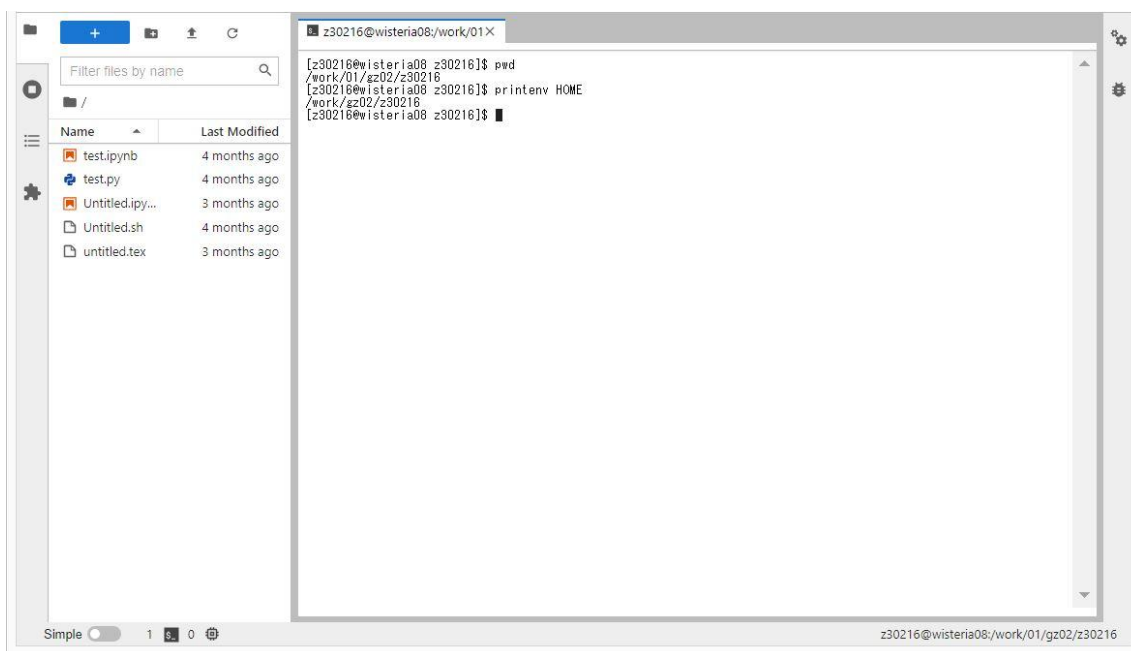


図 3-4 JupyterHub の Terminal 画面

3.2. 共有ファイルシステム

3.2.1. 共有ファイルシステムにおけるデータ格納方式

共有ファイルシステムでは、データはファイル単位に複数の I/O サーバ(OSS)を介して複数のディスク装置(OST)に分散(ラウンドロビン)して格納されます。ラウンドロビンにより、多数のファイルをアクセスする場合でも、複数の I/O サーバ(OSS)を使った高速なファイルアクセスが可能です。

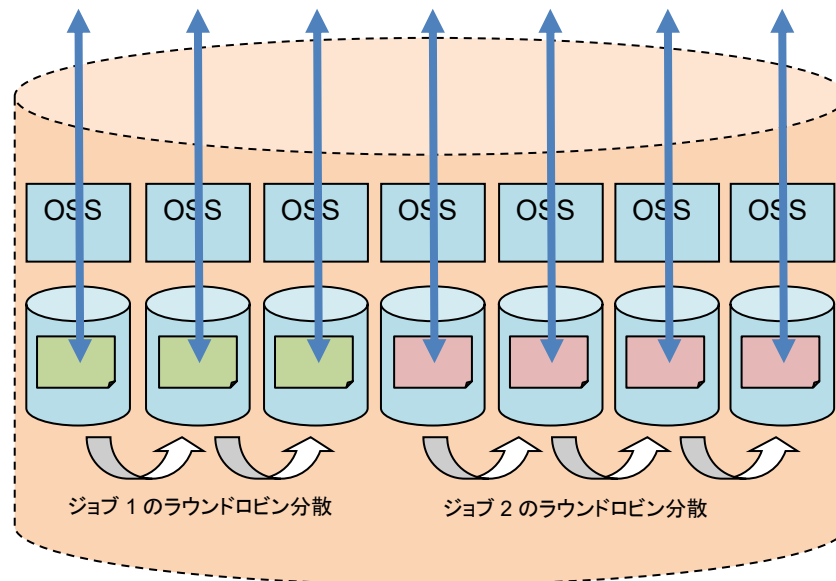


図 3-5 ラウンドロビン分散

3.2.2. ストライプ機能

3.2.2.1. ラウンドロビンとストライプ

共有ファイルシステムでは、ファイルのデータは OSS を介して OST に分散して格納します。分散方式として、2つの方式を選択できます。

(1) ラウンドロビン方式

ファイル単位で OST をラウンドロビンで選択して格納します。最大のファイルサイズは物理的な OST の容量となります。デフォルトの分散方式はラウンドロビン方式です。

(2) ストライプ方式

1 つのファイルのデータを OST に分散させて格納します。ファイルサイズは物理的な OST の容量を上回ることが可能となります。使用するためには事前に設定が必要です。

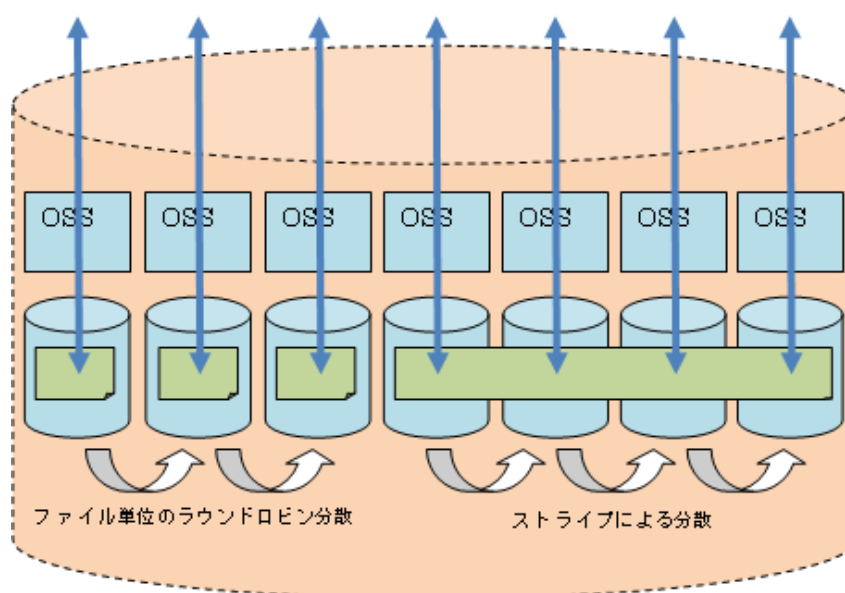


図 3-6 ラウンドロビン分散とストライプ分散

3.2.2.2. ストライプ機能の効果

ストライプ機能は、1つのファイルのデータを、先頭から指定したサイズ単位で複数の OST に分散して格納し、ファイルアクセスを分散させる機能です。以下の効果があります。

- 物理的な1つのOSTの容量を超えるサイズのファイルが作成可能
- 1つのファイルのデータを複数のOSTに分散して格納することによりファイルアクセスの帯域幅が向上

MDS がファイルの拡張

属性として保持

- ・ ストライプサイズ
- ・ ストライプカウント
- ・ OSTインデックス

MDS

クライアント
(ログインノード)

1ファイル

Strine#1
Strine#2
Strine#3
...
Strine#102

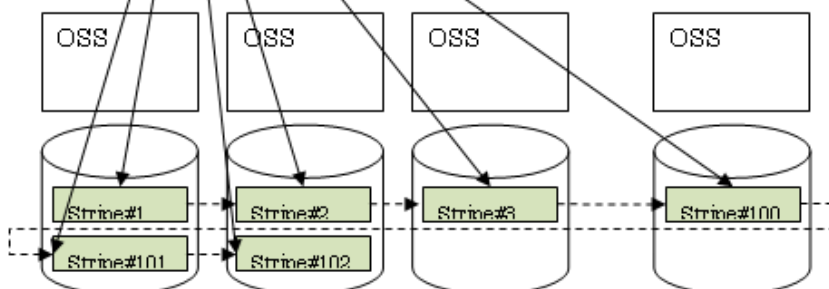
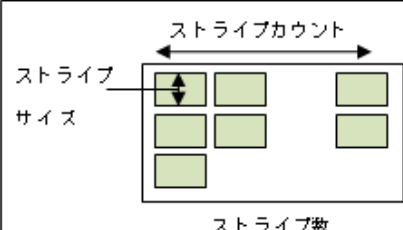


図 3-7 ストライプ機能

3.2.2.3. ストライプ機能の設定方法

ストライプの設定は、*lfs setstripe* コマンドで行います。

```
$ lfs setstripe [options] <dirname | filename>
```

表 3-6 lfs setstripe コマンドオプション一覧

オプション名	説明
--stripe-size -S	ストライプサイズを設定。 -s #k,-s #m,-s #g とすることで、サイズを KiB,MiB,GiB で設定可能。
--start_ost_index -i	ファイル書き込みを開始する OST インデックスを指定。 -1 とした場合、ファイル書き込みを開始する OST はランダム。
--stripe-count -c	ストライプカウントを設定。 -1 とした場合、すべての OST に書き込みを実行。

lfs setstripe コマンドでは、データを分散させるストライプパターン（ストライプサイズ、範囲、ストライプカウント）を指定することができます。

図 3-7 では、ディスク 10 台にストライプさせておりストライプカウントは 10 になります。OST インデックスの設定により、分散させる範囲が設定可能です。

OST インデックスは、*lfs osts* の出力情報で参照可能です(下記赤字部分)。

```
[username@wisteria01 work]$ lfs osts
OBDS::
0: work-OST0000_UUID ACTIVE
1: work-OST0001_UUID ACTIVE
(snip)
126: work-OST007e_UUID ACTIVE
127: work-OST007f_UUID ACTIVE
OBDS:
0: data-OST0000_UUID ACTIVE
1: data-OST0001_UUID ACTIVE
(snip)
126: data-OST007e_UUID ACTIVE
127: data-OST007f_UUID ACTIVE
```

lfs setstripe コマンドは、ストライプパターンを持った新規ファイルを作成します。また、既存のディレクトリに対して、ストライプパターンを設定することもできます。

例) ストライプの設定例 1(ストライプパターンを持った新規ファイルの作成)

```
[username@wisteria01 work]$ lfs setstripe -S 1m -i 11 -c 4 test_stripe1
```

例) ストライプの設定例 2(ディレクトリに対して、ストライプパターンを設定)

```
[username@wisteria01 work]$ mkdir test_stripe2
[username@wisteria01 work]$ lfs setstripe -S 1m -i 11 -c 4 test_stripe2
```

3.2.2.4. ストライプ設定の表示方法

ファイルやディレクトリのストライプパターンの情報を表示する場合は、*lfs getstripe* コマンドで行います。

```
$ lfs getstripe <dirname | filename>
```

例) ストライプ設定の表示例 1(ファイルのストライプパターン情報の表示)

```
[username@wisteria01 work]$ lfs getstripe test_stripe1
test_stripe1
lmm_stripe_count: 4
lmm_stripe_size: 1048576
lmm_pattern: 1
lmm_layout_gen: 0
lmm_stripe_offset: 11
      obdidx      objid      objid      group
        11      849584      0xcf6b0         0
        70      455883      0x6f4cb         0
        49      455990      0x6f536         0
        58      455918      0x6f4ee         0
```

例) ストライプ設定の表示例 2(ディレクトリのストライプパターン情報の表示)

```
[username@wisteria01 work]$ lfs getstripe test_stripe2
test_stripe2
stripe_count: 4 stripe_size: 1048576 stripe_offset: 11
```

3.3. ディスク使用状況と上限値の表示

ログインノード用共有ファイルシステムと共有ファイルシステムのディスク使用状況と上限値を表示するには、`show_quota` コマンドを実行してください。

```
[username@wisteria01 work]$ show_quota
Disk quotas for user username
Directory          used(MB)          limit(MB)          nfiles    nlimit
/home              9,327             51,200             36,837    unlimited

Disk quotas for project gz02 pz0201
Directory    User    used(MB)          limit(MB)          nfiles    nlimit
/work/gz02   -       200,000          204,800           1,000    unlimited
              share    0               unlimited         10       unlimited
              z30201  100,000          unlimited         500      unlimited
/work/gz03   -       388,877,960      2,147,483,648     52,392,599 unlimited
              share    0               unlimited         10       unlimited
              z30201  100,000          unlimited         500      unlimited

Disk quotas for project gz02 gz03
Directory          used(MB)          limit(MB)          nfiles    nlimit
/data/scratch/gz02 200,000          204,800           1,000    unlimited
/data/scratch/gz03 388,877,960      2,147,483,648     52,392,599 unlimited

Disk quotas for project gz02
Directory          used(MB)          limit(MB)          nfiles    nlimit
/data/perm/gz02    200,000          204,800           1,000    unlimited
```

表 3-7 show_quota の表示項目

項目	説明
Directory	割り当てディレクトリ /home : ログインノード用共有ファイルシステム /work/groupname : 共有ファイルシステム /data/scratch/groupname : 高速ファイルシステム(一時領域) /data/perm/groupname : 高速ファイルシステム(恒久領域)
used(MB)	ディスク使用量
limit(MB)	ディスク使用上限
nfiles	ファイル数
nlimit	ファイル数上限



共有ファイルシステムおよび高速ファイルシステムでは、1 ディレクトリ配下に作成できるファイル数に上限があります。上限値はファイル名の長さに依存し、約4M～12M個です。上限値を超えてファイルを作成しようとした場合、「No space left on device」のエラーが発生します。

4. コンパイル/リンク

4.1. 環境設定

Wisteria/BDEC-01 システムのログインノードでは、Odyssey または Aquarius に対応するコンパイラを用いて、各システムの計算ノード向けに最適化されたロードモジュールを作成することが可能です。ログインノードで利用可能なコンパイラを以下に示します。

表 4-1 ログインノードで利用可能なコンパイラ一覧

ソフトウェア名称	対象システム	概要
Fujitsu Development Studio	Odyssey	富士通社製コンパイラ
Intel OneAPI	Aquarius	インテルコンパイラ
NVIDIA HPC SDK	Aquarius	GPU 対応コンパイラ
NVIDIA CUDA SDK	Aquarius	GPGPU 統合開発環境
GCC	Aquarius	GNU コンパイラ

`module` コマンドにてシステム名(Odyssey/Aquarius)を指定することで、各システムのデフォルト設定を load することができます。Odyssey では富士通コンパイラ及び富士通 MPI、Aquarius では GCC を load します。設定例を以下に示します。

例1) Odyssey を指定した設定

```
[username@wisteria01 work]$ module load odyssey
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) fjmpi/1.2.31   2) fj/1.2.31   3) odyssey
```

例2) Aquarius を指定した設定

```
[username@wisteria01 work]$ module load aquarius
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) gcc/8.3.1   2) aquarius
```

また、各コンパイラの設定例を以下に示します。

例1) Fujitsu Development Studio のコンパイラおよび MPI ライブラリを使用する設定

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fjmpi
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) fj/1.2.31   2) fjmpi/1.2.31
```

例2) Intel OneAPI のコンパイラおよび MPI ライブラリを使用する設定

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load impi
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) intel/2021.2.0  2) impi/2021.2.0
```

例3) NVIDIA HPC SDK のコンパイラおよび OpenMPI ライブラリを使用する設定

```
[username@wisteria01 work]$ module load nvidia
[username@wisteria01 work]$ module load ompi
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) nvidia/21.3  2) ompi/4.1.1
```

例4) NVIDIA CUDA SDK を使用する設定

```
[username@wisteria01 work]$ module load cuda
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) cuda/11.2
```

例5) MPI 並列で NVIDIA CUDA SDK を使用する設定

```
[username@wisteria01 work]$ module load cuda
[username@wisteria01 work]$ module load gcc
[username@wisteria01 work]$ module load ompi
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) cuda/11.2  2) gcc/8.3.1  3) ompi/4.1.1
```

例6) GCC を使用する設定

```
[username@wisteria01 work]$ module load gcc
[username@wisteria01 work]$ module list
Currently Loaded Modulefiles:
  1) gcc/8.3.1
```

module コマンドの利用方法については、「2.6.4 実行パス/ライブラリ設定」を参照してください。

4.2. Fujitsu Development Studio のコンパイル/リンク概要

4.2.1. クロスコンパイラとオウンコンパイラ

Odyssey の計算ノード（FX1000）と、ログインノード（PRIMERGY）は異なる CPU アーキテクチャーです。そのため、ログインノードには FX1000 向けの実行モジュールを作成するためのクロス開発環境が提供されています。一方、Odyssey の計算ノードではオウン開発（ネイティブ開発）環境が提供されており、ジョブとしてプログラムのコンパイル/リンクを行うことが可能です。

表 4-2 Fujitsu Development Studio の開発環境

種別	説明
クロスコンパイラ	ログインノード上で使用します。
オウンコンパイラ	Odyssey 計算ノード上で使用します。

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-3 Fujitsu Development Studio のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	推奨オプション ^{注1}	自動並列 ^{注2}	OpenMP ^{注2}
逐次 (非 MPI)	Fortran	frrpx	-Kfast	-Kparallel	-Kopenmp
	C	fccpx			
	C++	FCCpx			
MPI 並列	Fortran	mpifrrpx			
	C	mpifccpx			
	C++	mpiFCCpx			

注 1: 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: 自動並列、OpenMP オプションはデフォルトでは無効です。

4.2.2. 逐次プログラムのコンパイル/リンク (Fujitsu Development Studio)

逐次プログラムを Odyssey 計算ノード向けにコンパイル/リンクする場合、*frtpx/fccpx/FCCpx* コマンドを使用します。

```
$ frtpx/fccpx/FCCpx [option] file [...]
```

例1) 逐次 Fortran プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ frtpx -Kfast seq.f
```

例2) 逐次 C プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ fccpx -Kfast seq.c
```

4.2.3. スレッド並列プログラムのコンパイル/リンク (Fujitsu Development Studio)

スレッド並列プログラムを Odyssey 計算ノード向けにコンパイル/リンクする場合、*frtpx/fccpx/FCCpx* コマンドを使用します。

```
$ frtpx/fccpx/FCCpx thread-option [option] file [...]
```

例1) Fortran プログラムを Odyssey 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ frtpx -Kfast -Kparallel para.f
```

例2) C プログラムを Odyssey 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ fccpx -Kfast -Kparallel para.c
```

例3) OpenMP Fortran プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ frtpx -Kfast -Kopenmp omp.f
```

例4) OpenMP C プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria work]$ fccpx -Kfast -Kopenmp omp.c
```

例5) OpenMP Fortran プログラムを Odyssey 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ frtpx -Kfast -Kparallel -Kopenmp omp_para.f
```

例6) OpenMP C プログラムを Odyssey 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ fccpx -Kfast -Kparallel -Kopenmp omp_para.c
```

4.2.4. MPI 並列プログラムのコンパイル/リンク (Fujitsu Development Studio)

MPI 並列プログラムを Odyssey 計算ノード向けにコンパイル/リンクする場合は、*mpifrtpx/mpifccpx/mpiFCCpx* コマンドを使用します。

```
$ mpifrtpx/mpifccpx/mpiFCCpx [option] file [...]
```

例1) MPI Fortran プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifrtpx -Kfast mpi.f
```

例2) MPI C プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifccpx -Kfast mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifrtpx -Kfast -Kopenmp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Odyssey 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifccpx -Kfast -Kopenmp mpi_omp.c
```

4.2.5. Fujitsu Development Studio 推奨オプションによるプログラムの最適化

(Fortran/C/C++)

Odyssey 計算ノード向けに実行性能を追求する最適化オプションとして、-Kfast を推奨します。

このオプションで翻訳すると、内部で以下のオプションが有効となります。

表 4-4 -Kfast で有効となる最適化オプション

オプション	説明
-O3	最適化レベル 3(最大)で翻訳します。
-Keval	演算の評価方法を変更する最適化を適用します。
-Kfast_matmul (C/C++のみ)	行列積のループを高速なライブラリ呼び出しに変換します。
-Kfp_contract	Floating-Point Multiply-Add/Subtract 演算命令を使用した最適化を行います。
-Kfp_relaxed	逆数近似演算を行います。
-Kfz	flush-to-zero モードを使用します。
-Kilfunc	組込み関数およびべき乗演算をインライン展開します。
-Kmfunc	組込み関数および演算をマルチ演算関数に変換する最適化を行います。
-Komitfp	手続き呼出しにおけるフレームポインタレジスタを保証しない最適化を行います。
-Ksimd_packed_promotion	packed-SIMD 化を促進します。



上記の最適化オプションには副作用を伴うものが含まれているため、演算結果に影響を与える場合があります。使用する場合は、実行結果に問題がないかをご確認ください。

4.2.6. Odyssey 計算ノード上でコンパイル/リンクを行う場合

Fujitsu Development Studio では Odyssey 計算ノード上で動作するコンパイラ（OWNコンパイラ）を使用することにより、計算ノード上でコンパイル/リンクを行うことが可能です。計算ノード上のコンパイル/リンクコマンド名は、以下のようにログインノード上のものと異なります。オプションの指定方法は共通です。

表 4-5 コンパイル/リンクコマンドの違い

種別	言語処理系	ログインノード上の コンパイル/リンクコマンド	Odyssey 計算ノード上の コンパイル/リンクコマンド
逐次 (非 MPI)	Fortran	frtpx	frt
	C	fccpx	fcc
	C++	FCCpx	FCC
MPI 並列	Fortran	mpifrtpx	mpifrt
	C	mpifccpx	mpifcc
	C++	mpiFCCpx	mpiFCC

4.2.7. C コンパイラの 2 種類のモードと並列処理ライブラリの組合せについて

Fujitsu Development Studio の C コンパイラは、ユーザーインターフェースの異なる 2 種類のモードを持ちます。使用するモードは、翻訳コマンドのオプションで選択します。OSS 翻訳で問題がある場合、clang モードを試してください。

また、並列処理用に 2 つのライブラリ（LLVM OpenMP ライブラリと富士通 OpenMP ライブラリ）を提供しています。

表 4-6 C コンパイラの 2 種類のモード

モード名	説明
trad モード (-Nnoclang オプション)	京および PRIMEHPC FX100 以前のシステム向け富士通コンパイラをベースとしています。trad モードは、従来の富士通コンパイラとの互換を重視する場合に適しています。サポートしている仕様は、C89/C99/C11、OpenMP 3.1/OpenMP 4.5(一部)です。オプション省略時は、-Nnoclang オプションが適用されます。
clang モード (-Nclang オプション)	オープンソースソフトウェアである Clang/LLVM コンパイラをベースとしています。clang モードは、最新言語仕様を使用したプログラムや、オープンソースソフトウェアを翻訳する場合に適しています。サポートしている仕様は、C89/C99/C11、OpenMP 4.5/OpenMP 5.0(一部)です。

表 4-7 2 つの並列処理ライブラリ

ライブラリ名	説明
LLVM OpenMP ライブラリ (-Nlibomp オプション)	オープンソースソフトウェアである LLVM OpenMP Runtime Library をベースにした並列化機能用のライブラリです。サポートしている仕様は、OpenMP 4.5/OpenMP 5.0(一部)です。trad モードおよび clang モードで利用できます。
富士通 OpenMP ライブラリ (-Nfjompilib オプション)	京および PRIMEHPC FX100 以前のシステム向けの富士通 OpenMP ライブラリをベースとした並列化機能用のライブラリです。従来の富士通 OpenMP ライブラリとの互換を重視する場合に適しています。サポートしている仕様は、OpenMP 3.1/OpenMP 4.5(一部)です。trad モードでのみ利用できます。

並列処理用ライブラリ（LLVM OpenMP ライブラリと富士通 OpenMP ライブラリ）と結合可能なオブジェクトファイルの組合せを下表に示します。

表 4-8 結合可能なオブジェクトファイルの組み合わせ

	Fortran オブジェクト ファイル	C/C++オブジェクトファイル	
		trad モード (-Nnoclang オプション)	clang モード (-Nclang オプション)
LLVM OpenMP ライブラリ (-Nlibomp オプション)	結合可	結合可	結合可
富士通 OpenMP ライブラリ (-Nfjompilib オプション)	結合可	結合可	結合不可

以下の条件をすべて満たしている場合、本処理系は富士通 OpenMP ライブラリを使用します。

- ・プログラムのリンク時に、-Nfjompilib オプションが有効である。
- ・プログラムの翻訳時およびリンク時に、trad モードを使用している。

以下の条件を満たしている場合、本処理系は LLVM OpenMP ライブラリを使用します。

- ・プログラムのリンク時に、-Nlibomp オプションが有効である。

4.2.8. ラージページ機能

HPC 分野における大規模なメモリを使用するアプリケーションプログラムでは、OS がメモリを管理するコストが実行性能に影響を与えます。このコストを減らすために、HPC 拡張機能では独自に拡張したラージページ機能を提供しています。

FX1000 の OS ではページサイズとして、ノーマルページで 64KiB、ラージページで 2MiB を利用可能です。ラージページ機能ではデフォルトでラージページ化が有効となっています。環境変数 (XOS_MMM_L_HPAGED_TYPE) によってラージページの有効化/無効化を選択できます。ラージページを無効化する場合は、環境変数の値に "none" を設定ください。以下はログインシェルが bash の場合の設定例です。

```
export XOS_MMM_L_HPAGED_TYPE=none
```

4.2.8.1. ノーマルページとラージページ

ページサイズに 64KiB、2MiB を利用した際のそれぞれのメリット・デメリットを以下に示します。

表 4-9 ページサイズによるメリット・デメリット

評価項目	64KiB	2MiB
TLB ミス率	高い	低い
メモリ初期化コスト	小さい	大きい
メモリ使用効率	高い	低い

ページサイズ 64KiB は、ノーマルページです。TLB ミス率は高いものの、メモリ初期化コストが小さく、メモリ使用効率が高いため、メモリ使用量が少ないアプリケーションプログラムに対しては有効な場合があります。

ページサイズ 2MiB は、ラージページです。64KiB よりメモリ初期化コストが大きくなり、メモリ使用効率が低くなりますが、TLB ミス率が低くなります。そのため、FX1000 のラージページ機能では、ラージページ(2MiB)利用をデフォルトとしています。

4.2.8.2. ページング方式

ページング方式は、デマンドページング方式とプリページング方式の 2 種類があります。FX1000 のラージページ機能では、環境変数(XOS_MMM_L_PAGING_POLICY)によって、静的データ(.bss 領域)、スタック/スレッドスタック領域、および動的メモリ確保領域の各メモリ領域にページング方式を設定できます。

- ・デマンドページング方式とは、アプリケーションプログラム実行中に必要なページが主記憶に存在しない場合に、必要に応じて ページを主記憶に割り当てる方式のことです。最初にメモリ領域にアクセスしたタイミングで物理ページを割り当てます。
- ・プリページング方式とは、あらかじめ主記憶にページを割り当てておく方式のことです。メモリ領域を割り当てたタイミングで物理ページを割り当てます。

ページング方式の違いによるメモリアクセスコストは以下の通りです。

表 4-10 ページング方式の違いによるメモリアクセスコスト

ページング方式	NUMA 内/外のメモリアクセス	初回メモリアクセス
デマンドページング方式	できる限り NUMA 内のメモリを使用 (コスト：低い)	メモリアクセス時にページを物理メモリにロード (コスト：高い)
プリページング方式	NUMA 内外のメモリに関係なく使用 (コスト：高い)	ページをあらかじめ物理メモリにロード (コスト：低い)

ラージページライブラリのページング方式を設定するための環境変数を以下に示します。

表 4-11 ページング方式設定環境変数

変数名	指定値	デフォルト	詳細
XOS_MMM_L_PAGING_POLICY	[demand prepage]: [demand prepage]: [demand prepage]	prepage: demand: prepage	各メモリ領域のページング方式(ページの割り当て契機)を選択する設定です。 「demand」はデマンドページング方式、「prepage」はプリページング方式を意味します。 第 1 指定は、静的データの.bss 領域です(静的データの.data 領域はページング方式指定の対象外で常に prepage となります)。第 2 指定は、スタック領域およびスレッドスタック領域です。第 3 指定は、動的メモリ確保領域です。指定値以外の値を指定した場合は、「prepage:demand:prepage」を指定したものとみなします。 プリページング方式では、その後のアプリケーションプログラムのページフォルトの発生を抑えられ、性能改善および性能ブレが小さくなる傾向があります。ただし、アプリケーションプログラムが自前でプリページング(例えばゼロクリアなど)を実装している場合や、メモリの一部の領域にだけアクセスする場合などには、プリページング方式を選択しないほうが良いケースもあります。

4.3. Intel OneAPI のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-12 Intel OneAPI のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション ^{注1}	自動並列 ^{注2}	OpenMP ^{注2}
逐次 (非 MPI)	Fortran	ifort	-axCORE-AVX512	-parallel	-qopenmp
	C	icc			
	C++	icpc			
MPI 並列 ^{注3}	Fortran	mpiifort			
	C	mpiicc			
	C++	mpiicpc			

注 1: Aquarius 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: 自動並列、OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は Intel MPI ライブラリの module のロードが必要です。

4.3.1. 逐次プログラムのコンパイル/リンク (Intel OneAPI)

逐次プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*ifort/icc/icpc* コマンドを使用します。

\$ ifort/icc/icpc [option] file [...]

例1) 逐次 Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ ifort -axCORE-AVX512 seq.f
```

例2) 逐次 C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ icc -axCORE-AVX512 seq.c
```

4.3.2. スレッド並列プログラムのコンパイル/リンク (Intel OneAPI)

スレッド並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*ifort/icc/icpc* コマンドを使用します。

\$ ifort/icc/icpc thread-option [option] file [...]

例1) Fortran プログラムを Aquarius 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ ifort -axCORE-AVX512 -parallel para.f
```

例2) C プログラムを Aquarius 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ icc -axCORE-AVX512 -parallel para.c
```

例3) OpenMP Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ ifort -axCORE-AVX512 -qopenmp omp.f
```

例4) OpenMP C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ icc -axCORE-AVX512 -qopenmp omp.c
```

例5) OpenMP Fortran プログラムを Aquarius 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ ifort -axCORE-AVX512 -parallel -qopenmp omp_para.f
```

例6) OpenMP C プログラムを Aquarius 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ icc -axCORE-AVX512 -parallel -qopenmp omp_para.c
```

4.3.3. MPI 並列プログラムのコンパイル/リンク (Intel OneAPI)

MPI 並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合は、*mpiifort/mpiicc/mpiicpc* コマンドを使用します。

```
$ mpiifort/mpiicc/mpiicpc [option] file [...]
```

例1) MPI Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpiifort -axCORE-AVX512 mpi.f
```

例2) MPI C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpiicc -axCORE-AVX512 mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpiifort -axCORE-AVX512 -qopenmp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpiicc -axCORE-AVX512 -qopenmp mpi_omp.c
```

4.3.4. インテルコンパイラの最適化オプション(Fortran/C/C++)

計算ノード向けの最適化を促進するオプションとして代表的なものを以下に示します。

表 4-13 一般的な最適化オプション

オプション	説明
-ipo	複数のソースファイルにまたがるインライン展開やその他のプロシージャ間の最適化を許可します。
-O[0,1,2,3]	最適化のレベルを指定します。-O 未指定時は-O2 が有効です。
-no-prev-div	浮動小数点除算の精度を下げることでパフォーマンスの向上を図ります。
-fp-model name	特定の最適化を制限することで、浮動小数点演算のパフォーマンス、精度、再現性を制御します。name の値には、fast=[1 2], precise, double, extended, source, except, strict を指定できます。 -fp-model 未指定時はパフォーマンスを重視する fast=1 が有効です。
-qopt-report[=n]	最適化レポートを生成します。デフォルトでは、レポートは.optrpt 拡張子を持つファイルに出力されます。n には、0(レポートなし)から 5(最も詳しい)の詳細レベルを指定します。デフォルトは 2 です。



上記の最適化オプションには副作用を伴うものが含まれているため、演算結果に影響を与える場合があります。使用する場合は、実行結果に問題がないかをご確認ください。

4.3.5. Java MPI

Intel MPI がサポートする Java binding を用いた Java MPI 並列プログラムをコンパイル/リンクする場合は、*javac* コマンドを使用します。

```
$ javac [option] file [...]
```

例1) Java MPI プログラムをコンパイル/リンク

```
[username@wisteria01 work]$ javac sample.java
```

4.4. NVIDIA HPC SDK のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-14 NVIDIA HPC SDK のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化 オプション ^{注1}	自動並列 ^{注2}	OpenMP ^{注2}	OpenACC ^{注2}
逐次 (非 MPI)	Fortran	nvfortran	-fast	-Mconcur	-mp	-acc
	C	nvc				
	C++	nvc++				
MPI 並列 ^{注3}	Fortran	mpifort				
	C	mpicc				
	C++	mpic++				

注 1: Aquarius 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: 自動並列、OpenMP、OpenACC オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は OpenMPI ライブラリの module のロードが必要です。

4.4.1. 逐次プログラムのコンパイル/リンク (NVIDIA HPC SDK)

逐次プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*nvfortran/nvcc/nvc++* コマンドを使用します。

```
$ nvfortran/nvc/nvc++ [option] file [...]
```

例1) 逐次 Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvfortran -fast seq.f
```

例2) 逐次 C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvc -fast seq.c
```

4.4.2. スレッド並列プログラムのコンパイル/リンク (NVIDIA HPC SDK)

スレッド並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*nvfortran/nvc/nvc++* コマンドを使用します。

```
$ nvfortran/nvc/nvc++ thread-option [option] file [...]
```

例1) Fortran プログラムを Aquarius 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ nvfortran -fast -Mconcur para.f
```

例2) C プログラムを Aquarius 計算ノード向けに自動並列を有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ nvcc -fast -Mconcur para.c
```

例3) OpenMP Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvfortran -fast -mp omp.f
```

例4) OpenMP C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvc -fast -mp omp.c
```

例5) OpenMP Fortran プログラムを Aquarius 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ nvfortran -fast -Mconcur -mp omp_para.f
```

例6) OpenMP C プログラムを Aquarius 計算ノード向けに自動並列も有効にしてコンパイル/リンク

```
[username@wisteria01 work]$ nvc -fast -Mconcur -mp omp_para.c
```

4.4.3. OpenACC プログラムのコンパイル/リンク (NVIDIA HPC SDK)

OpenACC プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*nvfortran/nvc/nvc++* コマンドを使用します。

```
$ nvfortran/nvc/nvc++ -acc [option] file [...]
```

例1) OpenACC を使用した Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvfortran -fast -acc acc.f
```

例2) OpenACC を使用した C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvc -fast -acc acc.c
```

4.4.4. MPI 並列プログラムのコンパイル/リンク (NVIDIA HPC SDK + OpenMPI)

MPI 並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合は、*mpifort/mpicc/mpic++* コマンドを使用します。

\$ mpifort/mpicc/mpic++ [option] file [...]

例1) MPI Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpifort -fast mpi.f

例2) MPI C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpicc -fast mpi.c

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpifort -fast -mp mpi_omp.f

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpicc -fast -mp mpi_omp.c

例5) MPI 並列と OpenACC を使用した Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpifort -fast -acc mpi_acc.f

例6) MPI 並列と OpenACC を使用した C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

[username@wisteria01 work]\$ mpicc -fast -acc mpi_omp.c

4.4.5. NVIDIA HPC SDK 推奨オプションによるプログラムの最適化

(Fortran/C/C++)

計算ノード向けに実行性能を追求する最適化オプションとして、-fast を推奨します。

このオプションで翻訳すると、内部で以下のオプションが有効となります。

表 4-15 -fast で有効となる最適化オプション

オプション	説明
-O2	最適化レベル 2(最大 4)で翻訳します。
-Munroll=c:1	ループをアンロールします。
-Mlre	ループの冗長な処理を除去する最適化を行います。
-Mautoinline	関数をインライン展開します。
-Mvect=simd	SIMD 命令を生成します。
-Mflushz	flush-to-zero モードを使用します。
-Mcache_align	キャッシュ境界に合わせたオブジェクト割付けを行います。



上記の最適化オプションには副作用を伴うものが含まれているため、演算結果に影響を与える場合があります。使用する場合は、実行結果に問題がないかをご確認ください。

4.5. NVIDIA CUDA SDK のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-16 NVIDIA CUDA SDK のコンパイル/リンクコマンド一覧

種別	言語 処理系	コマンド	最適化オプション ^{注1}	OpenMP ^{注2}
逐次 (非 MPI)	C	nvcc	-O3	-Xcompiler -fopenmp
	C++	nvcc -ccbin g++		
MPI 並列 ^{注3}	C	nvcc -lmpi		
	C++	nvcc -ccbin g++ -lmpi		

注 1: Aquarius 計算ノードの CPU で実行されるコード部分の最適化オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は GCC および OpenMPI ライブラリの module のロードが必要です。

4.5.1. 逐次プログラムのコンパイル/リンク (NVIDIA CUDA SDK)

逐次プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*nvcc* コマンドを使用します。

```
$ nvcc [option] file [...]
```

例1) 逐次 CUDA プログラムを Aquarius 計算ノード (GPU) 向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvcc -O3 cuda_seq.cu
```

4.5.2. スレッド並列プログラムのコンパイル/リンク (NVIDIA CUDA SDK)

スレッド並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*nvcc* コマンドを使用します。

```
$ nvcc thread-option [option] file [...]
```

例1) OpenMP 並列化された CUDA プログラムを Aquarius 計算ノード (GPU) 向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvcc -O3 -Xcompiler -fopenmp cuda_omp.cu
```

4.5.3. MPI 並列プログラムのコンパイル/リンク (NVIDIA CUDA SDK + OpenMPI)

MPI 並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合は、*nvcc* コマンドを使用し、MPI のインクルードファイルやライブラリのパスをオプションで指定します。

```
$ nvcc [option] file [...] -lmpi
```

例1) MPI 並列化された CUDA プログラムを Aquarius 計算ノード (GPU) 向けにコンパイル/リンク

```
[username@wisteria01 work]$ nvcc -O3 -I$OMPI_INC -L$LIBRARY_PATH cuda_mpi.cu -lmpi
```

4.6. GCC のコンパイル/リンク概要

コンパイル/リンクのコマンド一覧は以下のとおりです。

表 4-17 GCC のコンパイル/リンクコマンド一覧

種別	言語処理系	コマンド	最適化オプション ^{注1}	OpenMP ^{注2}
逐次 (非 MPI)	Fortran	gfortran	-O3 -march=native	-fopenmp
	C	gcc		
	C++	g++		
MPI 並列 ^{注3}	Fortran	mpifort		
	C	mpicc		
	C++	mpic++		

注 1: Aquarius 計算ノード向けに最適化されたロードモジュールを作成するための推奨オプションです。

注 2: OpenMP オプションはデフォルトでは無効です。

注 3: MPI 並列を使用する場合は OpenMPI ライブラリの module のロードが必要です。

4.6.1. 逐次プログラムのコンパイル/リンク (GCC)

逐次プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*gfortran/gcc/g++* コマンドを使用します。

```
$ gfortran/gcc/g++ [option] file [...]
```

例1) 逐次 Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ gfortran -O3 -march=native seq.f
```

例2) 逐次 C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ gcc -O3 -march=native seq.c
```

4.6.2. スレッド並列プログラムのコンパイル/リンク (GCC)

スレッド並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合、*gfortran/gcc/g++* コマンドを使用します。

```
$ gfortran/gcc/g++ thread-option [option] file [...]
```

例1) OpenMP Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ gfortran -O3 -march=native -fopenmp omp.f
```

例2) OpenMP C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ gcc -O3 -march=native -fopenmp omp.c
```

4.6.3. MPI 並列プログラムのコンパイル/リンク (GCC + OpenMPI)

MPI 並列プログラムを Aquarius 計算ノード向けにコンパイル/リンクする場合は、*mpifort/mpicc/mpic++* コマンドを使用します。

```
$ mpifort/mpicc/mpic++ [option] file [...]
```

例1) MPI Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifort -O3 -march=native mpi.f
```

例2) MPI C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpicc -O3 -march=native mpi.c
```

例3) ハイブリッド並列(MPI+OpenMP)Fortran プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpifort -O3 -march=native -fopenmp mpi_omp.f
```

例4) ハイブリッド並列(MPI+OpenMP)C プログラムを Aquarius 計算ノード向けにコンパイル/リンク

```
[username@wisteria01 work]$ mpicc -O3 -march=native -fopenmp mpi_omp.c
```

4.7. ライブラリ

4.7.1. Fujitsu Development Studio の数学ライブラリ

Fujitsu Development Studio の開発環境では、富士通独自の数学ライブラリ(SSL II および C-SSL II)に加えて、BLAS、LAPACK、ScaLAPACK、および高速4倍精度基本演算ライブラリ(fast_dd)を提供します。これらの数学ライブラリを利用する場合は、コンパイル/リンク時に以下のオプションを指定します。

表 4-18 BLAS/LAPACK/ScaLAPACK オプション一覧

ライブラリ	並列性	指定オプション	備考
SSL II	逐次/スレッド並列共通	-SSL2	Fortran 用
C-SSL II	逐次/スレッド並列共通	-SSL2	C/C++用
SSL II /MPI	MPI 並列	-Kopenmp -SSL2MPI -SSL2	Fortran 用
BLAS/LAPACK	逐次	-SSL2	Fortran/C/C++用
	スレッド並列	-SSL2BLAMP	Fortran/C/C++用
ScaLAPACK	MPI 並列	-SCALAPACK -SSL2	Fortran 用
fast_dd	逐次/スレッド並列共通	-SSL2	Fortran/C/C++用

例1) Fortran プログラムから逐次版 SSL II を利用する。

```
[username@wisteria01 work]$ frtpx -SSL2 sample.f
```

例2) C プログラムから逐次版 C-SSL II を利用する。

```
[username@wisteria01 work]$ fccpx -SSL2 sample.c
```

例3) Fortran プログラムから逐次版 BLAS/LAPACK を利用する。

```
[username@wisteria01 work]$ frtpx -SSL2 sample.f
```

例4) OpenMP Fortran プログラムからスレッド並列版 BLAS/LAPACK を利用する。

```
[username@wisteria01 work]$ frtpx -Kopenmp -SSL2BLAMP sample.f
```

例5) Fortran プログラムから ScaLAPACK を利用する(逐次版 BLAS/LAPACK をリンク)。

```
[username@wisteria01 work]$ mpifrtpx -SCALAPACK -SSL2 sample.f
```

例6) Fortran プログラムから ScaLAPACK を利用する(スレッド版 BLAS/LAPACK をリンク)。

```
[username@wisteria01 work]$ mpifrtpx -Kopenmp -SCALAPACK -SSL2BLAMP sample.f
```

4.7.2. インテル MKL

インテル MKL は、主要な計算分野における包括的な機能を提供します。

- BLAS(レベル 1、2、3) および LAPACK 線形代数ルーチン
- ScaLAPACK 分散処理線形代数ルーチン、BLACS および PBLAS
- インテル MKL PARDISO、反復法スパースソルバー、スパース BLAS(レベル 1、2、3)
- 高速フーリエ変換(FFT)関数
- ベクトルマス(VM)ルーチン
- ベクトル統計(VS)ルーチン
- データ・フィッティング・ライブラリ
- 拡張固有値ソルバ
- C 言語インターフェイスのディープ・ニューラル・ネットワーク(DNN)プリミティブ関数

BLAS/LAPACK/ScaLAPACK を利用するには、コンパイル/リンク時に以下のオプションを指定します。

表 4-19 BLAS/LAPACK/ScaLAPACK オプション一覧

ライブラリ	並列性	指定オプション	備考
BLAS/LAPACK	逐次	-qmkl=sequential	
	スレッド並列	-qmkl=parallel	
ScaLAPACK	MPI 並列	-qmkl=cluster	

例1) Fortran プログラムから逐次版 BLAS/LAPACK を利用する。

```
[username@wisteria01 work]$ ifort -qmkl=sequential sample.f
```

例2) Fortran プログラムからスレッド並列版 BLAS/LAPACK を利用する。

```
[username@wisteria01 work]$ ifort -qmkl=parallel sample.f
```

例3) Fortran プログラムから ScaLAPACK を利用する(逐次版 BLAS/LAPACK をリンク)。

```
[username@wisteria01 work]$ mpiifort -qmkl=cluster sample.f
```

上記以外の組み合わせについては、下記 URL を参照してください。

<https://software.intel.com/en-us/articles/intel-mkl-link-line-advisor>

4.7.3. FFTW

Fortran/C プログラムにて FFTW(<http://www.fftw.org/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

FFTW(Odyssey 用、逐次、スレッド並列)の環境設定（事前に Fujitsu Development Studio の *module* ロードが必要）

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load fftw
```

FFTW(Odyssey 用、MPI 並列)の環境設定（事前に Fujitsu Development Studio のコンパイラ、MPI の *module* ロードが必要）

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load fjmp  
[username@wisteria01 work]$ module load mpi-fftw
```

FFTW(Aquarius 用、逐次、スレッド並列)の環境設定（事前に Intel コンパイラの *module* ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load fftw
```

FFTW(Aquarius 用、MPI 並列)の環境設定（事前に Intel コンパイラ、Intel MPI の *module* ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load impi  
[username@wisteria01 work]$ module load mpi-fftw
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

FFTW ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-20 FFTW (Odyssey 用) リンクオプション一覧

利用ライブラリ	精度	並列性	指定オプション	備考
FFTW (Odyssey 用)	単精度	逐次	-lfftw3f	
		スレッド並列	-lfftw3f_threads	同時に-lfftw3f を指定する。
		MPI 並列	-lfftw3f_mpi	同時に-lfftw3f を指定する。
	倍精度	逐次	-lfftw3	
		スレッド並列	-lfftw3_threads	同時に-lfftw3 を指定する。
		MPI 並列	-lfftw3_mpi	同時に-lfftw3 を指定する。

表 4-21 FFTW (Aquarius 用) リンクオプション一覧

利用ライブラリ	精度	並列性	指定オプション	備考
FFTW (Aquarius 用)	単精度	逐次	-lfftw3f	
		スレッド並列	-lfftw3f_omp	同時に-lfftw3f を指定する。
		MPI 並列	-lfftw3f_mpi	同時に-lfftw3f を指定する。
	倍精度	逐次	-lfftw3	
		スレッド並列	-lfftw3_omp	同時に-lfftw3 を指定する。
		MPI 並列	-lfftw3_mpi	同時に-lfftw3 を指定する。

例1) Odyssey 計算ノード向けに Fortran プログラムから逐次版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ frtpx sample.f -lfftw3
```

例2) Odyssey 計算ノード向けに C プログラムからスレッド並列版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ fccpx -Kopenmp sample.c -lfftw3_threads -lfftw3
```

例3) Odyssey 計算ノード向けに C プログラムから MPI 並列版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ mpifccpx sample.c -lfftw3_mpi -lfftw3
```

例4) Aquarius 計算ノード向けに Fortran プログラムから逐次版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ ifort sample.f -lfftw3
```

例5) Aquarius 計算ノード向けに C プログラムからスレッド並列版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ icc -qopenmp sample.c -lfftw3_omp -lfftw3
```

例6) Aquarius 計算ノード向けに C プログラムから MPI 並列版 FFTW(倍精度)を利用する。

```
[username@wisteria01 work]$ mpiicc sample.c -lfftw3_mpi -lfftw3
```

4.7.4. GSL

C プログラムにて GSL(<https://www.gnu.org/software/gsl/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

GSL(Odyssey 用)の環境設定（事前に Fujitsu Development Studio の *module* ロードが必要）

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load gsl
```

GSL(Aquarius 用)の環境設定（事前に Intel コンパイラの *module* ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load gsl
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

GSL ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-22 GSL（Odyssey/Aquarius 共通）リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
GSL	逐次	-lgsl -lgslcblas	

例1) Odyssey 計算ノード向けに C プログラムから GSL を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lgsl -lgslcblas
```

例2) Odyssey 計算ノード向けに C プログラムから GSL および SSL II を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lgsl -SSL2
```

例3) Aquarius 計算ノード向けに C プログラムから GSL を利用する。

```
[username@wisteria01 work]$ icc -I$GSL_INC/gsl sample.c -lgsl -lgslcblas
```

例4) Aquarius 計算ノード向けに C プログラムから GSL および MKL を利用する。

```
[username@wisteria01 work]$ icc -I$GSL_INC/gsl sample.c -lgsl -qmkl
```

4.7.5. SuperLU/SuperLU_MT/SuperLU_DIST

Fortran/C プログラムにて SuperLU/SuperLU_MT (pthread 版) /SuperLU_DIST (<http://crd-legacy.lbl.gov/~xiaoye/SuperLU/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

SuperLU (Odyssey用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load superlu
```

SuperLU_MT (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load superlu_mt
```

SuperLU_DIST (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fjmp
[username@wisteria01 work]$ module load superlu_dist
```

SuperLU (Aquarius 用) の環境設定 (事前に Intel コンパイラの module ロードが必要)

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load superlu
```

SuperLU_MT (Aquarius 用) の環境設定 (事前に Intel コンパイラの module ロードが必要)

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load superlu_mt
```

SuperLU_DIST (Aquarius 用) の環境設定 (事前に Intel コンパイラ、MPI の module ロードが必要)

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load impi
[username@wisteria01 work]$ module load superlu_dist
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

SuperLU/SuperLU_MT/SuperLU_DIST ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-23 SuperLU/SuperLU_MT/SuperLU_DIST (Odyssey 用) リンクオプション一覧

利用ライブラリ	指定オプション	備考
SuperLU	-lsuperlu	同時に-SSL2 を指定する。
SuperLU_MT	-lsuperlu_mt	同時に-SSL2BLAMP を指定する。
SuperLU_DIST	-lsuperlu_dist	ParMETIS の利用環境も設定する。同時に-SCALAPACK -SSL2 -lparmetis -lmetis を指定する。

表 4-24 SuperLU/SuperLU_MT/SuperLU_DIST (Aquarius 用) リンクオプション一覧

利用ライブラリ	指定オプション	備考
SuperLU	-lsuperlu	同時に-qmkl=sequential を指定する。
SuperLU_MT	-lsuperlu_mt	同時に-qmkl=sequential を指定する。
SuperLU_DIST	-lsuperlu_dist	ParMETIS の利用環境も設定する。同時に-qmkl=cluster -lparmetis -lmetis を指定する。

例1) Odyssey 計算ノード向けに C プログラムから SuperLU を利用する。

```
[username@wisteria01 work]$ fccpx -SSL2 sample.c -lsuperlu
```

例2) Odyssey 計算ノード向けに C プログラムから SuperLU_MT を利用する。

```
[username@wisteria01 work]$ fccpx -SSL2BLAMP sample.c -lsuperlu_mt -lpthread
```

例3) Odyssey 計算ノード向けに C プログラムから SuperLU_DIST 利用する。

```
[username@wisteria01 work]$ mpifccpx -SCALAPACK -SSL2 sample.c -lsuperlu_dist -lparmetis -lmetis
```

例4) Aquarius 計算ノード向けに C プログラムから SuperLU を利用する。

```
[username@wisteria01 work]$ icc -qmkl=sequential sample.c -lsuperlu
```

例5) Aquarius 計算ノード向けに C プログラムから SuperLU_MT を利用する。

```
[username@wisteria01 work]$ icc -qmkl=sequential sample.c -lsuperlu_mt -lpthread
```

例6) Aquarius 計算ノード向けに C プログラムから SuperLU_DIST 利用する。

```
[username@wisteria01 work]$ mpiicc -qmkl=cluster sample.c -lsuperlu_dist -lparmetis -lmetis
```

4.7.6. METIS/ParMETIS

Fortran/C プログラムにて METIS/ParMETIS (<http://glaros.dtc.umn.edu/gkhome/views/metis/>)を利用することが可能です。利用するためには、事前に `module` コマンドを用いて利用環境を設定する必要があります。ログインノードで `module` コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで `module` コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

METIS (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load metis
```

ParMETIS (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fjmpi
[username@wisteria01 work]$ module load metis
[username@wisteria01 work]$ module load parmetis
```

METIS (Aquarius 用) の環境設定 (事前に Intel コンパイラの module ロードが必要)

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load metis
```

ParMETIS (Aquarius 用) の環境設定 (事前に Intel コンパイラ、MPI の module ロードが必要)

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load impi
[username@wisteria01 work]$ module load metis
[username@wisteria01 work]$ module load parmetis
```



ジョブ実行時にも `module` コマンドによる利用環境設定が必要です。

METIS/ParMETIS ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-25 METIS/ParMETIS (Odyssey/Aquarius 共通) リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
METIS	逐次	-lmetis	
ParMETIS	MPI 並列	-lparmetis	同時に -lmetis を指定する。

例1) Odyssey 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lmetis
```

例2) Odyssey 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@wisteria01 work]$ mpifccpx sample.c -lparmetis -lmetis
```

例3) Aquarius 計算ノード向けに C プログラムから METIS を利用する。

```
[username@wisteria01 work]$ icc sample.c -lmetis
```

例4) Aquarius 計算ノード向けに C プログラムから ParMETIS を利用する。

```
[username@wisteria01 work]$ mpiicc sample.c -lparmetis -lmetis
```

4.7.7. Scotch/PT-Scotch

Fortran/C プログラムにて Scotch/PT-SCotch(<http://www.labri.fr/perso/pelegrin/scotch/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

Scotch (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load scotch
```

PT-Scotch (Odyssey 用) の環境設定 (事前に Fujitsu Development Studio の module ロードが必要)

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load fmpi  
[username@wisteria01 work]$ module load scotch  
[username@wisteria01 work]$ module load pt-scotch
```

Scotch (Aquarius 用) の環境設定 (事前に Intel コンパイラの module ロードが必要)

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load scotch
```

PT-Scotch (Aquarius 用) の環境設定 (事前に Intel コンパイラ、MPI の module ロードが必要)

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load impi  
[username@wisteria01 work]$ module load scotch  
[username@wisteria01 work]$ module load pt-scotch
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Scotch/PT-Scotch ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-26 Scotch/PT-Scotch (Odyssey/Aquarius 共通) リンクオプション一覧

利用ライブラリ	並列性	指定オプション	備考
Scotch	逐次	-lscotch -lscotcherr	
PT-Scotch	MPI 並列	-lptscotch -lscotch -lptscotcherr	

例1) Odyssey 計算ノード向けに C プログラムから Scotch を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lscotch -lscotcherr
```

例2) Odyssey 計算ノード向けに C プログラムから PT-Scotch を利用する。

```
[username@wisteria01 work]$ mpifccpx sample.c -lptscotch -lscotch -lptscotcherr
```

例3) Aquarius 計算ノード向けに C プログラムから Scotch を利用する。

```
[username@wisteria01 work]$ icc sample.c -lscotch -lscotcherr
```

例4) Aquarius 計算ノード向けに C プログラムから PT-Scotch を利用する。

```
[username@wisteria01 work]$ mpiicc sample.c -lptscotch -lscotch -lptscotcherr
```

4.7.8. HDF5/Parallel HDF5

Fortran/C/C++プログラムにて HDF5 (<http://www.hdfgroup.org/HDF5/>)を利用することが可能です。また、Fortran/C プログラムにて Parallel HDF5 を利用することが可能です。利用するためには、事前に module コマンドを用いて利用環境を設定する必要があります。ログインノードで module コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで module コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

HDF5（Odyssey 用）の環境設定（事前に Fujitsu Development Studio の module ロードが必要）

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load hdf5
```

Parallel HDF5（Odyssey 用）の環境設定（事前に Fujitsu Development Studio の module ロードが必要）

```
[username@wisteria01 work]$ module load fj  
[username@wisteria01 work]$ module load fjmp  
[username@wisteria01 work]$ module load hdf5  
[username@wisteria01 work]$ module load phdf5
```

HDF5（Aquarius 用）の環境設定（事前に Intel コンパイラの module ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load hdf5
```

Parallel HDF5（Aquarius 用）の環境設定（事前に Intel コンパイラ、MPI の module ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load impi  
[username@wisteria01 work]$ module load hdf5  
[username@wisteria01 work]$ module load phdf5
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

HDF5, Parallel HDF5 ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-27 HDF5, Parallel HDF5 (Odyssey/Aquarius 共通) リンクオプション一覧

利用ライブラリ	並列性	使用コンパイラ	指定オプション	備考
HDF5	逐次	C	-lhdf5	
			-lhdf5_hl	high level library
		C++	-lhdf5_cpp	
			-lhdf5_hl_cpp	high level library
		Fortran	-lhdf5_fortran	
			-lhdf5hl_fortran	high level library
PHDF5	MPI 並列	C	-lhdf5	
			-lhdf5_hl	high level library
		Fortran	-lhdf5_fortran	
			-lhdf5hl_fortran	high level library

例1) Odyssey 計算ノード向けに C プログラムから HDF5 を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lhdf5
```

例2) Odyssey 計算ノード向けに C++ プログラムから HDF5 を利用する。

```
[username@wisteria01 work]$ FCCpx sample.c -lhdf5_cpp -lhdf5
```

例3) Odyssey 計算ノード向けに Fortran プログラムから Parallel HDF5 を利用する。

```
[username@wisteria01 work]$ mpiifrtpx sample.f90 -lhdf5_fortran -lhdf5
```

例4) Aquarius 計算ノード向けに C プログラムから HDF5 を利用する。

```
[username@wisteria01 work]$ icc sample.c -lhdf5
```

例5) Aquarius 計算ノード向けに C++ プログラムから HDF5 を利用する。

```
[username@wisteria01 work]$ icpc sample.c -lhdf5_cpp -lhdf5
```

例6) Aquarius 計算ノード向けに Fortran プログラムから Parallel HDF5 を利用する。

```
[username@wisteria01 work]$ mpiifort sample.f90 -lhdf5_fortran -lhdf5
```

4.7.9. netCDF

Fortran/C/C++プログラムにて netCDF(<http://www.unidata.ucar.edu/software/netcdf/>)を利用することが可能です。利用するためには、事前に `module` コマンドを用いて利用環境を設定する必要があります。ログインノードで `module` コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで `module` コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。なお、netCDF 利用時には、HDF5 の利用環境の設定が必要です。HDF5 の設定の詳細については、4.7.8 を参照してください。

netCDF(Odyssey 用、C 言語版)の環境設定（事前に Fujitsu Development Studio の `module` ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load netcdf
```

netCDF(Odyssey 用、Fortran 版)の環境設定（事前に Fujitsu Development Studio の `module` ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load netcdf-fortran
```

netCDF(Odyssey 用、C++版)の環境設定（事前に Fujitsu Development Studio の `module` ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load netcdf-cxx
```

netCDF 並列版(Odyssey 用、MPI 並列版)の環境設定（事前に Fujitsu Development Studio の `module` ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fjmp
[username@wisteria01 work]$ module load netcdf-parallel
[username@wisteria01 work]$ module load phdf5
```

netCDF(Aquarius 用、C 言語版)の環境設定（事前に Intel コンパイラの `module` ロードが必要）

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load netcdf
```

netCDF(Aquarius 用、Fortran 版)の環境設定（事前に Intel コンパイラの `module` ロードが必要）

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load netcdf-fortran
```

netCDF(Aquarius 用、C++版)の環境設定（事前に Intel コンパイラの module ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load netcdf-cxx
```

netCDF 並列版(Aquarius 用、MPI 並列版)の環境設定（事前に Intel コンパイラ、MPI の module ロードが必要）

```
[username@wisteria01 work]$ module load intel  
[username@wisteria01 work]$ module load impi  
[username@wisteria01 work]$ module load netcdf-parallel  
[username@wisteria01 work]$ module load phdf5
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

netCDF ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-28 netCDF (Odyssey/Aquarius 共通) リンクオプション一覧

利用ライブラリ	並列性	使用コンパイラ	指定オプション	備考
netCDF	逐次 並列	C	-lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。
		C++	-lnetcdf_c++4 -lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。
		Fortran	-lnetcdf -lnetcdf	同時に-lhdf5_hl -lhdf5 も指定する。

例1) Odyssey 計算ノード向けに C プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ fccpx sample.c -lnetcdf -lhdf5_hl -lhdf5
```

例2) Odyssey 計算ノード向けに Fortran プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ frtpx sample.f -lnetcdf -lnetcdf -lhdf5_hl -lhdf5
```

例3) Odyssey 計算ノード向けに C++プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ FCCpx sample.f90 -lnetcdf_c++4 -lnetcdf -lhdf5_hl -lhdf5
```

例4) Odyssey 計算ノード向けに C プログラムから netCDF 並列版を利用する。

```
[username@wisteria01 work]$ mpifccpx sample.c -lnetcdf -lhdf5_hl -lhdf5
```

例5) Aquarius 計算ノード向けに C プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ icc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

例6) Aquarius 計算ノード向けに Fortran プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ ifort sample.f -lnetcdf -lhdf5_hl -lhdf5
```

例7) Aquarius 計算ノード向けに C++ プログラムから netCDF を利用する。

```
[username@wisteria01 work]$ icpc sample.f90 -lnetcdf_c++4 -lnetcdf -lhdf5_hl -lhdf5
```

例8) Aquarius 計算ノード向けに C プログラムから netCDF 並列版を利用する。

```
[username@wisteria01 work]$ mpiicc sample.c -lnetcdf -lhdf5_hl -lhdf5
```

4.7.10. Parallel netCDF

Fortran/Cプログラムにて Parallel netCDF(<http://trac.mcs.anl.gov/projects/parallel-netcdf/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

Parallel netCDF(Odyssey 用)の環境設定（事前に Fujitsu Development Studio の *module* ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fmpi
[username@wisteria01 work]$ module load pnetcdf
```

Parallel netCDF(Aquarius 用)の環境設定（事前に Intel コンパイラ、MPI の *module* ロードが必要）

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load impi
[username@wisteria01 work]$ module load pnetcdf
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

Parallel netCDF ライブラリをリンク時に指定するオプションは以下のとおりです。

表 4-29 Parallel netCDF（Odyssey/Aquarius 共通）リンクオプション一覧

利用ライブラリ	指定オプション	備考
Parallel netCDF	-lpnetcdf	

例1) Odyssey 計算ノード向けに C プログラムから Parallel netCDF を利用する。

```
[username@wisteria01 work]$ mpifccpx sample.c -lpnetcdf
```

例2) Aquarius 計算ノード向けに C プログラムから Parallel netCDF を利用する。

```
[username@wisteria01 work]$ mpiicc sample.c -lpnetcdf
```

4.7.11. PETSc/PETSc-CPP

C/C++/Fortran コンパイラから PETSc(<http://www.mcs.anl.gov/petsc/>)を利用することが可能です。利用するためには、事前に *module* コマンドを用いて利用環境を設定する必要があります。C++言語を利用する場合は PETSc-CPP を指定します。ログインノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数にヘッダファイルおよびライブラリのサーチパスが自動で設定されます。計算ノードで *module* コマンドを用いて利用環境を設定すると、コンパイル用環境変数に加え、実行用環境変数も自動で設定されます。

PETSc/PETSc-CPP(Odyssey 用)の環境設定（事前に Fujitsu Development Studio の *module* ロードが必要）

```
[username@wisteria01 work]$ module load fj
[username@wisteria01 work]$ module load fjmpi
[username@wisteria01 work]$ module load petsc
```

PETSc/PETSc-CPP (Aquarius 用)の環境設定（事前に Intel コンパイラの *module* ロードが必要）

```
[username@wisteria01 work]$ module load intel
[username@wisteria01 work]$ module load impi
[username@wisteria01 work]$ module load petsc
```



ジョブ実行時にも *module* コマンドによる利用環境設定が必要です。

例 1) KSP の tutorials/ex1.c をコンパイルする場合

- KSP の tutorials をコピー

```
[username@wisteria01 ~]$ cp -r $PETSC_DIR/share/petsc/examples/src/ksp/ksp/tutorials .
```

- ex1.c のコンパイル(Odyssey クロスコンパイラ環境の場合)

```
[username@wisteria01 ~]$ cd tutorials
[username@wisteria01 tutorials]$ export fccpx_ENV=-Nclang
[username@wisteria01 tutorials]$ make PCC_LINKER=mpifccpx ex1
```

- ・ ex1.c のコンパイル(Odyssey オウンコンパイラ環境の場合)

バッチジョブおよびインタラクティブジョブを用いて計算ノードにてコンパイルする必要があります。以下はインタラクティブジョブの場合です。

```
[username@wisteria01 ~]$ cd tutorials
[username@wisteria01 tutorials]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-o,node=8
[INFO] PJM 0000 pjsub Job 12345 submitted.
[INFO] PJM 0081 .connected.
[INFO] PJM 0082 pjsub Interactive job 12345 started.
[username@wo0735 tutorials]$ export fcc_ENV=-Nclang
[username@wo0735 tutorials]$ make ex1
```

- ・ ex1.c のコンパイル(Aquarius の場合)

```
[username@wisteria01 ~]$ cd tutorials
[username@wisteria01 tutorials]$ make ex1
```

5. バッチジョブ・インタラクティブジョブ

5.1. ジョブ管理システム概要

Wisteria/BDEC-01 システムでは、ジョブ管理システムによりバッチジョブおよびインタラクティブジョブの実行が制御されます。ユーザーはジョブ実行に必要なリソースグループ名、ノード数、経過時間などを指定し、ジョブ管理システムにジョブ実行を依頼します。依頼されたジョブはジョブ管理システムにより資源の空き状況やジョブ間の優先度などを元に自動で実行開始されます。

Wisteria/BDEC-01 システムで利用可能なジョブ形式は以下の 2 種類です。

表 5-1 ジョブの形式

ジョブ形式	用途
バッチジョブ	バッチ形式でジョブを実行する。 ノードダウンなどの異常発生時、オプションによりジョブの自動再実行が可能。
インタラクティブジョブ	ユーザー端末からのデータ入力により、会話形式でジョブを実行する。 主にデバッグ等での利用を想定。 ノードダウンなどの異常発生時、ジョブは再実行されない。

バッチジョブは、以下の 3 種類の種別に分類されます。

表 5-2 バッチジョブの種別

バッチジョブ種別	用途	投入形式
通常ジョブ	スクリプト単位でジョブを実行する	「5.4.3.1」を参照
ステップジョブ	投入した複数のジョブを 1 つのまとまりとして扱い、その中で実行順序、依存関係をもつジョブ	「5.4.3.2」を参照
バルクジョブ	1 度のジョブ投入で、複数の通常ジョブを生成し、実行するジョブ	「5.4.3.3」を参照

ジョブ操作のためのコマンドを表 5-3 に示します。

表 5-3 ジョブ操作コマンド一覧

機能	コマンド名
ジョブ投入	pjsub
ジョブ参照	pjstat
ジョブ削除	pjdel
ジョブ保留	pjhold
ジョブ保留の解除	pjrls

5.2. ジョブ実行リソース

ジョブ管理システムは、リソースグループという単位で計算ノードを管理します。ユーザーは、ジョブを投入する際にどのリソースグループでジョブを実行するか指定します。

5.2.1. バッチジョブ用リソースグループ

Wisteria/BDEC-01 で指定可能なリソースグループを表 5-4、表 5-5、表 5-6 に示します。ジョブ投入時にコンプレックスリソースグループ「regular-o」または「regular-a」を指定した場合は、ノード数に応じて自動的にリソースグループが選択されます。

Aquarius のリソースグループは、ノード占有利用と GPU 専有利用の 2 種類が存在します。ノード占有利用とはジョブに割り当てるリソースを 1 ノード単位とする運用、GPU 専有利用とはジョブに割り当てるリソースを GPU 単位とし、残りのリソースを他のジョブが利用可能とする運用です。

表 5-4 Odyssey バッチジョブ用リソースグループ

コンプレックス リソースグループ	リソース グループ	割り当て ノード	最小 ノード数	最大 ノード数	ジョブあたりの バルクサブジョブ数	ジョブあたりの 実行時間制限	ノードあたりの 最大メモリ容量
debug-o		384	1	144	利用不可	30 分	28 GiB
short-o		288	1	72	5	8 時間	
regular-o	small-o	4608	1	144	10	48 時間	
	medium-o		145	576	10	48 時間	
	large-o		577	1152	10	48 時間	
	x-large-o		1153	2304	10	24 時間	
priority-o		384	1	288	10	48 時間	

表 5-5 Aquarius バッチジョブ用リソースグループ（ノード占有利用）

コンプレックス リソースグループ	リソース グループ	割り当て ノード	最小 ノード数	最大 ノード数	ジョブあたりの 実行時間制限	ノードあたりの 最大メモリ容量
debug-a		2	1	1	30 分	448 GiB
interactive-a		2	1	1	10 分	
short-a		4	1	2	2 時間	
regular-a	small-a	25	1	2	48 時間	
	medium-a		3	4	48 時間	
	large-a		5	8	24 時間	
ノード固定		最大 6	1	許可された ノード数	1200 時間	

表 5-6 Aquarius バッチジョブ用リソースグループ（GPU 専有利用）

コンプレックス リソースグループ	リソース グループ	割り当てノード	使用可能 GPU 数	ジョブあたりの 実行時間制限	GPU あたりの 最大メモリ容量
share-debug		1	1,2,4	30 分	56 GiB
share-interactive			1	10 分	
share-short		2	1,2,4	2 時間	
share	share-1	6	1	48 時間	
	share-2	2	2	48 時間	
	share-4	1	4	24 時間	



コンプレックスリソースグループを構成するリソースグループには直接ジョブを投入できません。

申請された基本セットにより表 5-7、表 5-8 に示す各種制限があります。

表 5-7 Odyssey バッチジョブのグループ（プロジェクト）毎の制限

制限	グループコース
ジョブ同時受付制限数	ジョブ同時実行制限数 × 8
ジョブ同時実行制限数	基本セット 2 セットまで 4 本、 4 セットまで 8 本、 8 セットまで 16 本、 以降 8 セット単位で 4 本ずつ追加する。
バルクサブジョブ同時受付制限数	80
バルクサブジョブ同時実行制限数	ジョブ同時実行制限数と同じ
ノード同時使用制限数	256
ジョブ 1 本あたりの最大ノード数	256

表 5-8 Aquarius バッチジョブのグループ（プロジェクト）毎の制限

制限	グループコース
ジョブ同時受付制限数	ジョブ同時実行制限数 × 2
ジョブ同時実行制限数	基本セット 12 セットまで 2 本、 24 セットまで 4 本、 以降 12 セット単位で 1 本ずつ追加する。 上限は、10 とする。
GPU 同時使用制限数 ^注	64
ジョブ 1 本あたりの最大 GPU 数 ^注	64
バルクジョブ制限	利用禁止

注：ノード占有利用の場合は 1 ノードあたり 8GPU 使用するものとして計算します。

5.2.2. インタラクティブジョブ用リソースグループ

インタラクティブジョブ実行時に指定可能なリソースグループを表 5-9、表 5-10 に示します。バッチジョブと異なり、利用区分によらず利用できるリソースグループは同一です。

インタラクティブジョブ用リソースグループは以下の特徴を有します。

- インタラクティブジョブとして利用可能です。
- ジョブを実行してもトークンが消費されません。
- 所属プロジェクトの数によらず、1 ユーザーあたり 1 本のみジョブの投入・実行が可能です。

表 5-9 Odyssey インタラクティブジョブ用リソースグループ

コンプレックス リソースグループ	リソース グループ	割り当て ノード	最小 ノード 数	最大 ノード 数	ジョブあたりの 実行時間制限	ノードあたりの 最大メモリ容量
interactive-o	interactive-o_n1	96	1	1	2 時間	28 GiB
	interactive-o_n12		2	12	10 分	

表 5-10 Aquarius インタラクティブジョブ用リソースグループ

リソース グループ	運用方式	割り当て ノード	最小 ノード 数	最大 ノード 数	使用可 能 GPU 数	ジョブあたりの 実行時 間制限	ノードあたりの 最大メモリ 容量
interactive-a	ノード占有	2	1	1	N/A	10 分	56 GiB
share-interactive	GPU 専有	1	N/A	N/A	1	10 分	



コンプレックスリソースグループを構成するリソースグループには直接ジョブを投入できません。

5.2.3. プリポストノード用リソースグループ

ログインノードで行う処理のうち負荷が高い処理を実行する場合、プリポスト処理用リソースグループを使用してください。

プリポスト処理用リソースグループは以下の特長を有します。

- インタラクティブジョブとして利用可能です。
- ログインノードと同一の環境で処理を行うことができます。
- ジョブを実行してもトークンは消費されません。
- 所属プロジェクトの数によらず、1 ユーザーあたり 1 本のみジョブの投入・実行が可能です。

表 5-11 プリポスト処理用リソースグループ

リソースグループ	指定可能ノード数	実行時間制限	ノードあたり 最大メモリ容量	備考
prepost	1	6 時間	338GiB	

5.3. ジョブ投入オプション

ジョブ投入時は、ジョブの実行に応じて、以下の 3 つのオプションを指定します。

- 基本オプション
- ジョブ資源オプション
- MPI オプションおよび OpenMP オプション

5.3.1. 基本オプション

ジョブ投入時に指定する主な基本オプションを表 5-12 に示します。**赤字**のオプションは、必ず指定する必要があります。

表 5-12 主なジョブ投入基本オプション (Odyssey/Aquarius 共通)

オプション	説明
-g <projectcode>	ジョブ実行時にトークンを消費するプロジェクトを指定
--fs <filesystem>[, <filesystem>] ^{注1}	使用するファイルシステムを指定(ファイルシステム状態が異常の場合、ジョブの実行が抑止される。ファイルシステムが復旧したら、抑止ジョブが再開される。)
-o <filename>	ジョブの標準出力を指定されたファイルに出力
-e <filename>	ジョブの標準エラー出力を指定されたファイルに出力
-j	ジョブの標準エラー出力を標準出力へ出力
-N <jobname>	ジョブ名を指定
--interact	インタラクティブジョブとしてジョブを投入
--step	ステップジョブとしてジョブを投入
jid=<jobid>	関連付けるジョブ ID を指定
sn=<subjobid>	自サブジョブ番号を指定
sd=<form>	依存関係式を指定
--bulk -sparam <start>--<end>	バルクジョブとしてジョブを投入
--at	ジョブの実行開始時刻を指定する
--restart	障害発生時にジョブを再実行する
--norestart	障害発生時にジョブを再実行しない(システムのデフォルト)
--mail-list	メールの送信先を指定
-m	メール通知を指定
b	ジョブ開始時にメール通知を指定
e	ジョブ終了時にメール通知を指定
r	ジョブ再実行時にメール通知を指定
--notify	バッチジョブが計画停止終了後に実行を開始する予定の場合、メッセージを標準エラー出力に通知
-X	ジョブ投入時の環境変数をジョブ実行環境に継承
-s	ジョブ統計情報をファイルに出力
-S	-s オプションで表示される項目に続いて、ノードごとの詳細情報をファイルに出力

注 1: 引数の *filesystem* には「/work」または「/data」を指定してください。

例) --fs /work,/data

「/work」または「/data」以外の文字列を指定してもエラーにはなりませんが、異常発生時のジョブ実行抑止・解除の対象には含まれませんのでご注意ください。

-o、-e オプションで指定したファイル名、および、-s、-S オプションと--spath オプションで指定したファイル名には以下の表記を使用できます。

表 5-13 ファイル名の表記方法

書式	意味
%j	ジョブ ID
%J	サブジョブ ID
%b	バルク番号
%s	ステップ番号
%n	ジョブ名

5.3.2. ジョブ資源オプション

ジョブが利用する資源に関する主要オプションを表 5-14、表 5-15 に示します。ジョブ資源オプションは-L オプションを用いて指定します。赤文字のオプションは、必ず指定する必要があります。

表 5-14 Odyssey 用ジョブ資源オプション

オプション	説明
-L	ジョブ実行に必要な資源値を指定
rscgrp= <name>	リソースグループ名/コンプレックスリソースグループ名を指定
node= <shape>[:strict]:strict-io[:torus]:mesh[:noncont]	ノードの形状とノード数およびノードの割り当て方法を指定 <shape>は、ノード形状の次元に応じて各軸方向の大きさを指定します。 1 次元指定の表記：node=N1 2 次元指定の表記：node=N1xN2 3 次元指定の表記：node=N1xN2xN3 ノード形状が 3 次元かつ torus 指定の場合、 N1xN2xN3:strict:torus のように strict 指定ができます。これは、通常、空きノード空間に指定したノード形状が収まるように自動的に回転させて割り当てますが、strict 指定の場合は、回転させずに割り当てを試みます。2 次元形状で同様なことをする場合は、3 次元形状で表現して strict 指定を使用ください。 また、strict-io 指定を使用すると、Tofu 座標における I/O ノードの位置が常に同じになるように指定できます。 ジョブがノード専有ジョブの場合、ノードの割り当て方法を以下の 3 種類から指定できます。 torus：トーラスモード（Tofu（12 ノード）を単位としてジョブを割り当てる） mesh：メッシュモード（隣接するノードを単位としてジョブを割り当てる） noncont：離散割り当てモード（隣接しないノードを割り当てる場合がある） 省略時は、regular-o と priority-o はメッシュモード、その他は離散割り当てモードになります。
elapse= <elapselimit>	経過時間制限値を指定（[[hour:]minute:]second で指定） 適切な経過時間を指定することでバックフィル機能(*1)によりジョブの実行開始が早まる可能性があります。 (*1) 先に投入したジョブの実行開始予定を遅らせることなく、空いているリソースにジョブを割り当てて実行する機能
node-mem= <limit>	ノード単位の使用メモリ制限を指定 (デフォルト：0、最大：unlimited)
proc-core= <size>	プロセス単位の最大コアファイルサイズリミットを指定 (デフォルト：0、最大：unlimited)

proc-data=<size>	プロセス単位の最大データセグメントサイズリミットを指定 (デフォルト: unlimited)
proc-stack=<size>	プロセス単位の最大スタックセグメントサイズリミットを指定 (デフォルト: unlimited) RHEL の仕様により unlimited 設定の場合、実際の設定値は 2MiB となります。

表 5-15 Aquarius 用ジョブ資源オプション

オプション	説明
-L	ジョブ実行に必要な資源値を指定
rscgrp=<name>	リソースグループ名/コンプレックスリソースグループ名を指定
node=<num>	ノード数の指定（ノード占有利用の場合）
gpu=<num>	GPU 数の指定（GPU 専有利用の場合） 1GPU あたり CPU コアを 9 コア、最大メモリ使用量として 56GiB 割り当てます。
elapsed=<elapselimit>	経過時間制限値を指定（[[hour:]minute:]second で指定） 適切な経過時間を指定することでバックフィル機能(*1)によりジョブの実行開始が早まる可能性があります。 (*1) 先に投入したジョブの実行開始予定を遅らせることなく、空いているリソースにジョブを割り当てて実行する機能
node-mem=<limite>	ノード単位の使用メモリ制限を指定 (デフォルト : 0、最大: unlimited)
proc-core=<size>	プロセス単位の最大コアファイルサイズリミットを指定 (デフォルト : 0、最大: unlimited)
proc-data=<size>	プロセス単位の最大データセグメントサイズリミットを指定 (デフォルト: unlimited)
proc-stack=<size>	プロセス単位の最大スタックセグメントサイズリミットを指定 (デフォルト: unlimited) RHEL の仕様により unlimited 設定の場合、実際の設定値は 2MiB となります。

例) 8 ノードを使用するジョブの投入

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-o,node=8
[INFO] PJM 0000 pjsub Job 12345 submitted.
[INFO] PJM 0081 .connected.
[INFO] PJM 0082 pjsub Interactive job 12345 started.
[username@wo0735 work]$
```

5.3.3. MPI オプション、OpenMP オプション

MPI 並列ジョブ、OpenMP 並列ジョブを実行する際に指定するオプションを表 5-16、表 5-17 に示します。
赤文字のオプション（MPI 並列 ジョブ は --mpi proc、OpenMP 並列 ジョブ は --omp thread）は、必ず指定する必要があります。省略すると意図しないプロセス数でプログラムが実行される可能性があり、性能劣化を招く恐れがあります。

表 5-16 Odyssey 用 MPI オプション

オプション	説明
--mpi	MPI ジョブの各種パラメタを指定
proc=<num>	プロセス数を指定(複数ノードジョブ、かつ、ノード内に複数プロセスを生成する場合は指定必須)
rank-map-bynode[=rankmap]	ランク番号を 1 ノードに 1 つずつラウンドロビン方式で割付ける
rank-map-bychip[:rankmap]	ランク番号を 1 ノードに N 個ずつ順に割り当てる
rank-map-hostfile=<filename>	ランク番号を割り当てるノードの順序を記入したファイルを filename で指定する
--omp	OpenMP の各種パラメタを指定
thread=<num>	OpenMP スレッド数を指定

表 5-17 Aquarius 用 MPI オプション

オプション	説明
--mpi	MPI ジョブの各種パラメタを指定
proc=<num>	プロセス数を指定(複数ノードジョブ、かつ、ノード内に複数プロセスを生成する場合は指定必須)
rank-map-bynode[=rankmap]	ランク番号を 1 ノードに 1 つずつラウンドロビン方式で割付ける
rank-map-bychip[:rankmap]	ランク番号を 1 ノードに N 個ずつ順に割り当てる
--omp	OpenMP の各種パラメタを指定
thread=<num>	OpenMP スレッド数を指定

5.3.4. Odyssey 用 FEFS クライアントパラメータの変更オプション

Odyssey の計算ノードは搭載メモリ容量が小さいため、できるだけユーザジョブへのメモリ確保を大きくするために、また、I/O 処理の安定性を考慮して FEFS のファイルキャッシュサイズは小さく設定されています。そのため、ジョブのファイル I/O 次第とはなりますが、ファイルキャッシュの競合により、I/O 処理時間が長くなる場合があります。ファイルキャッシュサイズ等のパラメータを変更することにより、I/O 処理時間を短縮できる場合があるため、いくつかの FEFS クライアントパラメータを変更する環境変数 PJM_FEFS_CACHE_MODE を用意しています。環境変数は、*pjsub* コマンドの -x オプションで指定ください。また、本オプションを使用することにより I/O 性能が改善されない場合やこれまで動作していたプログラムが動作しなくなる場合があります。なお、本オプションは、Odyssey でのみ有効です。

表 5-18 Odyssey 用 FEFS クライアントパラメータ変更オプション

オプション	説明
-x PJM_FEFS_CACHE_MODE=<mode>	FEFS クライアントパラメータを変更する mode の設定値の説明は表 5-19 を参照ください。

表 5-19 PJM_FEFS_CACHE_MODE により変更される FEFS クライアントパラメータ

mode	FEFS クライアントパラメータ		
	max_cached_mb	max_dirty_mb	direct_io_pages
1	128	512	0
2	512	64	0
3	1024	64	0

FEFS クライアントパラメータ	説明
max_cached_mb	クライアントで保持する Write/Read で使用するデータキャッシュの最大量です。
max_dirty_mb	クライアントが各 OST に対して保持する Dirty page のデータ量です。Write を行った場合にサーバに未書き込みの状態保持します。
direct_io_pages	以下の場合に DirectI/O で書き込みます。 1) ファイルに対して Write を行う。かつ 2) 書き込みオフセットがページ境界。かつ 3) I/O 長が direct_io_pages で指定した値 * PAGE_SIZE より大きい。かつ 4) I/O 長が PAGE_SIZE の整数倍

5.4. バッチジョブ実行

5.4.1. ジョブスクリプト作成

バッチジョブを投入するためには、実行する計算プログラムとは別に「ジョブスクリプト」を作成します。ジョブスクリプトには利用するリソースグループ、ノード数、経過時間などの資源や利用するプロジェクトを記載したオプションを記述した上で、実行するコマンド列を記載します。

- (1) 先頭行は“#!”に続けて、シェルのパスを指定します。

[記述例]

#!/bin/bash	bash を利用
-------------	----------



ログインシェルが *bash* 以外で、*sh* で記述されたジョブスクリプト内で *module* コマンドを実行する場合は、「#!/bin/sh -l」を指定する必要がある。 *bash* で記述されたジョブスクリプト内で *module* コマンドを実行する場合は、「#!/bin/bash -l」を指定する必要があります。

- (2) 2 行目以降にジョブ投入オプションをディレクティブ“**#PJM**”を用いて指定します。

[記述例]

#PJM -L node=1	ノード数を指定
#PJM -L elapse=1:00:00	経過時間を指定
#PJM -j	標準エラー出力をマージして出力

- (3) ジョブ投入オプションに続けて、実行時の環境変数設定と、プログラム実行を指定します。

[記述例]

./a.out	プログラムを実行
---------	----------

5.4.2. ジョブ環境変数

ジョブスクリプト内で利用可能な環境変数を表 5-20 に示します。

表 5-20 ジョブスクリプト内で参照可能な環境変数（Odyssey/Aquarius 共通）

環境変数	説明
PJM_ENVIRONMENT	バッチジョブでは“BATCH”、会話型ジョブでは“INTERACT”
PJM_JOBID	ジョブ ID
PJM_JOBNAME	ジョブ名
PJM_O_WORKDIR	pjsub コマンド実行時のカレントディレクトリ
PJM_COMMENT	pjsub コマンドの--comment オプションで指定した文字列
PJM_MAILSENDTO	pjsub コマンドの--mail-list オプションで指定したメール送信先ユーザー
PJM_STEPNUM	ステップ番号(ステップジョブのみ設定)
PJM_BULKNUM	バルク番号(バルクジョブのみ設定)
PJM_SUBJOBID	サブジョブ ID(ステップジョブ、バルクジョブのみ設定)
PJM_NODE	pjsub コマンドの -L node オプションで指定された値。Odyssey に投入されたジョブでのみ有効
PJM_VNODE	pjsub コマンドの -L node オプションで指定された値。Aquarius に投入されたジョブでのみ有効
PJM_MPI_PROC	pjsub コマンドの --mpi proc オプションで指定した、MPI プログラムの起動時に生成される最大プロセス数
PJM_PROC_BY_NODE	MPI プログラムが 1 ノードに生成する最大プロセス数

ログインノード上で設定した環境変数は、指示がない限り、ジョブ実行環境には引き継がれません。ジョブ実行環境に引き継ぐためには、*pjsub* コマンドの-X オプションを指定します。

5.4.3. バッチジョブ投入

バッチジョブを投入する場合、*pjsub* コマンドの引数にバッチジョブとして実行するジョブスクリプトを指定します。



バッチジョブ、インタラクティブジョブは、共有ファイルシステム(/work)または高速ファイルシステム(/data)上で投入する必要があります。

5.4.3.1. 通常ジョブ投入

通常ジョブを投入する場合、*pjsub* コマンドの引数にバッチジョブとして実行するジョブスクリプトを指定します。

\$ pjsub [option] [job-script]

- ジョブスクリプトを指定しない場合、標準入力から実行命令を読み込みます。
- ジョブ投入オプションは、ジョブスクリプト内もしくは標準入力に PJM ディレクティブを用いて指定することが可能です。

- ジョブ投入が完了後、ジョブに対して識別用 ID(ジョブ ID)が割り当てられます。

例) 通常ジョブ投入例

```
[username@wisteria01 work]$ pjsub run.sh
[INFO]PJM 0000 pjsub Job 12345 submitted.
```

5.4.3.2. ステップジョブ投入

ステップジョブは、複数のバッチジョブを 1 つのまとまりとして扱い、その中で実行の順序関係や依存関係を指定することで、ジョブチェーン機能を実現するジョブモデルです。ステップジョブは複数サブジョブから構成され、各サブジョブは同時に実行されることはありません。ステップジョブの動作イメージを図 5-1 に示します。

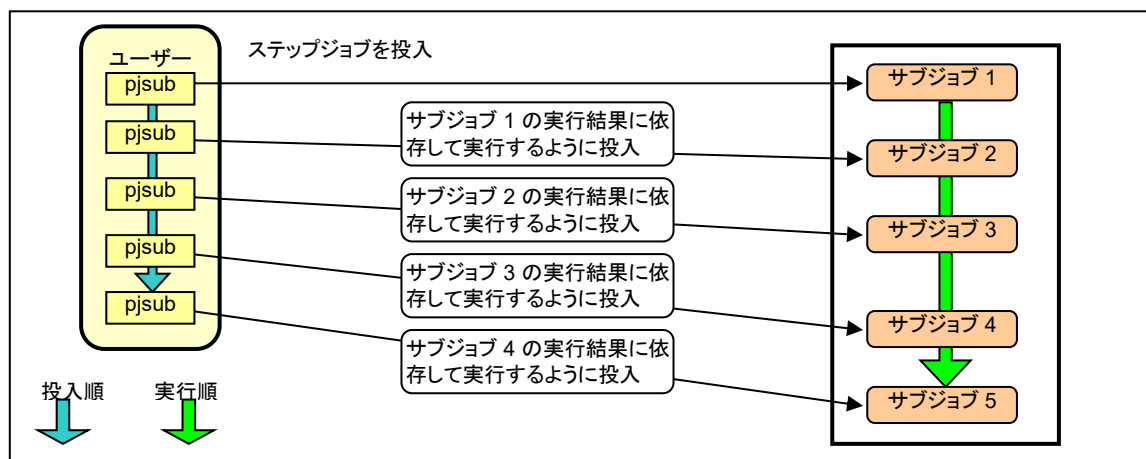


図 5-1 ステップジョブイメージ

ステップジョブの投入形式は以下のとおりです。

```
$ pjsub --step [--sparam "sn=stepno[,依存関係式]"] jobscript
```

表 5-21 ステップジョブ依存関係式

条件	説明
NONE	依存関係がないことを示す
終了ステータス == value[,value,..]	value には任意の値を指定可能
終了ステータス != value[,value,..]	「==」「!=」の場合は","(カンマ)を用いて、value を複数指定可能
終了ステータス > value	例：
終了ステータス >= value	ec==1,3,5 → 終了ステータスが 1,3,5 のいずれかであれば真
終了ステータス < value	ec!=1,3,5 → 終了ステータスが 1,3,5 のいずれでもない場合真
終了ステータス <= value	

表 5-22 ステップジョブ依存関係式で指定可能な削除タイプ

削除タイプ	説明
one	当該ジョブのみを削除します。
after	当該ジョブおよび当該ジョブに依存するジョブを再帰的に削除します。
all	当該ジョブ及び後続のすべてのジョブを削除します。

例1) 3本のサブジョブをもつステップジョブを投入

```
[username@wisteria01 work]$ pjsub --step step1.sh
[INFO]PJM 0000 pjsub Job 12345_0 submitted.
[username@wisteria01 work]$ pjsub --step --sparam jid=12345 step2.sh
[INFO]PJM 0000 pjsub Job 12345_1 submitted.
[username@wisteria01 work]$ pjsub --step --sparam jid=12345 step3.sh
[INFO]PJM 0000 pjsub Job 12345_2 submitted.
```

例2) ステップ番号および依存式関係式を指定して投入

```
[username@wisteria01 work]$ pjsub --step --sparam "sn=1" step1.sh
[INFO]PJM 0000 pjsub Job 12345_1 submitted.
[username@wisteria01 work]$ pjsub --step --sparam ¥
    "jid=12345, sn=2, sd=ec!=0:after:1" step2.sh
[INFO]PJM 0000 pjsub Job 12345_2 submitted.
[username@wisteria01 work]$ pjsub --step --sparam ¥
    "jid=12345, sn=3, sd=ec==0:one:1" step3.sh
[INFO]PJM 0000 pjsub Job 12345_3 submitted.
```

5.4.3.3. バルクジョブ投入

バルクジョブは、複数の同じバッチジョブを同時に投入し、実行するジョブです。例えば、ジョブのパラメタを変えて、それぞれの実行結果を確認したい場合、通常のバッチジョブであれば、ジョブを 1 つ 1 つ投入する必要がありますが、バルクジョブを使えば、1 度に複数パターンの投入ができます。

バルクジョブの投入形式は以下のとおりです。

```
$ pjsub --bulk --sparam start-end jobscript
```

バルクジョブを実行する場合、サブジョブごとに設定されるバルク番号を用いてプログラムの入出力ファイルを変更することが可能です。バルク番号は環境変数 PJM_BULKNUM で参照することが可能です。

例) 8 つのサブジョブが input.1～input.8 の入力ファイルをそれぞれ読み込み、出力ファイル output.1～output.8 をそれぞれ出力する

```
[username@wisteria01 work]$ ls input.*
input.1 input.2 input.3 input.4 input.5 input.6 input.7 input.8
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
./a.out < input.$PJM_BULKNUM > output.$PJM_BULKNUM
[username@wisteria01 work]$ pjsub --bulk --sparam 1-8 run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

5.4.4. バッチジョブの標準出力と標準エラー出力

バッチジョブの標準出力ファイルと標準エラー出力ファイルは、ジョブ投入ディレクトリもしくは、ジョブ投入時に指定されたファイルに出力されます。

標準出力ファイルにはジョブ実行中の標準出力、標準エラー出力ファイルにはジョブ実行中のエラーメッセージが出力されます。ジョブ投入時に、標準出力ファイル、標準エラー出力ファイルを指定しなかった場合は、以下のファイルに出力されます。

- **ジョブ名.XXXXX.out** --- 標準出力ファイル
- **ジョブ名.XXXXX.err** --- 標準エラー出力ファイル
(XXXXX はジョブ投入時に割り当てられたジョブ ID)

ステップ、バルクジョブ実行時の出力

- **ジョブ名.XXXXX[zz].out** --- 標準出力ファイル
- **ジョブ名.XXXXX[zz].err** --- 標準エラー出力ファイル
(XXXXX はジョブ投入時に割り当てられたジョブ ID)
(zz は割り当てられたステップ番号またはバルク番号)

5.4.5. Odyssey 用バッチジョブスクリプト例

Odyssey へのバッチジョブ実行時に自動で富士通コンパイラ(fj)及び富士通 MPI(fjmpi)が load されるため、バッチジョブスクリプト内での記述(*module load fj, module load fjmpi*)は不要です。

5.4.5.1. 単一ノード逐次ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-2 に示します。

- ・リソースグループ : regular-o
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi seq.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
./a.out
```

図 5-2 Odyssey 単一ノード逐次ジョブ用スクリプト

5.4.5.2. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-3 に示します。

- ・リソースグループ : regular-o
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 12 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi para.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=1
#PJM --omp thread=12
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
./a.out
```

図 5-3 Odyssey 単一ノードスレッド並列ジョブ用スクリプト

5.4.5.3. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-4 に示します。

- ・リソースグループ : regular-o
- ・ノード数 : 1 ノード
- ・プロセス数 : 48 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi single-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=1
#PJM --mpi proc=48
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-4 Odyssey 単一ノード MPI 並列ジョブ用スクリプト



MPI プロセス数は *pjsub* コマンドの *--mpi proc* オプションで指定します。

5.4.5.4. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-5 に示します。

・リソースグループ	: regular-o
・ノード数	: 1 ノード
・プロセス数	: 4 プロセス
・1 プロセスあたりのスレッド数	: 12 スレッド
・経過時間	: 1 時間
・課題番号	: group1
・標準エラー出力を標準出力にマージ	: あり

```
[username@wisteria01 work]$ vi single-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=1
#PJM --mpi proc=4
#PJM --omp thread=12
#PJM -L elapse=01:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-5 Odyssey 単一ノードハイブリッド並列ジョブ用スクリプト



ハイブリッド並列ジョブを実行する場合、pjsub コマンドの--mpi proc オプション、--omp thread オプションの指定が必要です。

5.4.5.5. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-6 に示します。

- ・リソースグループ : regular-o
- ・ノード数 : 2 ノード
- ・プロセス数 : 96 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2
#PJM --mpi proc=96
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-6 Odyssey 複数ノード MPI 並列ジョブ用スクリプト



MPI 並列ジョブを実行する場合、pjsub コマンドの--mpi proc オプションを指定してください。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は 48 プロセス (=96 プロセス÷2 ノード) になります。

形状を指定する場合のジョブスクリプトサンプルを図 5-7、図 5-8、図 5-9、図 5-10 に示します。

例) 複数ノード MPI 並列ジョブ (2 次元、メッシュ) 用スクリプト

- ・ノード数 (形状) : 12 ノード (2 次元 : 2x6、メッシュ)
- ・プロセス数 : 576 プロセス

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2x6:mesh
#PJM --mpi proc=576
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-7 Odyssey 複数ノード MPI 並列ジョブ (2 次元メッシュ) 用スクリプト

例) 複数ノード MPI 並列ジョブ (2 次元、トーラス) 用スクリプト

- ・ノード数 (形状) : 12 ノード (2 次元 : 2x6、トーラス)
- ・プロセス数 : 576 プロセス

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2x6:torus
#PJM --mpi proc=576
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-8 Odyssey 複数ノード MPI 並列ジョブ (2 次元トーラス) 用スクリプト

例) 複数ノード MPI 並列ジョブ (2 次元、離散割り当て) 用スクリプト

- ・ノード数 (形状) : 12 ノード (2 次元 : 2x6、離散割り当て)
- ・プロセス数 : 576 プロセス

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2x6:noncont
#PJM --mpi proc=576
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-9 Odyssey 複数ノード MPI 並列ジョブ (2 次元離散割り当て) 用スクリプト

例) 複数ノード MPI 並列ジョブ (3 次元、トーラス) 用スクリプト

- ・ノード数 (形状) : 12 ノード (3 次元 : 2x2x3、トーラス)
- ・プロセス数 : 576 プロセス

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2x2x3:torus
#PJM --mpi proc=576
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-10 Odyssey 複数ノード MPI 並列ジョブ (3 次元トーラス) 用スクリプト

5.4.5.6. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-11 に示します。

・リソースグループ	: regular-o
・ノード数	: 2 ノード
・プロセス数	: 8 プロセス
・1 プロセスあたりのスレッド数	: 12 スレッド
・経過時間	: 1 時間
・課題番号	: group1
・標準エラー出力を標準出力にマージ	: あり

```
[username@wisteria01 work]$ vi multi-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2
#PJM --mpi proc=8
#PJM --omp thread=12
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
mpiexec ./a.out
```

図 5-11 Odyssey 複数ノードハイブリッド並列ジョブ用スクリプト



ハイブリッド並列ジョブを実行する場合、pjsub コマンドの--mpi proc オプション、--omp thread オプションの指定が必要です。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は、4 プロセス（=8 プロセス÷2 ノード）になります。

5.4.6. Aquarius (Intel コンパイラ+MPI) 用バッチジョブスクリプト例

5.4.6.1. 単一ノード逐次ジョブ用スクリプト (ノード占有利用の場合)

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-12 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi seq.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
./a.out
```

図 5-12 Aquarius 単一ノード逐次ジョブ用スクリプト (Intel コンパイラ、ノード占有利用)

5.4.6.2. 単一ノード逐次ジョブ用スクリプト（GPU 専有利用の場合）

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-13 に示します。

- ・リソースグループ : share
- ・GPU 数 : 1GPU
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi seq.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=share
#PJM -L gpu=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
./a.out
```

図 5-13 Aquarius 単一ノード逐次ジョブ用スクリプト（Intel コンパイラ, GPU 専有利用）

5.4.6.3. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-14 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi para.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --omp thread=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
./a.out
```

図 5-14 Aquarius 単一ノードスレッド並列ジョブ用スクリプト (Intel コンパイラ)

5.4.6.4. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-15 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 18 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi single-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --mpi proc=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
module load impi
mpiexec.hydra ./a.out
```

図 5-15 Aquarius 単一ノード MPI 並列ジョブ用スクリプト (Intel コンパイラ+Intel MPI)



MPI 並列プロセス数は、*pjsub* コマンドの *--mpi proc* オプションで指定することも可能です。

5.4.6.5. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-16 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 2 プロセス
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi single-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --mpi proc=2
#PJM --omp thread=18
#PJM -L elapse=01:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
module load impi
mpiexec.hydra ./a.out
```

図 5-16 Aquarius 単一ノードハイブリッド並列ジョブ用スクリプト (Intel コンパイラ+Intel MPI)



ハイブリッド並列ジョブを実行する場合、*pjsub* コマンドの--mpi proc オプション、--omp thread オプションの指定が必要です。

5.4.6.6. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-17 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 2 ノード
- ・プロセス数 : 72 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=2
#PJM --mpi proc=72
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
module load impi
mpiexec.hydra ./a.out
```

図 5-17 Aquarius 複数ノード MPI 並列ジョブ用スクリプト (Intel コンパイラ+Intel MPI)



MPI 並列ジョブを実行する場合、*pjsub* コマンドの *--mpi proc* オプションを指定してください。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は、36 プロセス (=72 プロセス÷2 ノード) になります。

5.4.6.7. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-18 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 2 ノード
- ・プロセス数 : 4 プロセス
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi multi-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=2
#PJM --mpi proc=4
#PJM --omp thread=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load intel
module load impi
mpiexec.hydra ./a.out
```

図 5-18 Aquarius 複数ノードハイブリッド並列ジョブ用スクリプト (Intel コンパイラ+Intel MPI)



ハイブリッド並列ジョブを実行する場合、*pjsub* コマンドの--mpi proc オプション、--omp thread オプションの指定が必要です。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は、2 プロセス (=4 プロセス÷2 ノード) になります。

5.4.7. Aquarius(GCC+OpenMPI)用バッチジョブスクリプト例

5.4.7.1. 単一ノード逐次ジョブ用スクリプト（ノード占有利用）

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-19 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi seq.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
./a.out
```

図 5-19 Aquarius 単一ノード逐次ジョブ用スクリプト（GCC、ノード占有利用）

5.4.7.2. 単一ノード逐次ジョブ用スクリプト(GPU 専有利用)

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-20 に示します。

- ・リソースグループ : share
- ・GPU 数 : 1GPU
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi seq.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=share
#PJM -L gpu=1
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
./a.out
```

図 5-20 Aquarius 単一ノード逐次ジョブ用スクリプト (GCC, GPU 専有利用)

5.4.7.3. 単一ノードスレッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-21 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 1 プロセス(オプション不要)
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi para.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --omp thread=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
./a.out
```

図 5-21 Aquarius 単一ノードスレッド並列ジョブ用スクリプト (GCC)

5.4.7.4. 単一ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-22 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 18 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi single-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --mpi proc=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
module load omp
mpiexec -machinefile $PJM_O_NODEINF -n $PJM_MPI_PROC -npnnode 18 ./a.out
```

図 5-22 Aquarius 単一ノード MPI 並列ジョブ用スクリプト (GCC+OpenMPI)



MPI 並列ジョブを実行する場合、*pjsub* コマンドの *--mpi proc* オプション、*mpiexec* コマンドの *-n* オプションは指定必須です。

5.4.7.5. 単一ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-23 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 1 ノード
- ・プロセス数 : 2 プロセス
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi single-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM --mpi proc=2
#PJM --omp thread=18
#PJM -L elapse=01:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
module load omp
mpiexec -machinefile $PJM_O_NODEINF -n $PJM_MPI_PROC -npnnode 2 ./a.out
```

図 5-23 Aquarius 単一ノードハイブリッド並列ジョブ用スクリプト（GCC+OpenMPI）



ハイブリッド並列ジョブを実行する場合、*pjsub* コマンドの--mpi proc オプション、--omp thread オプションの指定が必要です。

5.4.7.6. 複数ノード MPI 並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-24 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 2 ノード
- ・プロセス数 : 72 プロセス
- ・1 プロセスあたりのスレッド数 : 1 スレッド(オプション不要)
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi multi-mpi.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=2
#PJM --mpi proc=72
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
module load ompi
mpiexec -machinefile $PJM_O_NODEINF -n $PJM_MPI_PROC -npnnode 36 ./a.out
```

図 5-24 Aquarius 複数ノード MPI 並列ジョブ用スクリプト (GCC+OpenMPI)



MPI 並列ジョブを実行する場合、*pjsub* コマンドの *--mpi proc* オプション、*mpiexec* コマンドの *-n* オプションは指定必須です。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は、36 プロセス (=72 プロセス÷2 ノード) になります。

5.4.7.7. 複数ノードハイブリッド並列ジョブ用スクリプト

以下のジョブ実行を想定したジョブスクリプトサンプルを図 5-25 に示します。

- ・リソースグループ : regular-a
- ・ノード数 : 2 ノード
- ・プロセス数 : 4 プロセス
- ・1 プロセスあたりのスレッド数 : 18 スレッド
- ・経過時間 : 1 時間
- ・課題番号 : group1
- ・標準エラー出力を標準出力にマージ : あり

```
[username@wisteria01 work]$ vi multi-hybrid.sh
#!/bin/sh

#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=2
#PJM --mpi proc=4
#PJM --omp thread=18
#PJM -L elapse=1:00:00
#PJM -g group1
#PJM -j

#----- Program execution -----#
module load gcc
module load omp
mpiexec -machinefile $PJM_O_NODEINF -n $PJM_MPI_PROC -npnnode 2 ./a.out
```

図 5-25 Aquarius 複数ノードハイブリッド並列ジョブ用スクリプト (GCC+OpenMPI)



ハイブリッド並列ジョブを実行する場合、*pjsub* コマンドの--mpi proc オプション、--omp thread オプション、*mpiexec* コマンドの-n オプションの指定が必要です。上記スクリプトの場合、1 ノードあたりに生成されるプロセス数は、2 プロセス (=4 プロセス÷2 ノード) になります。

5.5. インタラクティブジョブ実行

インタラクティブジョブを実行するためには、pjsub コマンドに--interact オプションを指定して実行します。ジョブ管理システムがインタラクティブジョブに対して計算資源を割り当てることでジョブの実行が開始され、会話処理が可能になります。

インタラクティブジョブ投入時は、ジョブ投入オプションを引数として指定します。

\$ pjsub --interact [--sparam wait-time=waittime] [option...]

待ち時間を指定することで、資源不足などにより資源がすぐに割り当てられない場合に待機することが可能です（デフォルトの待機時間は 10 秒）。



バッチジョブ、インタラクティブジョブは、共有ファイルシステム(/work)または高速ファイルシステム(/data)上で投入する必要があります。

5.5.1. Odyssey 用インタラクティブジョブ実行例

Odyssey へのインタラクティブジョブ実行時に自動で富士通コンパイラ(fj)及び富士通 MPI(fjmpi)が load されるため、インタラクティブジョブ実行中の *module load fj*, *module load fjmpi* は実行不要です。

例1) Odyssey インタラクティブジョブ逐次実行

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-o,node=1
[INFO] PJM 0000 pjsub Job 12345 submitted.
[INFO] PJM 0081 .connected.
[INFO] PJM 0082 pjsub Interactive job 12345 started.
[username@wo0735 work]$ frt hello_world.f95
[username@wo0735 work]$ ./a.out
Hello world
[username@wo0735 work]$ exit
logout
[INFO] PJM 0083 pjsub Interactive job 12345 completed.
```

例2) Odyssey インタラクティブジョブ単一ノード MPI 並列実行

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥  
-L rscgrp=interactive-o,node=1 --mpi proc=48  
[INFO] PJM 0000 pjsub Job 12345 submitted.  
[INFO] PJM 0081 .connected.  
[INFO] PJM 0082 pjsub Interactive job 12345 started.  
[username@wo0735 work]$ mpifrt hello_world_mpi.f95  
[username@wo0735 work]$ mpiexec -n ${PJM_MPI_PROC} ./a.out  
Hello world from rank 0 process  
(snip)  
Hello world from rank 47 process  
[username@wo0735 work]$ exit  
logout  
[INFO] PJM 0083 pjsub Interactive job 12345 completed.
```

例3) Odyssey インタラクティブジョブ複数ノード MPI 並列実行

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥  
-L rscgrp=interactive-o,node=2 --mpi proc=96  
[INFO] PJM 0000 pjsub Job 12345 submitted.  
[INFO] PJM 0081 .connected.  
[INFO] PJM 0082 pjsub Interactive job 12345 started.  
[username@wo0735 work]$ mpifrt hello_world_mpi.f95  
[username@wo0735 work]$ mpiexec -n ${PJM_MPI_PROC} ./a.out  
Hello world from rank 0 process  
(snip)  
Hello world from rank 95 process  
[username@wo0735 work]$ exit  
logout  
[INFO] PJM 0083 pjsub Interactive job 12345 completed.
```

5.5.2. Aquarius 用インタラクティブジョブ実行例

Aquarius 用のインタラクティブジョブ実行例を以下に記載します。

例1) Aquarius インタラクティブジョブ逐次実行

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥  
-L rscgrp=interactive-a,node=1  
[INFO] PJM 0000 pjsub Job 12345 submitted.  
[INFO] PJM 0081 .connected.  
[INFO] PJM 0082 pjsub Interactive job 12345 started.  
[username@wa01 work]$ module load intel  
[username@wa01 work]$ ifort hello_world.f95  
[username@wa01 work]$ ./a.out  
Hello world  
[username@wa01 work]$ exit  
logout  
[INFO] PJM 0083 pjsub Interactive job 12345 completed.
```

例2) Aquarius インタラクティブジョブ単一ノード MPI 並列実行

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥  
-L rscgrp=interactive-a,node=1 --mpi proc=36  
[INFO] PJM 0000 pjsub Job 12345 submitted.  
[INFO] PJM 0081 .connected.  
[INFO] PJM 0082 pjsub Interactive job 12345 started.  
[username@wa01 work]$ module load intel impi  
[username@wa01 work]$ mpiifort hello_world_mpi.f95  
[username@wa01 work]$ mpiexec -n ${PJM_MPI_PROC} ./a.out  
Hello world from rank 0 process  
(snip)  
Hello world from rank 35 process  
[username@wa01 work]$ exit  
logout  
[INFO] PJM 0083 pjsub Interactive job 12345 completed.
```

5.6. MPI プログラムの実行

5.6.1. Odyssey での MPI プログラムの実行

MPI プログラムを実行する場合、*mpiexec* コマンドを使用します。

\$ mpiexec [global_option] [local_option] 実行モジュール :

Odyssey 向けの *mpiexec* コマンドの主なオプションを表 5-23 に示します。

表 5-23 *mpiexec* コマンドの主なオプション（Odyssey 向け）

オプション	説明
-n <i>proc</i>	MPI プログラムの並列プロセス数を指定します。 設定しない場合、 <i>pjsub</i> の <i>--mpi</i> オプションで指定したプロセス数が設定されます。
-of <i>fname</i>	並列プロセスの標準出力および標準エラー出力をファイル名 <i>fname</i> に出力します。
-of-proc <i>fname</i>	並列プロセスの標準出力および標準エラー出力をプロセス毎に ファイル名 " <i>fname</i> .ランク番号" に出力します。
-oferr <i>fname</i>	並列プロセスの標準エラー出力をファイル名 <i>fname</i> に出力します。
-oferr-proc <i>fname</i>	並列プロセスの標準エラー出力をプロセス毎にファイル名 " <i>fname</i> .ランク番号" に出力しま す。
-ofout <i>fname</i>	並列プロセスの標準出力をファイル名 <i>fname</i> に出力します。
-ofout-proc <i>fname</i>	並列プロセスの標準出力をプロセス毎にファイル名 " <i>fname</i> .ランク番号" に出力します。
-stdin <i>fname</i>	全並列プロセスの標準入力をファイル名 <i>fname</i> から読み込みます。

5.6.1.1. 標準出力／標準エラー出力／標準入力

Odyssey での MPI プログラムの標準出力／標準エラー出力／標準入力の指定方法を示します。Fujitsu Development Studio の *mpiexec* では、標準出力／標準エラー出力をファイルに出力するオプション、ファイルから標準入力を読み込むオプションを用意しています。

- 各並列プロセスと *mpiexec* コマンドの標準出力／標準エラー出力は、通常はジョブ運用ソフトウェアによって生成されるジョブ実行結果ファイル（ジョブ名.ジョブ ID.out／ジョブ名.ジョブ ID.err）に出力されます。
- *mpiexec* コマンドのリダイレクション指定による標準入力は、各並列プロセスの標準入力として使用することはできません。

例1) 標準出力を 1 ファイル（ファイル名 : file_stdout）に出力する場合

```
mpiexec -ofout file_stdout -n 8 ./a.out
```

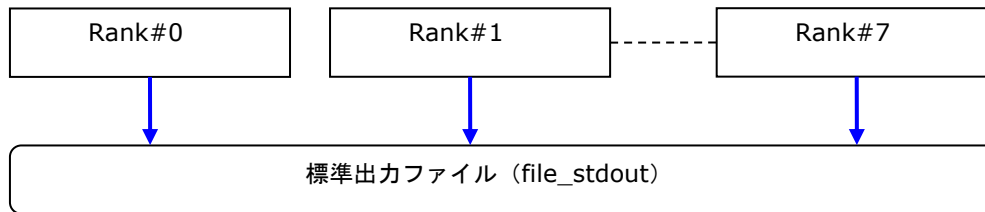


図 5-26 並列プロセスの標準出力を 1 ファイルに出力する場合

例2) 標準出力および標準エラー出力を 1 ファイル（ファイル名 : file_outfile）に出力する場合

```
mpiexec -of file_outfile -n 8 ./a.out
```

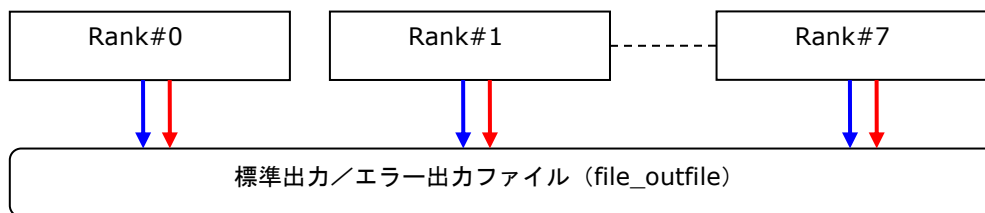


図 5-27 並列プロセスの標準出力および標準エラー出力を 1 ファイルに出力する場合

例3) 標準出力をプロセス毎の別ファイル（ファイル名 : file_stdout.ランク番号）に出力する場合

```
mpiexec -ofout-proc file_stdout -n 8 ./a.out
```

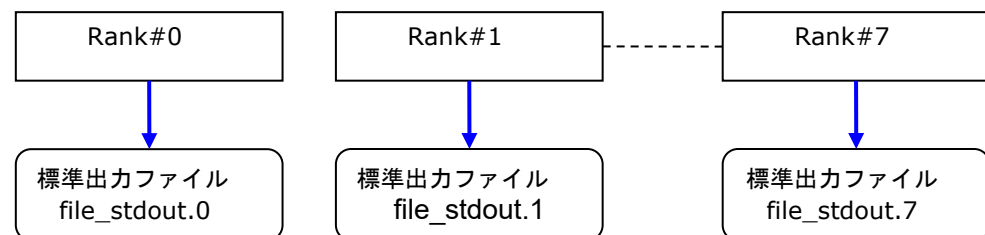


図 5-28 並列プロセスの標準出力を別ファイルに出力する場合

例4) 標準出力および標準エラー出力をプロセス毎の別ファイル（ファイル名：file_outfile.ランク番号）に出力する場合

```
mpiexec -ofout-proc file_stdout -n 8 ./a.out
```

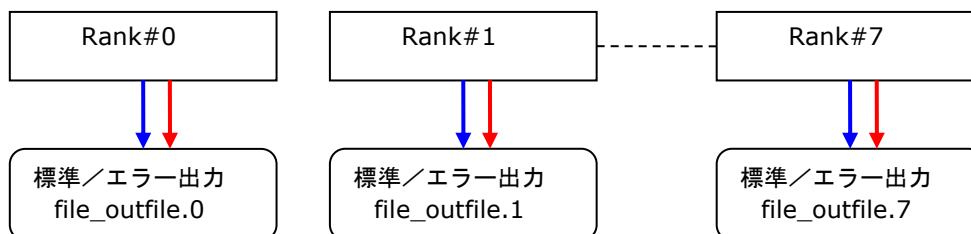


図 5-29 並列プロセスの標準出力および標準エラー出力を別ファイルに出力する場合

5.6.1.2. MPI ジョブ投入時の指定

MPI ジョブ投入時には、pjsub コマンドの--mpi オプションを利用して、起動プロセス形状の指定、ランク割付ルールの指定、起動プロセスの最大値の指定が可能です。

表 5-24 pjsub コマンドの mpi オプションの主な指定方法（Odyssey 向け）

指定方法	指定オプション
起動プロセス形状を指定する	pjsub --mpi shape
起動プロセスの最大数を指定する	pjsub --mpi proc
生成するプロセスのランク割付ルールを指定する	pjsub --mpi rank-map-bynode pjsub --mpi rank-map-hostfile pjsub --mpi rank-map-bychip

5.6.1.3. 静的プロセスの形状指定

pjsub コマンドの--mpi shape オプションを使用することで、静的に起動するプロセスの形状を指定できます。プロセスの形状は、1 次元、2 次元、3 次元の形状で、--rsc-list (-L)の node パラメタで指定するノード形状と同じ次元数を指定する必要があります。--mpi オプションの shape パラメタが省略された場合は、-L オプションで指定された node パラメタの値が使用されます。

shape パラメタ指定例

```
[1 次元形状] --mpi "shape=X"
[2 次元形状] --mpi "shape=XxY"
[3 次元形状] --mpi "shape=XxYxZ"
```

例) 3次元のプロセス形状(X軸 2、Y軸 3、Z軸 2)を指定

```
$ vi sample1.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x3x2:torus         ノード数(3次元形状)
#PJM -L elapse=10:00             経過時間
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

5.6.1.4. 静的プロセスの最大数指定

生成するプロセス数を指定するには、pjsub --mpi proc で指定します。

- --mpi proc で指定可能なプロセス数は、(node 指定値)×48 以下となります。
- --mpi proc を省略した場合は、1 ノードに 1 プロセスを生成します。
- フラット MPI で実行する場合は、本オプションを使用してプロセス数を指定します。4 ノードでフラット MPI を実行する場合は、--mpi proc=192 (4 ノード×48 プロセス)となります。

例) 割当て最大プロセス数(192)を指定

```
$ vi sample2.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=4:mesh              ノード数(1次元形状):メッシュモード
#PJM --mpi proc=192              静的プロセスの最大数 192
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out                  192 プロセスで実行
```

5.6.1.5. MPI ランク割り当て

MPI では、プロセス識別のために、プロセス番号に相当する「ランク」番号を割り当てます。

ランクの割り当てルールは、pjsub コマンドの 以下の--mpi オプションでユーザーが直接指定することも可能です。MPI ランクの割り当て方法は 3 種類です。

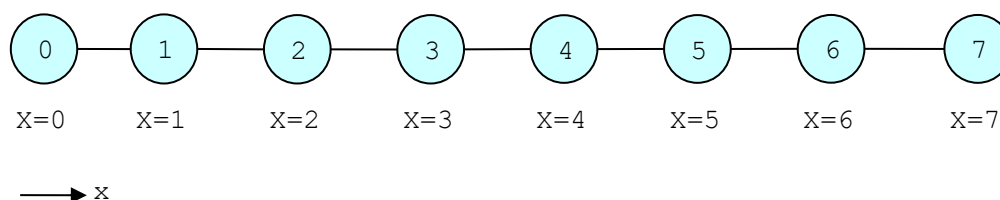
表 5-25 MPI ランク割り当て方法(Odyssey 向け)

オプション名	説明
rank-map-bynode	計算ノードに 1 プロセスを生成すると、次の計算ノードに移動し、ラウンドロビンで自動的に割り付けます。座標の原点をランク 0 とし、rank-map-bynode の先頭文字の軸方向にランクを並べ、上限まで達した時点で、次の文字に移動します。
rank-map-bychip (デフォルト指定)	計算ノードに n プロセスを生成すると、次の計算ノードに移動し、ラウンドロビンで自動的に割り付けます。座標の原点をランク 0 とし、rank-map-bynode の先頭文字の軸方向にランクを並べ、上限まで達した時点で、次の文字に移動します。
rank-map-hostfile	ランクマップファイルに指定された座標を基に、ランクを割り当てます。

5.6.1.6. rank-map-bynode

rank-map-bynode は、ジョブ割り当て時に先頭の計算ノードからラウンドロビンで自動的にランクを割り当てます。

例1) 1 次元形状の rank-map-bynode を指定する



```

$ vi sample3.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=8:mesh              ノード数の指定(1 次元形状)
#PJM --mpi rank-map-bynode        動的 MPI ランク割り当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out

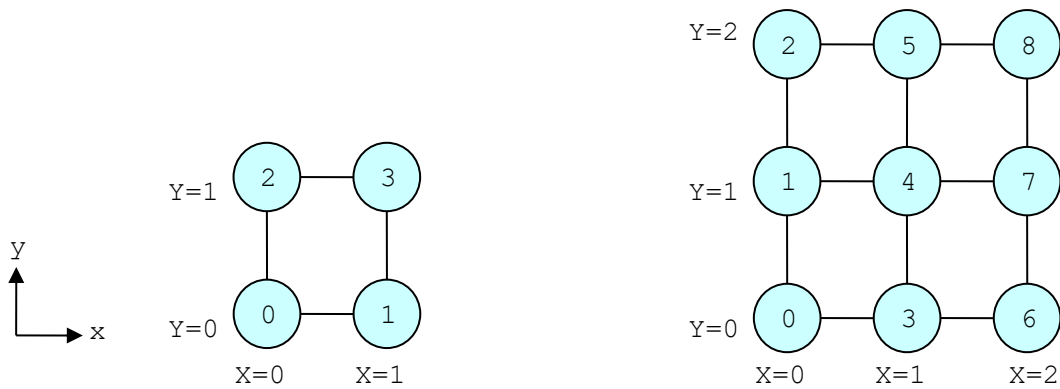
```

図 5-30 ランク割り当て例 (1 次元形状)

例2) 2次元形状のrank-map-bynodeを指定する

--mpi "rank-map-bynode=XY"

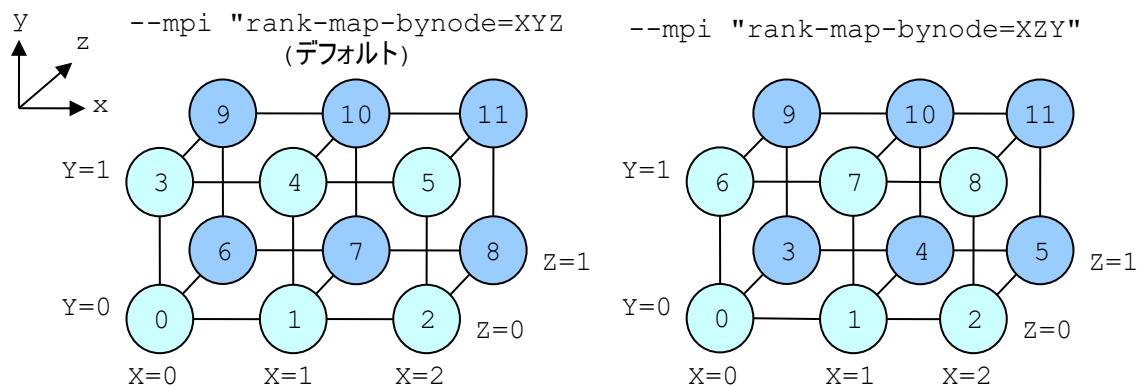
--mpi "rank-map-bynode=YX"



```
$ vi sample4.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x2:mesh            ノード数の指定(2次元形状)
#PJM --mpi rank-map-bynode=XY    動的 MPI ランク割当て
#PJM -L elapse=10:00            経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-31 ランク割り当て例 (2次元形状)

例3) 3次元形状のrank-map-bynodeを指定する

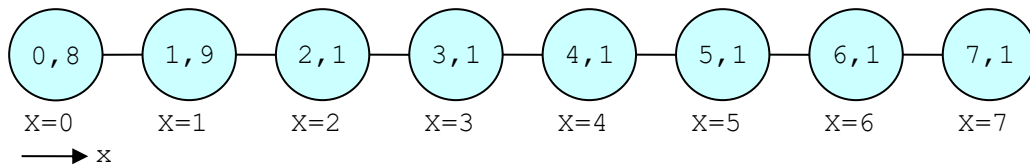


```
$ vi sample5.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o
#PJM -L node=2x3x2:mesh
#PJM --mpi "rank-map-bynode=XYZ"
#PJM -L elapse=10:00
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

リソースグループ指定
ノード数の指定(3次元形状)
動的 MPI ランク割当て
経過時間指定

図 5-32 ランク割り当て例 (3次元形状)

例4) 1次元形状の rank-map-bynode を指定し、ノード内に複数プロセスを起動する



```
$ vi sample6.sh
#!/bin/sh
#----- pjsup option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=8:mesh              ノード数の指定(1次元形状)
#PJM --mpi proc=16               16プロセスを起動
#PJM --mpi rank-map-bynode       静的プロセス形状
#PJM -L elapse=10:00            経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-33 ランク割り当て例（ノード内複数プロセス：1次元形状）

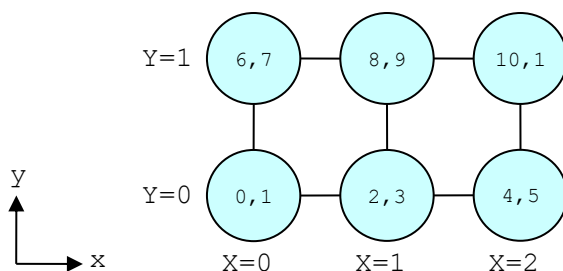
5.6.1.7. rank-map-bychip

rank-map-chip は、計算ノードに指定した n プロセスを生成すると、次の計算ノードに移動しラウンドロビンで自動的にランクを割り付けます。座標の原点をランク 0 とし、rank-map-bychip の先頭文字の軸方向にランクを並べ、上限まで達した時点で、次の文字に移動します。

以下に rank-map-bychip の指定例を示します。

```
#PJM --mpi "rank-map-bychip[:{XY|YX}]" (2 次元)
#PJM --mpi "rank-map-bychip[:{XYZ|XZY|YXZ|YZX|ZXY|ZYX}]" (3 次元)
```

例1) 2 次元形状の rank-map-bychip を指定し、ノード内に複数プロセスを起動する



```
$ vi sample7.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=3x2:mesh            ノード数の指定(2次元形状)
#PJM --mpi proc=12               12 プロセスを起動
#PJM --mpi rank-map-bychip:XY    動的 MPI ランク割当て
#PJM -L elapse=10:00            経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-34 ランク割り当て例（ノード内複数プロセス：2 次元形状）

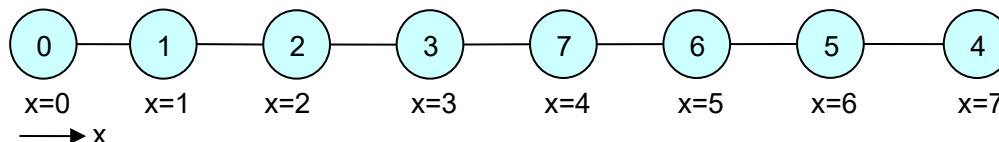
5.6.1.8. rank-map-hostfile

rank-map-hostfile は、ユーザーが指示するホストマップファイルの座標をもとに、ランクを割り当てます。
以下に rank-map-hostfile の指定例を示します

```
$ vi sample8.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x2x2:mesh          ノード数の指定(3次元形状)
#PJM --mpi rank-map-hostfile=rankmap 動的 MPI ランク割当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#----- program execution -----#
mpiexec ./a.out
```

- ファイル rankmap は、pjsub コマンドを実行するカレントディレクトリに配置します。
- ファイル rankmap 内のランクは、ノード形状に合わせて 1 次元、2 次元または 3 次元座標で指定します。
- ファイル rankmap には、1 行に 1 座標を記述し、括弧で囲んで指定します。

例1) 1 次元形状の rank-map-hostfile を指定し、ファイル rankmap に配置を記載する



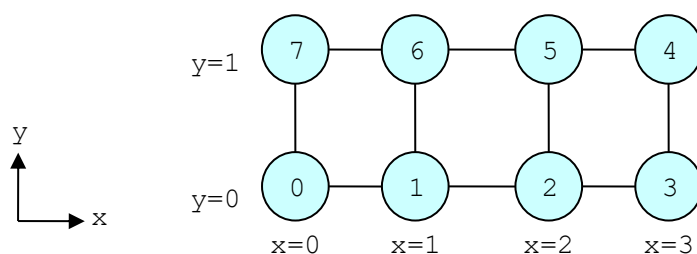
```
$ vi sample9.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=8:mesh              ノード数の指定(1次元形状)
#PJM --mpi rank-map-hostfile=rankmap1 動的 MPI ランク割当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-35 ランク割り当て例 (ホストマップファイル : 1 次元形状)

1 次元座標を指定する場合、ホストマップファイルは(X)を指定します

```
$ vi rankmap1
# ホストマップファイル(1 次元指定例)
(0)                                #rank0
(1)                                #rank1
(2)                                #rank2
(3)                                #rank3
(7)                                #rank4
(6)                                #rank5
(5)                                #rank6
(4)                                #rank7
```

例2) 2 次元形状の rank-map-hostfile を指定し、ファイル rankmap に配置を記載する



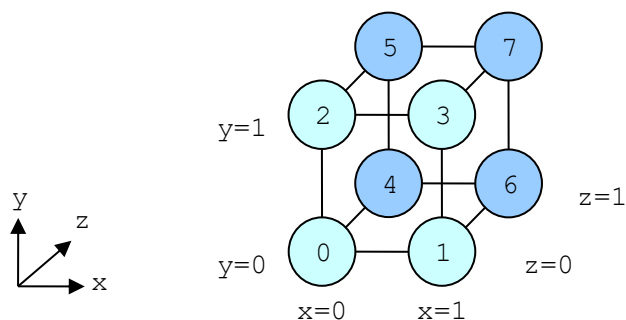
```
$ vi sample10.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L "rscgrp=regular-o"          リソースグループ指定
#PJM -L "node=4x2:mesh"            ノード数の指定(2 次元形状)
#PJM --mpi "rank-map-hostfile=rankmap2"  動的 MPI ランク割当て
#PJM -L "elapsed=10:00"            経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-36 ランク割り当て例 (ホストマップファイル : 2 次元形状)

2次元座標を指定する場合、ホストマップファイルは(X,Y)を指定します。

```
$ vi rankmap2
# ホストマップファイル(2次元指定例)
(0,0)                                #rank0
(1,0)                                #rank1
(2,0)                                #rank2
(3,0)                                #rank3
(3,1)                                #rank4
(2,1)                                #rank5
(1,1)                                #rank6
(0,1)                                #rank7
```

例3) 3次元形状の rank-map-hostfile を指定し、ファイル rankmap に配置を記載する



```
$ vi sample11.sh
#!/bin/sh
#----- pjsup option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x2x2:mesh          ノード数の指定(3次元形状)
#PJM --mpi rank-map-hostfile=rankmap3 動的 MPI ランク割当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

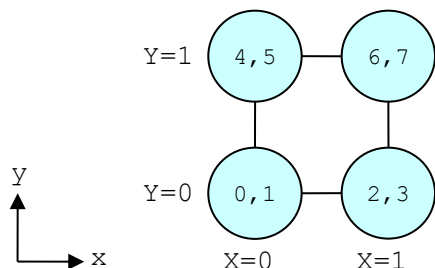
図 5-37 ランク割り当て例 (ホストマップファイル : 3次元形状)

3次元座標を指定する場合、ホストマップファイルは(X,Y,Z)を指定します。

```
$ vi rankmap3
# ホストマップファイル(3次元指定例)
(0,0,0)                                #rank0
(1,0,0)                                #rank1
(0,1,0)                                #rank2
(1,1,0)                                #rank3
(0,0,1)                                #rank4
(0,1,1)                                #rank5
(1,0,1)                                #rank6
(1,1,1)                                #rank7
```

ランクマップファイル割当て（rank-map-hostfile）と起動プロセス数の指定を組み合わせることで、ノード内に複数プロセスを割り当てることも可能です。

例4) 2次元形状の rank-map-hostfile を指定し、ノード内に複数ランクを指定する



```
$ vi sample12.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x2:mesh            ノード数の指定(2次元形状)
#PJM --mpi proc=8                プロセス数指定
#PJM --mpi rank-map-hostfile=rankmap4  動的 MPI ランク割当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

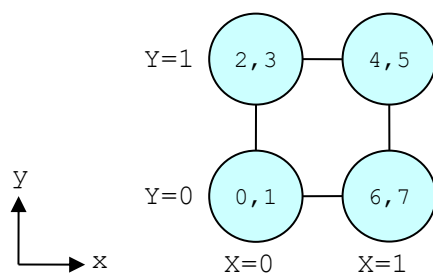
図 5-38 ランク割り当て例（ホストマップファイル：2次元形状+複数ランク割り当て）

起動するランク配置（2次元）をホストマップファイルに記載します。

```
$ vi rankmap4
# ホストマップファイル(2次元指定例)
(0,0)
(1,0)
(0,1)
(1,1)
```

rank-map-hostfile と rank-map-bychip を同時に指定することで、ホストマップファイルで指定される座標（1行）に、rank-map-bychip で指定された数のプロセスを割り当てることも可能です。

例5) 2次元形状の rank-map-hostfile を指定し、rank-map-bychip を同時に指定する



```
$ vi sample13.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=2x2:mesh            ノード数の指定(2次元形状)
#PJM --mpi proc=8                プロセス数指定
#PJM --mpi rank-map-hostfile=rankmap5  動的 MPI ランク割当て
#PJM --mpi rank-map-bychip:XY      動的 MPI ランク割当て
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#PJM -S
#----- program execution -----#
mpiexec ./a.out
```

図 5-39 ランク割り当て例（ホストマップファイル：2次元形状+rank-map-bychip 指定）

起動するランク配置（2次元）をホストマップファイルに記載します。

```
$ vi rankmap5
# ホストマップファイル(2次元指定例)
(0,0)                                #rank 0,1
(0,1)                                #rank 2,3
(1,1)                                #rank 4,5
(1,0)                                #rank 6,7
```

5.6.1.9. ホストマップ利用時の注意事項

- ファイル中の空行は、無視されます。
- ファイル中の有効な座標の行数が、--mpi proc（rank-map-bychipの場合は、 $\text{proc} \div n$ ）で指定した値よりも多い場合、残りの行は無視されます。
- ファイル中の有効な座標の行数が、--mpi proc（rank-map-bychipの場合は、 $\text{proc} \div n$ ）で指定した値よりも少ない場合、最後の行まで割り当てたら、先頭行に戻って割り当てます。
- ホストマップファイル中にノード座標を記述する場合は、各ノードに割り当てるプロセス数を均等にする必要があります。例えば、4ノードで7プロセスを実行する場合、4-1-1-1のプロセス数配置を指定してもジョブの実行はできません。2-2-2-1のように、プロセス数を均等にするように記述してください。

5.6.1.10. ジョブ形状（トーラス）

pjsub -L node で指定された形状（1 次元／2 次元／3 次元）、計算ノード数に応じて、ジョブに資源が割り当てられます。割り当てられた資源は、各軸単位にジョブ内でトーラスが構成されます。

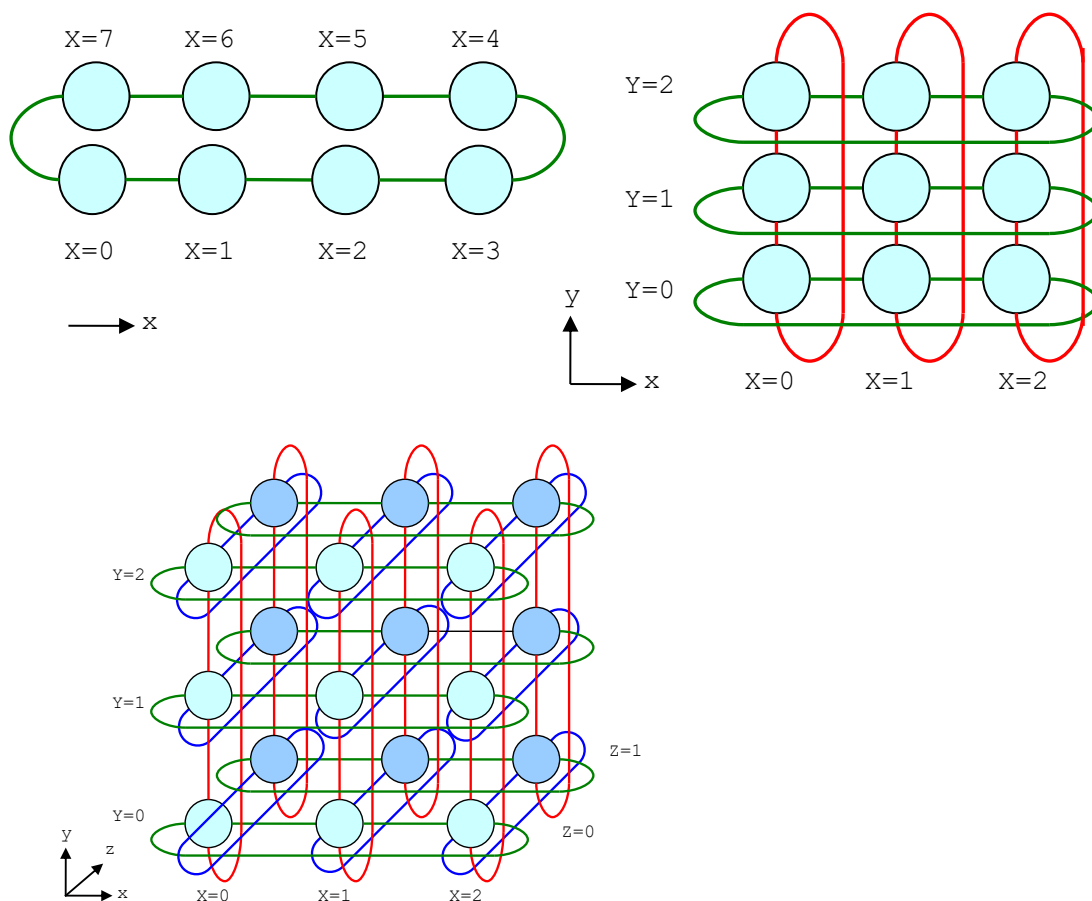


図 5-40 ジョブ形状（トーラス）

5.6.1.11. 複合 MPI オプション指定 (MPI 関連)

MPI プログラムを実行するには、pjsub のオプションを適切に指定する必要があります。1 次元の指定例を示します。

```
$ vi sample14.sh
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-o          リソースグループ指定
#PJM -L node=24:torus            ノード数の指定(1 次元形状)
#PJM --mpi proc=12               最大プロセス形状指定
#PJM -L elapse=10:00             経過時間指定
#PJM -g group1
#PJM -j
#----- program execution -----#
mpiexec ./a.out                  並列プロセス数 12 で a.out を実行
```

上記の-L node で要求した形状/計算ノード数が確保されます。1 次元形状(24)が指定されているため、利用するプロセスが 12 であっても、1 次元で 24 計算ノードが確保されます。

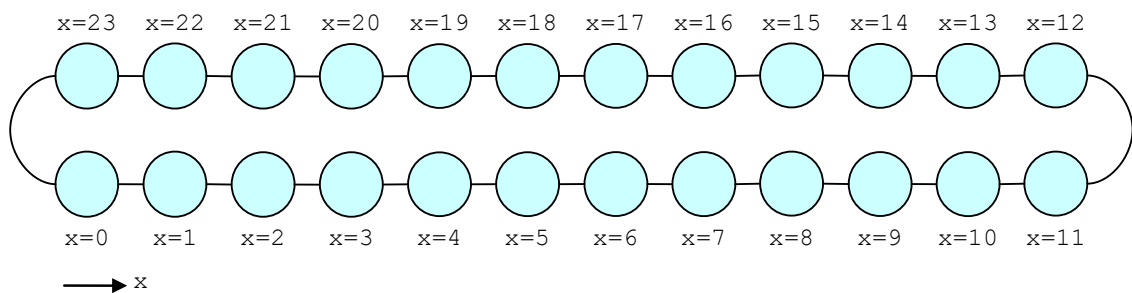


図 5-41 ジョブ形状 (トーラス)

5.6.2. Aquarius での MPI プログラムの実行 (Intel MPI 用)

Intel MPI でプログラムを実行する場合、`mpiexec.hydra` コマンドを使用します。

\$ mpiexec.hydra [global_option] [local_option] 実行モジュール :

`mpiexec.hydra` コマンドの主なオプションを表 5-26 に示します。

表 5-26 `mpiexec.hydra` コマンドの主なオプション

オプション	説明
-n <i>proc</i>	MPI プログラムの並列プロセス数を指定します。 <i>proc</i> には、ジョブ用環境変数 <code>PJM_MPI_PROC</code> を指定することが可能です。
-rr	ノード毎にラウンドロビンでプロセスを生成します。
-s <i>spec</i>	標準入力を指定された MPI プロセスにリダイレクトします(デフォルト: 0)。
-prepend-rank	並列プロセスの標準出力の行の先頭に、ランク番号を付加します。

5.6.2.1. MPI 実行に関する環境変数 (Intel MPI 用)

MPI プログラム実行時に使用可能な主な環境変数を表 5-27 に示します。

表 5-27 MPI プログラム実行に利用可能な主な環境変数 (Intel MPI 用)

環境変数	説明
<code>I_MPI_PIN</code>	プロセスのピンングを制御します。
<code>I_MPI_PIN_DOMAIN</code>	MPI/OpenMP ハイブリッドアプリケーションのプロセスピンングを制御します。
<code>I_MPI_FABRICS_LIST</code>	使用するファブリックリストを定義します。
<code>I_MPI_FABRICS</code>	ノード内通信およびノード間通信で使用するファブリックを指定します。
<code>I_MPI_FALLBACK</code>	使用できるファブリックにフォールバックするかを指定します。
<code>I_MPI_PERHOST</code>	ノード毎に割り当てるプロセス数を指定します。 Aquarius では、ジョブ投入時に指定されたノード数とプロセス数を元に自動で設定されます。
<code>I_MPI_EAGER_THRESHOLD</code>	全てのデバイスの eager/rendezvous メッセージサイズの閾値を変更します。
<code>I_MPI_DEBUG</code>	MPI プログラムの実行が開始するとデバッグ情報を出力します。

5.6.2.2. MPI 実行に関する注意事項

- ノード内にプロセスを複数生成する場合、ノード内のプロセス数が増えるにつれて MPI 初期化処理に時間がかかるようになります。

5.7. 高度なジョブ実行

5.7.1. Odyssey と Aquarius のジョブ連携

Odyssey と Aquarius で同時にジョブを実行し、共有ファイルシステム上でデータを共有して処理を連携させる、または、ジョブ連携用リソースグループと WaitIO ライブラリを使用して処理を連携させることが可能です。Odyssey と Aquarius でジョブの実行開始を揃える方法について説明します。

5.7.1.1. ジョブの開始時刻指定による運用

pjsub コマンドの--at オプションでジョブの実行開始時刻を指定することができます。Odyssey と Aquarius 向けのジョブ投入時に、それぞれ--at オプションで同じ開始時刻を指定してください。その際、以下のように、ジョブを投入するリソースグループの経過時間制限よりも未来の時刻を指定するようにすると、ジョブの優先度が上がり、指定時刻に実行開始する確率が高くなります。

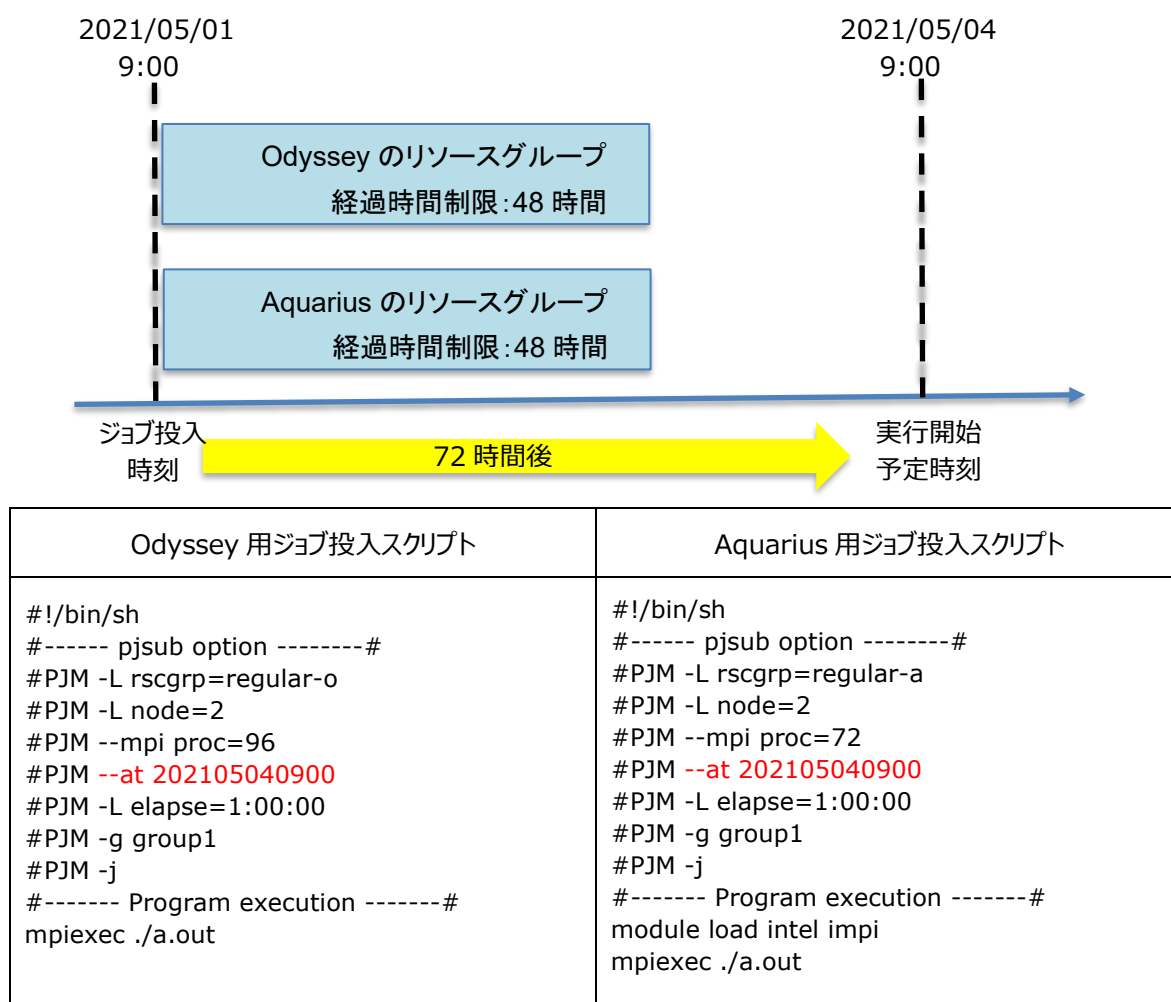


図 5-42 ジョブ投入時から3日後(72時間後)の実行開始を指定する場合

5.7.1.2. ジョブ連携用リソースグループと WaitIO ライブラリを使用した運用

ジョブ連携用リソースグループにジョブを投入することで、Odyssey と Aquarius で同時にジョブを実行することが可能です。また、WaitIO ライブラリを使用することでジョブの処理を連携させることが可能です。ジョブ連携用リソースグループを表 5-28 に示します。

ジョブ連携用リソースグループを使用して Odyssey と Aquarius で同時にジョブを実行させるためには、同時に実行したいジョブの**ジョブ名を同じに設定**する必要があります。ジョブ名は、`pjsub` コマンドの `-N` オプションを使用して設定します。また、ジョブ連携用リソースグループに投入されたジョブは HOLD 状態(ジョブ保留状態)となります。リソースグループに割り当てられたノードの空き状況を監視し、1 回のスケジューリングで 1 セット(ジョブ名、ユーザ名、グループ名が同じジョブの組み合わせ)のジョブの HOLD 状態を解除することで、Odyssey と Aquarius で同時にジョブを実行します。ジョブ連携用リソースグループに投入されたジョブは、ジョブ投入順にスケジューリング対象となります。

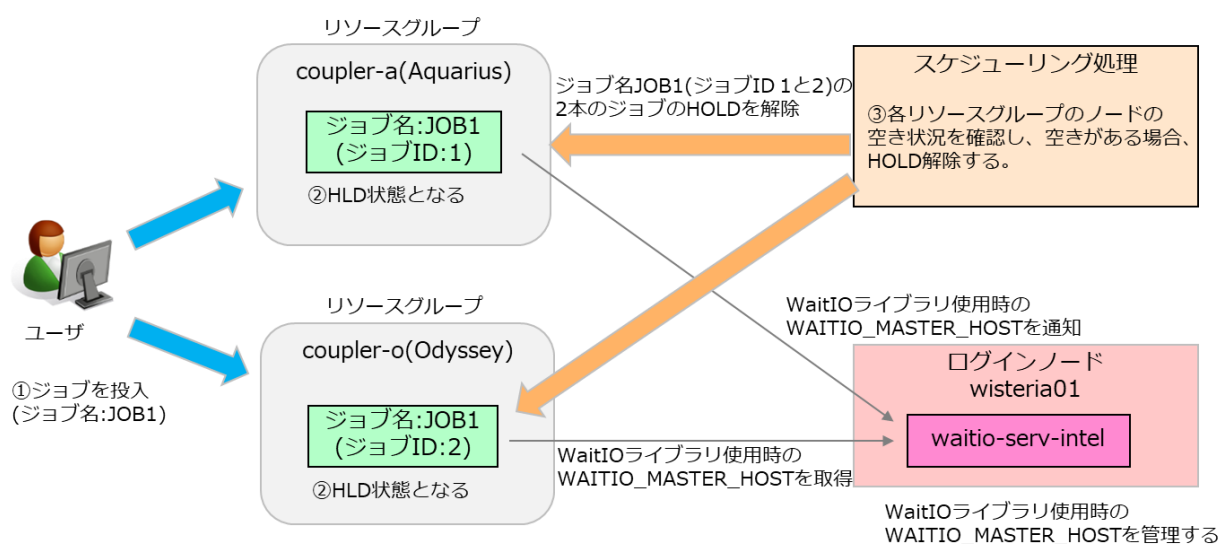


図 5-43 ジョブ連携用リソースグループと WaitIO ライブラリを使用した運用

表 5-28 ジョブ連携用リソースグループ

システム	リソースグループ	割り当てノード	最小ノード数	最大ノード数	ジョブあたりの実行時間制限	ノードあたりの最大メモリ容量
Odyssey	coupler-o	4608	1	1152	2 時間	28 GiB
Aquairus	coupler-a	4	1	2	2 時間	448GiB

以下に、OdysseyとAquariusのジョブ連携用リソースグループ(coupler-oとcoupler-a)に同時に実行したいジョブをそれぞれ 1 本投入し、WaitIO ライブラリを使用したプログラムを実行して処理を連携する場合のジョブスクリプト例を記載します。以下の例は、ジョブ名を”JOB1”とし、Aquarius で実行するジョブに割り当てられた計算ノードを WAITIO_MASTER_HOST に設定し、PB(Parallel Block)を 2 としています。WaitIO は *module load waitio* コマンドを実行することで利用できます。Aquarius で WaitIO の利用環境を設定するためには、*module load* コマンドでインテルコンパイラ環境の利用設定が必要です。また、Odyssey では、富士通コンパイラ環境の利用設定が必要です。Odyssey でのバッチジョブ実行時およびインタラクティブジョブ実行時にはデフォルトで富士通コンパイラ環境が設定されますので、明示的に *module load* コマンドでの利用環境設定は不要です。

Aquarius 用ジョブ投入スクリプト	
<pre>#!/bin/sh #PJM -L rscgrp=coupler-a # Aquarius のジョブ連携用リソースグループを指定します。 #PJM -N JOB1 # ジョブ名を指定します。 #PJM -L node=1 #PJM --mpi proc=2 #PJM -g group1 #PJM -j module load intel impi # Aquarius で WaitIO を利用するためにインテルコンパイラ環境を設定します。 module load waitio # WaitIO の利用環境を設定します。 export WAITIO_MASTER_HOST=`uname -n`-ib0 # WAITIO_MASTER_HOST を設定します。 export WAITIO_MASTER_PORT=7100 # WaitIO ライブラリで使用するポート番号を指定します。 export WAITIO_PPID=0 # PPID を指定します。 export WAITIO_NPB=2 # PB 数を指定します。 waitio-serv-intel -m \$WAITIO_MASTER_HOST # WAITIO_MASTER_HOST を通知します。 mpiexec.hydra -n \$PJM_MPI_PROC ./a.out</pre>	

Odyssey 用ジョブ投入スクリプト	
<pre>#!/bin/sh #PJM -L rscgrp=coupler-o # Odyssey のジョブ連携用リソースグループを指定します。 #PJM -N JOB1 # ジョブ名を指定します。 #PJM -L node=1 #PJM --mpi proc=2 #PJM -g group1 #PJM -j module load waitio # WaitIO の利用環境を設定します。 export WAITIO_MASTER_HOST=`waitio-serv-a64fx -c` # WAITIO_MASTER_HOST を取得・設定します。 export WAITIO_MASTER_PORT=7100 # WaitIO ライブラリで使用するポート番号を指定します。 export WAITIO_PPID=1 # PPID を指定します。 export WAITIO_NPB=2 # PB 数を指定します。 mpiexec ./a.out</pre>	

表 5-29 WaitIO 用環境変数

環境変数	説明
WAITIO_MASTER_HOST	WAITIO_MASTER_HOST を指定します。 <code>uname -n</code> コマンドで取得したホスト名の後ろに以下の文字を付与したホスト名を指定してください。 Odyssey の場合、"-tofu1" Aquarius の場合、"-ib0" 例) Odyssey の場合の設定 <code>export WAITIO_MASTER_HOST=`uname -n`-tofu1</code>
WAITIO_MASTER_PORT	WaitIO ライブラリで使用するポート番号を指定します。 7000～7999 の任意のポートを使用してください。 同時に実行するジョブ内で同じポート番号を指定してください。
WAITIO_PPID	PPID(0～WAITIO_NPB-1)を指定します。 WAITIO_MASTER_HOST は 0 に設定します。
WAITIO_NPB	PB(Parallel Block)数を指定します。

表 5-30 WAITIO_MASTER_HOST の通知コマンド

システム	WAITIO_MASTER_HOST の通知コマンド
Odyssey	<code>waitio-serv-a64fx -m <ホスト名></code>
Aquarius	<code>waitio-serv-intel -m <ホスト名></code>

表 5-31 WAITIO_MASTER_HOST の取得コマンド

システム	WAITIO_MASTER_HOST の取得コマンド
Odyssey	<code>waitio-serv-a64fx -c</code>
Aquarius	<code>waitio-serv-intel -c</code>



以下、注意事項を記載します。

- ジョブ連携用リソースグループに割り当てられた計算ノードは他のリソースグループに割り当てられた計算ノードとノードを共有しています。他のリソースグループのジョブ運用状況次第では、ジョブの同時実行に必要な空きノード/GUP を確保できずスケジューリングに時間を要する場合(HOLD 状態のままとなる)があります。
- ジョブ連携用リソースグループに投入されたジョブは、ジョブ投入順にスケジューリング対象(HOLD 解除対象)となります。1 回のスケジューリングで、ジョブ投入時刻が一番古いジョブセット(ジョブ名、ユーザ名、グループ名が同一のジョブ)が 1 つのみスケジューリング対象となります。そのため、一番古いジョブセットが HOLD 解除されるまで、後続のジョブセットは HOLD 状態となります。
- リソースグループ coupler-o では、基本的にノード指定ジョブを利用ください。形状指定ジョブの場合、Odyssey のジョブ運用状況次第では、同時にジョブを実行できない場合があります。
- ジョブ連携用リソースグループでは、バッチジョブのみ利用可能です(バルクジョブ、ステップジョブ、インタラクティブジョブは利用できません)。
- ジョブ連携用リソースグループにジョブを 1 本のみ投入した場合、スケジューリング対象となりません。

5.7.2. Aquarius での OpenMP スレッド並列プログラム実行

5.7.2.1. Intel コンパイラの OpenMP スレッド並列実行に関する環境変数

OpenMP スレッド並列実行時に指定可能な主な環境変数を表 5-32 に示します。

表 5-32 OpenMP スレッド並列実行に利用可能な主な環境変数

環境変数	説明
OMP_NUM_THREADS	OpenMP もしくは自動並列によりスレッド並列化されたプログラムのスレッド数を指定します。デフォルトではジョブ投入時に指定されたスレッド数が設定されますが、ジョブスクリプト中で再設定した場合は、その値が有効になります。
KMP_AFFINITY	スレッドのコアバインドを制御します。 Aquarius のデフォルトは <code>balanced,granularity=fine</code> です。
KMP_STACKSIZE	プライベートスタックとして使用する各 OpenMP スレッドに割り当てるバイト数を指定します。
KMP_BLOCKTIME	<code>parallel region</code> の実行が終了した後、スレッドがスリープ状態になるまでスレッドが待機する時間(ミリ秒単位)を指定します。

5.7.2.2. OpenMP スレッド並列実行におけるコア割り当て

Intel コンパイラの OpenMP を用いたスレッド並列プログラムでは、環境変数 `KMP_AFFINITY` の設定により、物理コアへのスレッド割り当てが変更されます。

表 5-33 環境変数 `KMP_AFFINITY`

設定値	説明
<code>balanced</code>	可能な限り異なる物理コアを使用し、ノード内のコア全体にスレッドを均等に分配します。 <code>scatter</code> と異なり、物理コアに割り当てられるスレッドが互いに隣接します。 Aquarius におけるデフォルト値です。
<code>scatter</code>	可能な限り異なる物理コアを使用し、ノード内のコア全体にスレッドを均等に分配します。 <code>scatter</code> は、 <code>compact</code> の逆の動作です。
<code>compact</code>	可能な限り同じ物理コアを使用します。
<code>none</code>	スレッドはバインドされません。

例) KMP_AFFINITY 環境変数に compact を指定してプログラムを実行するジョブスクリプト

```
#!/bin/sh
#----- pjsub option -----#
#PJM -L rscgrp=regular-a
#PJM -L node=1
#PJM -g group1
#PJM -j
#----- Program execution -----#
module load intel
export OMP_NUM_THREADS=18
export KMP_AFFINITY=compact
./a.out
```

5.8. ジョブ状態表示

投入したジョブ状態やリソース情報を確認する場合、*pjstat* コマンドを使用します。

```
$ pjstat [option] [JOBID[JOBID...]]
```

表 5-34 pjstat コマンドオプション一覧

オプション名	説明
なし	実行待ち、実行中のジョブ情報を表示
-a	全ユーザーのジョブ情報を表示する。 -v オプション、-E --expand オプションおよび--system --sys のみ併用可能。
-A --all	所属グループ全員のジョブ情報を表示する。システム管理者の場合、全ユーザー情報を表示する。
--system --sys	システム毎にジョブ情報を表示する。
-H --history	処理が終了したジョブ情報を表示する。過去 3 日に終了したジョブで直近最大 100 件を表示する。
--hday day	表示期間（日）を指定する。
--hnum num	最大表示件数を指定する。
-E --expand	ステップジョブ情報、バルクジョブ情報を表示する。
-v	標準形式で出力されない、いくつかのジョブ情報を追加で表示する。 ・ジョブの優先度 ・ジョブの待ち理由・ジョブ操作メッセージ
-s	-v オプションで出力される情報のほか、資源使用状況、資源制限値などの詳細情報を追加で表示する。 ・ステップジョブの依存情報 ・投入・開始・終了・削除・削除要求時刻、終了コード etc
-S	-s オプションで出力される情報のほか、そのジョブに設定されているノード単位の情報についても追加で表示する。
-X	RUNNING 中のジョブに割り当てられた計算ノード ID とランク番号を表示する。
--rsc	リソースグループの情報を表示する。ユーザーが実行を許可されていない情報は表示しない。
-x	リソースグループの制限値を表示する。
-b	リソースグループのジョブ情報（ジョブ情報毎の統計）を表示する。
--limit	プロジェクトのジョブ投入数・実行制限値を表示する。
--rscuse	リソースタイプのノード、GPU 使用量を表示する。
-v	全リソースグループのノード、GPU 使用量を表示する。
--nodeuse	リソースタイプのノード使用量を表示する。
-v	ノード専有リソースグループのノード使用量を表示する。
--gpuuse	リソースタイプの GPU 使用量を表示する。
-v	GPU 専有リソースグループの GPU 使用量を表示する。
--scal	システム停止、起動の予定時刻を表示する。
-l	ジョブ名を全て表示する（ロングフォーマット形式）

5.8.1. ジョブ状態表示

pjstat コマンドを実行すると、現在実行中もしくは実行待ちのジョブ状態を表示します。

```
[username@wisteria01 work]$ pjstat
Wisteria/BDEC-01 scheduled stop time: 2021/05/14(Fri) 09:00:00 (Remain:  -days --:--:--)
```

JOB_ID	JOB_NAME	STATUS	PROJECT	RSCGROUP	START_DATE	ELAPSE
TOKEN	NODE GPU					
12345	job.sh	RUNNING	group1	small-a	05/12 22:39:56	00:26:27
-	1 8					
12346	job.sh	RUNNING	group1	small-a	(05/12 23:18:55) --:--:--	-
1 8						
12347	job.sh	RUNNING	group1	small-a	(05/12 23:47:08) --:--:--	-
1 8						



START_DATE に出力されるジョブの実行開始予測時刻は目安であるため、システムの混雑状況や課題の優先度により変動することがあります。

表 5-35 ジョブ情報の表示項目

項目	説明
JOB_ID	ジョブ ID
JOB_NAME	ジョブ名 (10 文字以上は切り捨て) -a オプションを指定した場合、自ジョブのみ表示する。 -l オプションを指定した場合、ジョブ名を全て表示する。
STATUS	ジョブの状態 (表 5-36 参照)
USER	ユーザー名(-A オプション指定時のみ)
PROJECT	プロジェクト名 -a オプションを指定した場合、自ジョブのみ表示する。
RSCGROUP	リソースグループ名
START_DATE	ジョブが実行前の場合は開始予測時刻("()"で表示)、実行中および実行後の場合は実際に実行を開始した時刻。 実行開始時刻を指定して投入したジョブが実行を開始するまでの間、時刻の後ろに「@」が出力される。 バックフィルが適用されたジョブは、時刻の後ろに「<」が出力される。
ELAPSE	ジョブの経過時間
TOKEN	使用トークン時間 ジョブ実行開始前の場合はトークン予測使用量(ノード数と経過時間制限を元に算出)
NODE	ジョブのノード数
GPU	ジョブの GPU 数

表 5-36 ジョブの状態一覧

状態	説明
QUEUED	ジョブ実行待ち
RUNNING	ジョブ実行中
END	ジョブ終了(-H オプション指定時のみ)
CANCEL	ジョブキャンセルによる終了(-H オプション指定時のみ)
HOLD	ジョブ保留
ERROR	システムエラーによるジョブ固定

5.8.2. 詳細ジョブ情報の表示(-v オプション)

pjstat コマンドの-v オプションを指定すると、詳細なジョブ情報を表示します。

```
[username@wisteria01 work]$ pjstat -v
Wisteria/BDEC-01 scheduled stop time: 2021/05/14(Fri) 09:00:00 (Remain:  -days ---:--:--)
```

JOB_ID	JOB_NAME	STATUS	PROJECT	RSCGROUP	START_DATE	ELAPSE
ELAPSE_LIM	TOKEN	NODE	GPU	REASON		
12345	job.sh	RUNNING	group1	small-a	05/12 23:11:24	00:03:58 48:00:00
-	1 8 -					
12346	job.sh	RUNNING	group1	small-a	(05/12 23:18:55)	---:--:-- 48:00:00
-	1 8 -					
12347	job.sh	RUNNING	group1	small-a	(05/12 23:47:08)	---:--:-- 48:00:00
-	1 8 -					

表 5-37 詳細ジョブ情報の表示項目(追加項目のみ)

オプション名	説明
ELAPSE_LIM	ジョブの経過時間制限
REASON	通知メッセージ

5.8.3. 終了ジョブ情報の表示(-H オプション)

pjstat コマンドの-Hオプションを指定すると、実行が終了したジョブを最大 100 件表示します。デフォルトでは過去 3 日間で実行が終了したジョブが対象となりますが、--hday オプションにより最大 31 日間で終了したジョブが対象となります。

```
[username@wisteria01 work]$ pjstat -H
Wisteria/BDEC-01 scheduled stop time: 2021/05/14(Fri) 09:00:00 (Remain: -days ---:---:--)
```

JOB_ID TOKEN	JOB_NAME NODE	STATUS GPU	PROJECT	RSCGROUP	START_DATE	ELAPSE
7597	run.sh	END	group1	small-a	05/10 20:43:12	00:10:00
-	2 16					
7598	run.sh	END	group1	small-a	05/10 20:47:37	00:10:01
-	2 16					
7599	run.sh	END	group1	small-a	05/10 20:48:44	00:10:02
-	2 16					

5.8.4. リソースグループの表示(--rsc オプション)

pjstat コマンドの--rsc オプションを指定すると、利用可能なリソースグループを表示します。

```
[username@wisteria01 work]$ pjstat --rsc
SYSTEM: Odyssey
RSCGRP          STATUS          NODE
debug-o         [ENABLE,START]  4x12x16
short-o         [ENABLE,START]  2x12x16
regular-o
|---- small-o   [ENABLE,START]  14x24x16
|---- medium-o  [ENABLE,START]  14x24x16
|---- large-o   [ENABLE,START]  14x24x16
`---- x-large-o [ENABLE,START]  14x24x16
interactive-o
|---- interactive-o_n1 [ENABLE,START]  96
`---- interactive-o_n12 [ENABLE,START]  96
(省略)
SYSTEM: Aquarius
RSCGRP          STATUS          NODE
debug-a         [ENABLE,START]  2
short-a         [ENABLE,START]  4
regular-a
|---- small-a   [ENABLE,START]  25
|---- medium-a  [ENABLE,START]  25
`---- large-a   [ENABLE,START]  25
interactive-a   [ENABLE,START]  2
(省略)
```

表 5-38 リソースグループの表示項目

項目	説明
RSCGRP	リソースグループ名
STATUS	リソースグループの状態 ENABLE: ジョブの新規投入可能 DISABLE: ジョブの新規投入不可 START: 待ちジョブを新規実行可能 STOP: 待ちジョブの新規実行抑止
NODE	リソースグループのノード数
GPU	リソースグループの GPU 数

`pjstat` コマンド`--rsc` オプションに加え`-b` オプションを指定すると、全ジョブのリソースグループ毎のジョブ数を状態別に表示します。

```
[username@wisteria01 work]$ pjstat --rsc -b
SYSTEM: Odyssey
RSCGRP          STATUS          TOTAL RUNNING  QUEUED  HOLD OTHER      NODE
debug-o         [ENABLE,START]    0          0        0    0    0    4x12x16
short-o         [ENABLE,START]    0          0        0    0    0    2x12x16
regular-o
|---- small-o   [ENABLE,START]    0          0        0    0    0    14x24x16
|---- medium-o  [ENABLE,START]    0          0        0    0    0    14x24x16
|---- large-o   [ENABLE,START]    0          0        0    0    0    14x24x16
`---- x-large-o [ENABLE,START]    0          0        0    0    0    14x24x16
interactive-o
|---- interactive-o_n1 [ENABLE,START]    0          0        0    0    0     96
`---- interactive-o_n12 [ENABLE,START]    0          0        0    0    0     96
(省略)
RSCGRP          STATUS          TOTAL RUNNING  QUEUED  HOLD OTHER      NODE GPU
share-debug     [ENABLE,STOP]    0          0        0    0    0      1   8
share-short     [ENABLE,STOP]    0          0        0    0    0      2  16
share
|---- share-1    [ENABLE,STOP]    0          0        0    0    0      6  48
|---- share-2    [ENABLE,STOP]    0          0        0    0    0      2  16
`---- share-4    [ENABLE,STOP]    0          0        0    0    0      1   8
share-interactive [ENABLE,STOP]    0          0        0    0    0      1   8
```

表 5-39 リソースグループ毎の状態別ジョブ数の表示項目

項目	説明
RSCGRP	リソースグループ名
STATUS	リソースグループの状態
TOTAL	リソースグループに投入されたジョブの総数
RUNNING	実行中のジョブ数
QUEUED	ジョブ実行待ち状態のジョブ数
HOLD	保留状態のジョブ数
OTHER	ジョブ状態が RUNNING,QUEUED,HOLD のジョブ数
NODE	リソースグループのノード数
GPU	リソースグループの GPU 数

pjstat コマンドの--rsc オプションに加え-x オプションを指定すると、各リソースグループのノード数制限や経過時間制限を表示します。

```
[username@wisteria01 work]$ pjstat --rsc -x
SYSTEM: Odyssey
RSCGRP          STATUS      MIN_NODE MAX_NODE MAX_ELAPSE REMAIN_ELAPSE MEM(GiB) PROJECT
debug-o         [ENABLE,START]    1    144  00:30:00    00:30:00    28 fjtest
short-o         [ENABLE,START]    1     72  08:00:00    08:00:00    28 fjtest
regular-o
|---- small-o   [ENABLE,START]    1    144  48:00:00    48:00:00    28 fjtest
|---- medium-o  [ENABLE,START]   145   576  48:00:00    48:00:00    28 fjtest
|---- large-o   [ENABLE,START]   577  1152  48:00:00    48:00:00    28 fjtest
`---- x-large-o [ENABLE,START]  1153  2304  24:00:00    24:00:00    28 fjtest
interactive-o
|---- interactive-o_n1 [ENABLE,START]    1     1  02:00:00    02:00:00    28 fjtest
`---- interactive-o_n12 [ENABLE,START]    2    12  00:10:00    00:10:00    28 fjtest
(省略)
```

表 5-40 リソースグループの表示項目

項目	説明
RSCGRP	リソースグループ名
STATUS	リソースグループの状態
MIN_NODE	ジョブの最小割り当てノード数
MAX_NODE	ジョブの最大割り当てノード数
AVAIL_GPU	ジョブの割り当て GPU 数(Aquarius のみ)
MAX_ELAPSE	ジョブの最大経過時間制限
REMAIN_ELAPSE	定期保守までの残り時間
MEM(GB)	ジョブの最大メモリ制限(GB)
PROJECT	プロジェクト名

5.8.5. ノード利用状況の表示(--nodeuse オプション)

pjstat コマンドの--nodeuse オプションを指定すると、ノードの利用状況を表示します。割り当てノードが共通のリソースグループは同一行に表示されます。

```
[username@wisteria01 work]$ pjstat --nodeuse
SYSTEM: Odyssey
RSCGRP                                Ratio Used/Total(Node)
debug-o/interactive-o  -----  0%    0/ 96
short-o                -----  0%    0/ 384
regular-o              -----  0%    0/5376
priority-o             -----  0%    0/1152
challenge-o            -----  0%    0/7680

SYSTEM: Aquarius
RSCGRP                                Ratio Used/Total(Node)
debug-a/interactive-a  -----  0%    0/  2
short-a                -----  0%    0/  4
regular-a              -----  0%    0/ 25
challenge-a            -----  0%    0/ 45

SYSTEM: Prepost
RSCGRP                                Ratio Used/Total(Node)
prepost                -----  0%    0/  9
(省略)
```

表 5-41 ノード利用状況の表示項目

項目	説明
RSCGRP	リソースグループ名
Ratio	ノード使用率
Used	使用している計算ノード数 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた計算ノード総数

5.8.6. GPU 利用状況の表示(--gpuuse オプション)

pjstat コマンドの--gpuuse オプションを指定すると、GPU 専有利用における GPU の利用状況を表示します。割り当てノードが共通のリソースグループは同一行に表示されます。

```
[username@wisteria01 work]$ pjstat --gpuuse
SYSTEM: Aquarius
RSCGRP                                Ratio Used/Total(GPU)
share-debug/share-interactive ----- 0%    0/   8
share-short                          ----- 0%    0/  16
share                                ----- 0%    0/  72
```

表 5-42 GPU 利用状況の表示項目

項目	説明
RSCGRP	リソースグループ名
Ratio	ノード使用率
Used	使用している GPU 数 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた GPU 総数

5.8.7. ノードおよび GPU 利用状況の表示(--rscuse オプション)

pjstat コマンドの--rscuse オプションを指定すると、ノードおよび GPU の利用状況を表示します。割り当てノードが共通のリソースグループは同一行に表示されます。

```
[username@wisteria01 work]$ pjstat --rscuse
SYSTEM: Odyssey
RSCGRP                                Ratio Used/Total(Node)
debug-o/interactive-o  -----  0%    0/ 96
short-o                -----  0%    0/ 384
regular-o              -----  0%    0/5376
priority-o             -----  0%    0/1152
challenge-o            -----  0%    0/7680

SYSTEM: Aquarius
RSCGRP                                Ratio Used/Total(Node)
debug-a/interactive-a  -----  0%    0/  2
short-a                -----  0%    0/  4
regular-a              -----  0%    0/ 25
challenge-a            -----  0%    0/ 45

RSCGRP                                Ratio Used/Total(GPU)
share-debug/share-interactive -----  0%    0/  8
share-short            -----  0%    0/ 16
share                  -----  0%    0/ 72

SYSTEM: Prepost
RSCGRP                                Ratio Used/Total(Node)
prepost                -----  0%    0/  9
```

表 5-43 ノードおよび GPU 利用状況の表示項目

項目	説明
RSCGRP	リソースグループ名
Ratio	ノード使用率
Used	使用している計算ノード数、または GPU 数(share の場合) 使用ノード数の割合は小数点以下を四捨五入して表示する
Total	リソースタイプに割り当てられた計算ノード総数、または GPU 総数(share の場合)

5.8.8. ジョブ本数制限、ノード制限の表示(--limit オプション)

pjstat コマンドの--limit オプションを指定すると、各種制限値と現在の状況を表示します。

```
[username@wisteria01 work]$ pjstat --limit
SYSTEM: Odyssey
PROJECT   ACCEPT      RUN  BULK_ACCEPT  BULK_RUN    NODE
group1    11/10000    0/7680    0/10000    0/7680    0/7680

SYSTEM: Aquarius
PROJECT   ACCEPT      RUN  BULK_ACCEPT  BULK_RUN    GPU
group1    1/ 400    1/ 360    0/  -    0/  -    0/ 360
```

表 5-44 各種制限値の表示項目

項目	説明
PROJECT	プロジェクト名
ACCEPT	投入バッチジョブ本数/バッチジョブ同時受付制限
RUN	実行中のバッチジョブ本数/バッチジョブ同時実行制限
BULK_ACCEPT	投入バルクサブジョブ本数/バルクサブジョブ同時受付制限 Aquarius のバルクサブジョブ同時受付制限は、“-”を表示する。
BULK_RUN	実行中のバルクサブジョブ本数/バルクサブジョブ同時実行制限数 Aquarius のバルクサブジョブ同時実行制限数は、“-”を表示する。
NODE	実行中のバッチジョブの使用ノード数/プロジェクト単位の同時ノード使用制限 同時使用ノード数制限がないプロジェクトは、“-”を表示する。
GPU	実行中のバッチジョブの使用 GPU 数/プロジェクト単位の同時 GPU 使用制限

5.9. ジョブキャンセル

ジョブをキャンセルする場合、*pjdel* コマンドを実行します。

```
$ pjdel [option] JOBID [JOBID...]
```

表 5-45 ジョブキャンセルコマンドオプション一覧

オプション	説明
-R reasonmessage	ジョブを削除した理由を指定する。指定した文字列は、ジョブ参照コマンドの REASON に出力する。

例) ジョブ ID12345 をキャンセル

```
[username@wisteria01 work]$ pjdel 12345
[INFO] PJM 0100 pjdel Job 12345 canceled.
```

5.10. ジョブ保留

ジョブの実行を保留する場合、*pjhold* コマンドを実行します。

```
$ pjhold [option] JOBID [JOBID...]
```

表 5-46 ジョブ保留コマンドオプション一覧

オプション	説明
-R reasonmessage	ジョブを保留した理由を指定する。指定した文字列は、pjstat の REASON に出 力する。

例) ジョブ ID12345 を保留

```
[username@wisteria01 work]$ pjhold 12345  
[INFO] PJM 0100 pjdel Job 12345 held.
```



実行中ジョブを保留する場合、対象ジョブは--restart オプションを指定して投入されている必要があります。

5.11. ジョブ保留の解除

保留されたジョブを解除する場合、*pjrls* コマンドを実行します。

```
$ pjrls JOBID [JOBID...]
```

例) ジョブ ID12345 の保留を解除

```
[username@wisteria01 work]$ pjrls 12345  
[INFO] PJM 0100 pjdel Job 12345 released.
```

5.12. ジョブ履歴の参照

ジョブの状態遷移を表示する場合、pjlogf コマンドを使用します。

```
$ pjlogf [option] JOBID [JOBID...]
```

表 5-47 ジョブの状態遷移表示コマンドオプション一覧

オプション	説明
-d day	履歴の参照範囲(日数)を指定する。指定した日数分の過去のデータを表示する。指定範囲は 1～31 とする。オプションを省略した場合は、過去 1 日に終了したジョブの履歴情報を表示する。

例) ジョブ ID12345 の状態遷移を表示

```
[username@wisteria01 work]$ pjlogf 12345
DATE      TIME      JOB_ID  JOB_NAME      USER    RSCGRP  EVENT
2021/05/11 03:24:19    12345    run.sh      username small-o ACCEPT
2021/05/11 03:24:19    12345    run.sh      username small-o QUEUED
2021/05/11 03:24:20    12345    run.sh      username small-o RUNNING-A
2021/05/11 03:24:22    12345    run.sh      username small-o RUNNING_P
2021/05/11 03:24:23    12345    run.sh      username small-o RUNNING
2021/05/11 03:43:52    12345    run.sh      username small-o RUNNING_E
2021/05/11 03:43:52    12345    run.sh      username small-o RUNOUT
2021/05/11 03:43:52    12345    run.sh      username small-o END
```

表 5-48 ジョブの状態遷移表示項目一覧

項目	説明
DATE	日付 (YYYY/MM/DD)
TIME	時刻 (hh:mm:ss)
JOB_ID	ジョブ ID
JOB_NAME	ジョブ名
USER	ユーザー名
RSCGRP	リソースグループ名
EVENT	イベント情報。主なイベントは以下の通り。
	ACCEPT ジョブ仮受付
	QUEUED ジョブ実行待ち
	RUNNING-A ジョブ実行に必要な資源を獲得中
	RUNNING-P プロログ実行中
	RUNNING ジョブ実行中
	RUNNING_E エピログ実行中
	RUNOUT ジョブ終了処理待ち
	END ジョブ終了
	CANCEL ジョブ削除
	HOLD ジョブ保留

5.13. 計算ノードから外部リソースへのアクセス

OdysseyとAquariusの計算ノード、および、プリポストノードからバッチジョブ・インタラクティブジョブを使用して外部リソース(サーバ、ストレージ、センサーなど)に直接アクセスすることが可能です。ただし、外部リソースへのアクセスは以下のアクセス方法のみとなります。なお、外部リソースから各計算ノードへのアクセスはできません。

表 5-49 各計算ノードからの外部リソースへのアクセス方法一覧

システム	アクセス方法	アクセス設定
Odyssey	・HTTP/HTTPS	ジョブ実行時にジョブ投入ログインノードを Proxy サーバとして自動設定
Aquarius	・HTTP/HTTPS	NAT により外部リソースへアクセスできるように設定
プリポストノード	・SSH	

6. 開発ツール

6.1. 開発ツールの概要

Wisteria/BDEC-01 システムでは、Odyssey または Aquarius に対応する開発ツールを用いて、プログラムの性能分析やデバッグを行うことが可能です。利用可能な開発ツールを以下に示します。

表 6-1 Wisteria/BDEC-01 システムで利用可能な開発ツール一覧

ソフトウェア名称	対象システム	概要
プロファイラ	Odyssey	Fujitsu Development Studio の性能情報計測ツール
並列実行デバッガ	Odyssey	Fujitsu Development Studio のデバッグツール
統合開発環境	Odyssey	Eclipse に Fujitsu Development Studio 向け機能を拡張したコンパイル・ジョブ実行などを操作可能なツール
VTune プロファイラー	Aquarius	Intel OneAPI のパフォーマンス分析ツール
Arm DDT	Aquarius	C/C++/Fortran90 向けデバッガ
Arm MAP	Aquarius	C/C++/Fortran90 向けプロファイラー

6.2. Odyssey 向け開発ツール

6.2.1. プロファイラ

プログラムの実行性能を向上させるために有用な性能情報を計測および出力するツールとして、プロファイラが利用できます。プロファイラは下記の機能で構成されています。詳細な利用方法については、ポータルのドキュメントをご参照ください。

表 6-2 プロファイラの構成

機能	概要
基本プロファイラ	プログラム全体の性能傾向を把握する。
詳細プロファイラ	特定区間の詳細な傾向を把握する
CPU 性能解析レポート	特定区間の CPU 性能解析情報を表やグラフで可視化する

6.2.1.1. 基本プロファイラ

基本プロファイラの使用手順を以下に示します。

- (1) 対象プログラムをコンパイル/リンクして実行ファイルを生成します。
翻訳時オプションは通常と同じです。基本プロファイラ向けのオプション追加はありません。
- (2) 実行ファイルを計算ノードで実行し、プロファイルデータを計測します。

計測コマンド fipp を付加して実行します。データの計測を指示する -C と、データの格納ディレクトリを指定する -d が必須オプションです。計測コマンドの使用例を以下に示します。

例)基本プロファイラで MPI プログラムの CPU 動作状況を計測し、結果を ./tmp に格納する

```
fipp -C -d./tmp mpiexec ./a.out
```

(3) 計測したプロファイルデータの結果を出力します。

計算ノードでは計測コマンドと同じ fipp を使用します。ログインノードでは fipppx を使用します。オプションは共通で、結果の出力を指示する -A と、データの格納ディレクトリを指示する -d が必須オプションです。出力コマンドの使用例を以下に示します。

例)計算ノードで ./tmp に格納したデータを出力し、テキストファイル fipp.out に保存する

```
fipp -A -d./tmp > fipp.out
```

例)ログインノードで ./tmp に格納したデータを出力し、テキストファイル fipppx.out に保存する

```
fipppx -A -d./tmp > fipppx.out
```

基本プロファイラの出力結果から、CPU の動作状況やコスト分布情報を確認することができます。出力結果の例を以下に示します。

例)基本プロファイラの出力結果 (CPU の動作状況)

```
Performance monitor event:statistics

*****
Application - performance monitors
*****

Execution          Floating-point  Mem throughput  Mem throughput
   time (s)         GFLOPS    peak ratio (%)    (GB/s)    peak ratio (%)
-----
    62.3742         40.0743      7.1155      98.1196      9.5820  Application
-----
    62.3742         5.0093      7.1155      12.1929      1.1907  Process    2
    62.3740         5.0093      7.1155      12.1853      1.1900  Process    3
    62.3730         5.0094      7.1156      12.1999      1.1914  Process    1
    62.3724         5.0094      7.1157      12.3357      1.2047  Process    6
    62.3583         5.0106      7.1173      12.3480      1.2059  Process    7
    62.3425         5.0118      7.1191      12.3342      1.2045  Process    4
    62.2620         5.0183      7.1283      12.3562      1.2067  Process    5
    62.2525         5.0191      7.1294      12.2235      1.1937  Process    0
```

例)基本プロファイラの出力結果（コスト分布情報）

```

Loops profile (Total thread cost basis)

*****

Application - loops
Application and Process outputs the total value of the cost of each thread.
Procedure outputs the total value of the loop cost of each thread.
*****

      Cost      % Operation (s)      Barrier      % Nest  Kind  Exec  Start  End
-----
      4736    100.0000      473.6154      --      --   --   --   --   --   Application
-----

      4544    95.9459      454.4147      --      --    4  DO   SERIAL   380   396   jacobi_
      119     2.5127      11.9004      --      --   --  ARRAY  SERIAL   400   400   jacobi_
       3     0.0633       0.3000      --      --   --  ARRAY  SERIAL   318   318   initmt_
       3     0.0633       0.3000      --      --    3  DO   SERIAL   379   397   jacobi_
       3     0.0633       0.3000      --      --   --  ARRAY  SERIAL   316   316   initmt_
       1     0.0211       0.1000      --      --   --  ARRAY  SERIAL   317   317   initmt_
       1     0.0211       0.1000      --      --   --  ARRAY  SERIAL   319   319   initmt_
       1     0.0211       0.1000      --      --    2  DO   SERIAL   378   398   jacobi_

```

6.2.1.2. 詳細プロファイラ

詳細プロファイラの使用手順を以下に示します。

(1) 対象プログラムのソースに計測区間指定ルーチンを追加します。

プロファイルデータを計測する区間を指定するため、ソースコードに計測開始ルーチンと計測終了ルーチンを追加します。ソースコードへの追加例を以下に示します。

例)詳細プロファイラ計測区間指定ルーチンの追加例

```
...  
call fapp_start("loop1",1,0)  !計測開始ルーチン  
do k=2,kmax-1  
  do j=2,jmax-1  
    do i=2,imax-1  
      ...  
    enddo  
  enddo  
enddo  
call fapp_stop("loop1",1,0)  !計測終了ルーチン  
...
```

(2) 対象プログラムをコンパイル/リンクして実行ファイルを生成します。

翻訳時オプションは通常と同じです。詳細プロファイラ向けのオプション追加はありません。

(3) 実行ファイルを計算ノードで実行し、プロファイルデータを計測します。

計測コマンド fapp を付加して実行します。データの計測を指示する -C と、データの格納ディレクトリを指定する -d が必須オプションです。計測コマンドの使用例を以下に示します。

例)詳細プロファイラで MPI プログラムの CPU 動作状況を計測し、結果を ./tmp2 に格納する

```
fapp -C -d./tmp2 -Icpupa mpiexec ./a.out
```

(4) 計測したプロファイルデータを出力します。

計算ノードでは計測コマンドと同じ fapp を使用します。ログインノードでは fapppx を使用します。オプションは共通で、結果の出力を指示する -A と、データの格納ディレクトリを指定する -d が必須オプションです。出力コマンドの使用例を以下に示します。

例)計算ノードで ./tmp2 に格納されたデータを出力し、テキストファイル fapp.out に保存する

```
fapp -A -d./tmp2 -Icpupa > fapp.out
```

例)ログインノードで ./tmp2 に格納されたデータを出力し、テキストファイル fapppx.out に保存する

```
fapppx -A -d./tmp2 -Icpupa > fapppx.out
```

詳細プロファイラの出力結果から、対象区間の CPU 動作状況を確認することができます。出力結果の例を以下に示します。

例) 詳細プロファイラの出力結果 (CPU 動作状況)

Performance monitor event : statistics						

Application						

Kind	Execution time (s)	Floating-point GFLOPS	peak ratio (%)	Mem throughput (GB/s)	Mem throughput peak ratio (%)	
AVG	62.1934	5.0238	7.2406	12.2982	1.2010	all 0
MAX	62.1977	5.0239	7.2449	12.3759	1.2086	
MIN	62.1926	5.0235	7.2376	12.2208	1.1934	
Kind	Execution time (s)	Floating-point GFLOPS	peak ratio (%)	Mem throughput (GB/s)	Mem throughput peak ratio (%)	
AVG	56.8546	5.4967	7.9221	11.1413	1.0880	loop1 1
MAX	57.8173	5.5818	8.0495	11.2668	1.1003	
MIN	55.9764	5.4041	7.7860	10.9915	1.0734	
Kind	Effective instruction	Floating-point operation	SIMD inst. rate (%)	SVE operation rate (%)		
AVG	74001009915	312450007443	43.7231	99.9943		all 0
MAX	75279374634	312450008161	44.5598	99.9943		
MIN	72594604596	312450007340	42.9701	99.9943		
Kind	Effective instruction	Floating-point operation	SIMD inst. rate (%)	SVE operation rate (%)		
AVG	68261282379	312450001875	45.5552	99.9943		loop1
MAX	68261294415	312450001875	45.5552	99.9943		
MIN	68261276275	312450001875	45.5552	99.9943		

1

6.2.1.3. CPU 性能解析レポート

CPU 性能解析レポートの使用手順を以下に示します。

(1) 対象プログラムのソースに計測区間指定ルーチンを追加します。

プロファイルデータを計測する区間を指定するため、ソースコードに計測開始ルーチンと計測終了ルーチンを追加します。追加方法は詳細プロファイラと同じです。

(2) 対象プログラムをコンパイル/リンクして実行ファイルを生成します。

翻訳時オプションは通常と同じです。CPU 性能解析レポート向けのオプション追加はありません。

(3) 実行ファイルを計算ノードで実行し、プロファイルデータを計測します。

詳細プロファイラの計測コマンド `fapp` を付加して実行します。CPU 性能解析レポートは、表示する情報の種類や粒度によって以下の 4 つの段階に分かれています。

表 6-3 CPU 性能解析レポートの種別

段階	計測回数	概要
単体レポート	1	最少の計測回数で取得できる大まかな情報を表示する。
簡易レポート	5	単体レポートより多くの項目を表示する。
標準レポート	11	簡易レポートと同じ項目を、より詳細な分類で表示する。
詳細レポート	17	最も詳細な情報を表示する。

段階に応じた回数分のデータ計測を実施する必要があります。実行時は `-Hevent` オプションに `pa1` から最大 `pa17` まで番号を変えて、`-d` オプションにそれぞれ異なる格納ディレクトリを指定します。簡易レポート向けの計測コマンドの使用例を以下に示します。

例) CPU 性能解析レポート（簡易レポート）の計測を実施し、結果を `./rep1`～`./rep5` に格納する。

```
fapp -C -d./rep1 -Hevent=pa1 mpiexec ./a.out  
fapp -C -d./rep2 -Hevent=pa2 mpiexec ./a.out  
fapp -C -d./rep3 -Hevent=pa3 mpiexec ./a.out  
fapp -C -d./rep4 -Hevent=pa4 mpiexec ./a.out  
fapp -C -d./rep5 -Hevent=pa5 mpiexec ./a.out
```

(4) CPU 性能解析レポートの入力となる CSV 形式ファイルを出力します。

計算ノードでは計測コマンドと同じ fapp を使用します。ログインノードでは fapppx を使用します。オプションは共通で、CSV 形式で出力するためにオプション-tcsv を指定し、pa1.csv～pa17.csv のファイル名で格納します。簡易レポート向けの出力コマンドの使用例を以下に示します。

例)計算ノードで CPU 性能解析レポートの入力データを pa1.csv～pa5.csv に保存する。

```
fapp -A -d./rep1 -Icpupa -tcsv -o pa1.csv  
fapp -A -d./rep2 -Icpupa -tcsv -o pa2.csv  
fapp -A -d./rep3 -Icpupa -tcsv -o pa3.csv  
fapp -A -d./rep4 -Icpupa -tcsv -o pa4.csv  
fapp -A -d./rep5 -Icpupa -tcsv -o pa5.csv
```

例)ログインノードで CPU 性能解析レポートの入力データを pa1.csv～pa5.csv に保存する。

```
fapppx -A -d./rep1 -Icpupa -tcsv -o pa1.csv  
fapppx -A -d./rep2 -Icpupa -tcsv -o pa2.csv  
fapppx -A -d./rep3 -Icpupa -tcsv -o pa3.csv  
fapppx -A -d./rep4 -Icpupa -tcsv -o pa4.csv  
fapppx -A -d./rep5 -Icpupa -tcsv -o pa5.csv
```

(5) CSV 形式ファイルを取り込んで CPU 性能解析レポートを作成します。

Microsoft Excel が使用できる環境に以下のファイルをコピーし、全て同じディレクトリに格納します。

- (4)で作成した CSV 形式ファイル
- CPU 性能解析レポートファイル

CPU 性能解析レポートファイルはログインノードに以下のファイル名で格納されています。

```
/opt/FJSVxtclanga/tcsds-1.2.41/misc/cpupa/cpu_pa_report.xlsm
```

CPU 性能解析レポートファイル(cpu_pa_report.xlsm)を起動すると、内容を選択するためのダイアログが表示されますので、必要な情報を入力します。



CPU 性能解析レポートは Microsoft Excel のマクロ機能を使用しています。セキュリティ設定によってマクロが無効化された場合は手動でマクロを有効にしてください。

入力が完了すると CSV 形式ファイルを読み込んで CPU 性能解析レポートが表示されます。簡易レポートの表示例を以下に示します。

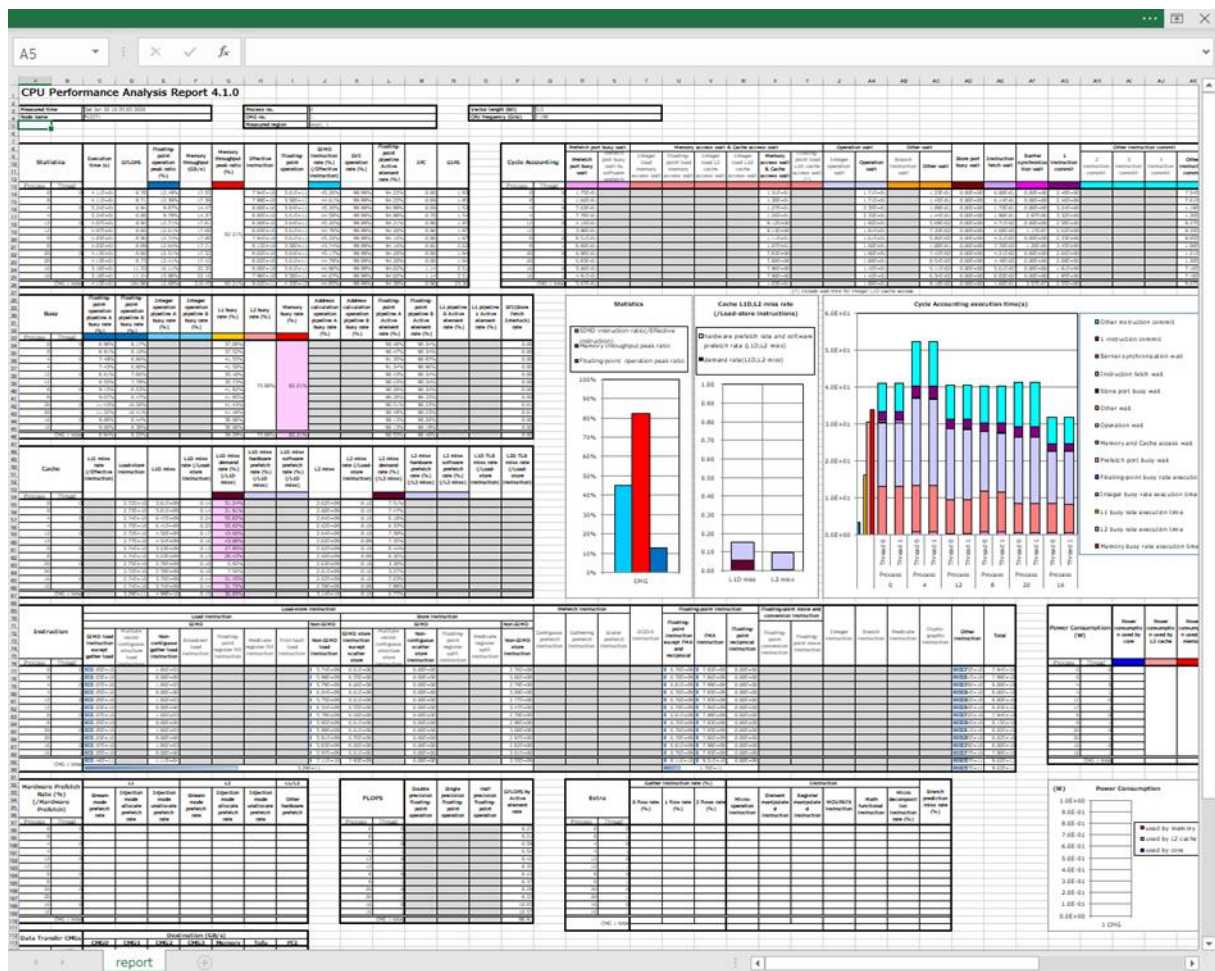


図 6-1 CPU 性能解析レポート(簡易レポート)の表示例

6.2.2. 並列実行デバグ

デバグとして、並列実行デバグが利用できます。並列実行デバグは下記の機能で構成されています。詳細な利用方法については、ポータルドキュメントをご参照ください。

表 6-4 並列実行デバグの構成

機能	概要
異常終了調査	プログラムの異常終了時にバクトレースなどの実行情報を取得する。
デッドロック調査	プログラムが終了しない場合や応答しない場合にバクトレースなどの実行情報を取得する。
重複除去	異常終了調査機能またはデッドロック調査機能と合わせて使用する。結果ファイルに可読性向上のためのデータ加工を行う。
コマンドファイルによるデバグ制御	プロセスごとに異なるデバグ制御を行う機能を提供する。デバグ制御には GDB を使用する。

6.2.2.1. コンパイル/リンク

コンパイル、リンクの際に、-g オプションを追加してください。

```
[username@wisteria01 work]$ frtpx -g [option] file ... # Fortran
[username@wisteria01 work]$ fccpx -g [option] file ... # C
[username@wisteria01 work]$ FCCpx -g [option] file ... # C++
[username@wisteria01 work]$ mpifrtpx -g [option] file... # MPI Fortran
[username@wisteria01 work]$ mpifccpx -g [option] file ... # MPI C
[username@wisteria01 work]$ mpiFCCpx -g [option] file ... # MPI C++
```

6.2.2.2. 異常終了調査機能

本機能は実行時の mpiexec コマンドに -fjdbg-sig オプションを追加することによって使用します。オプションの指定例を以下に示します。

例)異常終了調査で検出可能なシグナルを全て捕捉し、結果を ./log ディレクトリに出力する。

```
mpiexec -fjdbg-sig all -fjdbg-out-dir ./log -n 4 ./a.out
```

異常終了が発生した場合は出力ディレクトリの中に調査結果ファイルが生成されます。ファイルをそのまま参照することもできますが、重複除去機能コマンド fjdbg_summary を使用することで、重複を除去して整理された形で見るができます。コマンドの使用例を以下に示します。

例)異常終了調査の結果を、重複除去機能を使用して表示する。

```
fjdbg_summary -a ./log/signal
```

コマンド実行結果は標準出力に出力されます。出力結果の例を以下に示します。

例)異常終了調査の出力結果（重複除去機能を適用）

```
=====
Thread : 1   Signal: SIGSEGV [8]
-----
[ 8 ] <1 processes>
-----
0:   0x000000000040822c in main ()
1:   0x0000000000401984 in Test_f90_MPI () at /home/user/dbgtest
/Testxpr.f90:132
    locals:
        integer(kind=4) mx0 = 1025 [8]
        integer(kind=4) my0 = 513 [8]
        integer(kind=4) mz0 = 513 [8]
        (省略)
2:=>*   0x0000000000404ed8 in jacobi () at /home/user/dbgtest/
Testxpr.f90:382
    params:
        integer(kind=4) nn = 3 [8]
        real(kind=4) gosa = 0 [8]
    locals:
        real(kind=4) (:,:,) p = [8]
        real(kind=4) (:,:,) a = [8]
        real(kind=4) (:,:,) b = [8]
        real(kind=4) (:,:,) c = [8]
        real(kind=4) (:,:,) bnd = [8]
        real(kind=4) (:,:,) wrk1 = [8]
        real(kind=4) (:,:,) wrk2 = [8]
        integer(kind=4) ndx = 3 [8]
        integer(kind=4) ndy = 4 [8]
        integer(kind=4) ndz = 2 [8]
        integer(kind=4) (3) iop = [8]
        integer(kind=4) (2) npx = [8]
        integer(kind=4) (2) npy = [8]
        integer(kind=4) (2) npz = [8]
        (省略)
```

6.2.2.3. デッドロック調査機能

本機能は実行時の `mpiexec` コマンドに `-fjdbg-dlock` オプションを追加することによって使用します。また直前に `trap` コマンドでシグナル受信時の設定を変更する必要があります。コマンドの指定例を以下に示します。

例)実行コマンドにデッドロック調査機能を適用し、結果を `./log` ディレクトリに出力する。

```
trap `echo SIGHUP/SIGXCPU received.` HUP XCPU
mpiexec -fjdbg-dlock -fjdbg-out-dir `./log` -n 2 ./a.out
```

コマンドを指定したジョブを実行してデッドロックが疑われる状態になったとき、ログインノードから以下のコマンドを実行すると、調査結果ファイルを格納してジョブが終了します。

```
pjsig -s SIGHUP 対象ジョブ ID
```

調査結果ファイルをそのまま参照することもできますが、重複除去機能コマンド `fjdbg_summary` を使用することで、重複を除去して整理された形で見ることができます。コマンドの使用例を以下に示します。

例)デッドロック調査の結果を、重複除去機能を使用して表示する。

```
fjdbg_summary -a ./log/deadlock
```

コマンド実行結果は標準出力に出力されます。出力結果の例を以下に示します。

例)デッドロック調査の出力結果（重複除去機能を適用）

```
=====
Thread : 1   Signal: SIGHUP [0-1]
-----
[ 0-1 ] <2 processes>
-----
0:   0x000000000040855c in main ()
-----
[ 0 ] <1 processes>
-----
1:   0x00000000004018b4 in Test_f90_MPI () at /home/user/dlock
/Testxpr.f90:111
    locals:
        integer(kind=4) mx0 = 1025 [0]
        (省略)
        integer(kind=4) mx = 1024 [0]
2:   0x00004000001b1ffc in pmpi_send__ () from=/opt/FJSVxtc
langa/tcsds-1.2.26/lib64/libmpi_mpifh.so.0
-----
[ 1 ] <1 processes>
-----
1:   0x00000000004019e4 in Test_f90_MPI () at /home/user/dlock
/Testxpr.f90:114
    locals:
        integer(kind=4) mx0 = 1025 [1]
        (省略)
        integer(kind=4) mx = 1024 [1]
2:   0x00004000001b1ffc in pmpi_send__ () from=/opt/FJSVxtc
langa/tcsds-1.2.26/lib64/libmpi_mpifh.so.0
        (省略)
```

6.2.2.4. コマンドファイルによるデバグ制御機能

本機能は実行時の `mpirun` コマンドに `-gdbx` オプションを追加することによって使用します。コマンドの指定例を以下に示します。

例)実行コマンドにコマンドファイルによるデバグ制御機能を指定する。

```
mpirun -gdbx ./command.txt -fjdbg-out-dir ./log -n 2 ./a.out
```

`-gdbx` オプションに指定するコマンドファイルには、実行時に使用する GDB のコマンドを記述することができます。コマンドファイルの記述例を以下に示します。

例)コマンドファイルの記述例

```
break Testxpr.f90:374
run
print nn
print kmax
list
quit
```

`-fjdbg-out-dir` オプションで格納ディレクトリを指定すると、プロセス毎のデバグ結果ファイルを出力します。結果ファイルの出力例を以下に示します。

例) デバグ結果ファイルの出力結果

```
Breakpoint 1 at 0x404c20: file Testxpr.f90, line 375.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
[New Thread 0x400001dff010 (LWP 270)]
[New Thread 0x4000021ff010 (LWP 276)]

Thread 1 "testBMT" hit Breakpoint 1, jacobi (nn=3, gosa=0) at Testxpr.f90:375
375      do loop=1,nn
$1 = 3
$2 = 257
370      integer,intent(in) :: nn
371      real(4),intent(inout) :: gosa
372      integer :: i,j,k,loop,ierr
373      real(4) :: s0,ss,wgosa
374      !
375      do loop=1,nn
376          gosa=0.0
377          wgosa=0.0
378          do k=2,kmax-1
379              do j=2,jmax-1
(省略)
```

6.2.3. 統合開発環境

統合開発環境として、Eclipse Foundation が提供する Eclipse IDE for Scientific Computing パッケージが利用可能です。FUJITSU DEVELOPMENT STUDIO 向けに以下の拡張機能を提供します。

表 6-5 Eclipse に対する富士通拡張機能

機能名	概要
富士通コンパイラを使用したビルド	FUJITSU DEVELOPMENT STUDIO のコンパイラを使用可能にします。
ジョブの投入	ジョブの投入を可能とします。
ジョブの状況確認および操作	ジョブおよびノードの状況表示やジョブ操作を可能とします。
CPU 性能解析レポートの作成	プロファイラの CPU 性能解析レポートの計測およびレポート作成を可能とします。

6.2.3.1. 統合開発環境のインストール

統合開発環境を使用するには、ご利用のクライアントマシンに Eclipse を導入する必要があります。動作確認済の環境を以下に示します。

表 6-6 動作確認済み環境

基本ソフトウェア	Java Runtime Environment(JRE)	Eclipse
Microsoft Windows8.1(64bit)	Oracle JDK 8 Update 241	Eclipse IDE 2020-06 R Package
Microsoft Windows10(64bit)		
macOS Catalina		
Red Hat Enterprise Linux 8.1		

クライアントマシンに JRE または JDK と、Eclipse IDE for Scientific Computing をインストールした上で、富士通拡張機能の設定を行います。設定方法は「Development Studio 統合開発環境使用手引書」の「第 2 章 統合開発環境の導入手順」を参照してください。

6.2.3.2. 統合開発環境の利用

クライアントマシンで Eclipse を起動します。最初にワークスペースを指定するウィンドウが表示されますので、[Workspace:]の欄を設定した後、右下の Launch ボタンを押します。

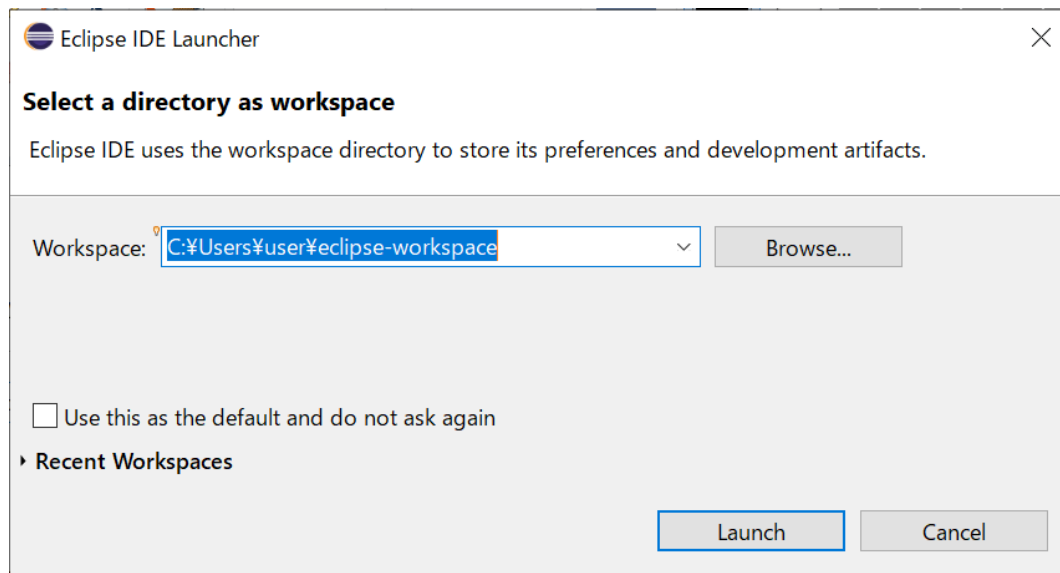


図 6-2 Eclipse 起動後の設定画面

Eclipse の初期画面が表示されるので、メニュー画面を操作してプロジェクトの作成などを行います。使用画面の例を以下に示します。

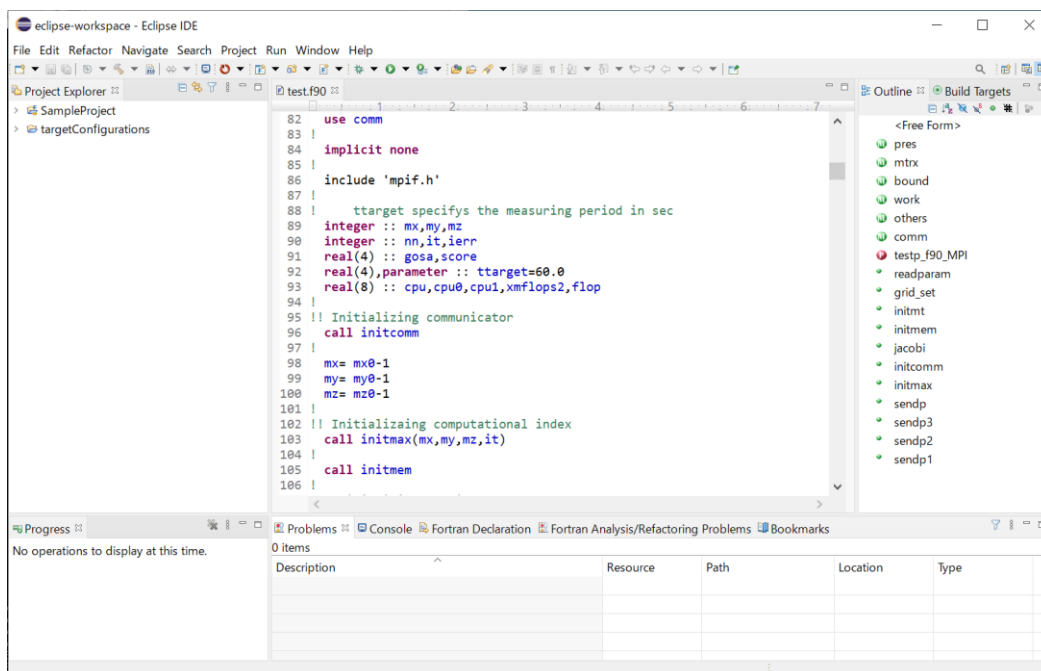


図 6-3 Eclipse 使用画面の例

6.3. Aquarius 向け開発ツール

6.3.1. インテル VTune プロファイラー

パフォーマンス分析ツールとして、インテル VTune プロファイラーが利用できます。詳細な利用方法については、利用支援ポータルのドキュメントをご参照ください。

6.3.1.1. コンパイル/リンク

コンパイル、リンクの際に、-g オプションおよび-O2 オプションを追加してください。

```
[username@wisteria01 work]$ ifort -g -O2 [option] file ...      # Fortran
[username@wisteria01 work]$ icc -g -O2 [option] file ...      # C
[username@wisteria01 work]$ icpc -g -O2 [option] file ...     # C++
[username@wisteria01 work]$ mpiifort -g -O2 [option] file...  # MPI Fortran
[username@wisteria01 work]$ mpiicc -g -O2 [option] file ...   # MPI C
[username@wisteria01 work]$ mpiicpc -g -O2 [option] file ...  # MPI C++
```

6.3.1.2. インタラクティブジョブの実行

GUI 版のインテル VTune プロファイラーを使用する場合は、X11 Forwarding を有効にして Wisteria/BDEC-01 システムにログインし、更にインタラクティブジョブ実行時に --X11 オプションを指定してください。

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-a,node=1 --X11
```

CUI 版を使用する場合は、通常のインタラクティブジョブと同様の指定で構いません。

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-a,node=1
```

6.3.1.3. 環境設定

インタラクティブが開始された後、以下のコマンドを実行し環境設定を行います。

```
[username@wa01 work]$ module load intel
```



計算ノードにて環境変数\$HOME(ホームディレクトリ)領域が参照できる必要があります。

chhome コマンドでホーム領域を変更する、または、以下のように環境変数\$HOME に/work 領域のアクセス可能なパスを設定ください。

```
[username@wa01 work]$ export HOME=/work/groupname/username
```

6.3.1.4. インテル VTune プロファイラーの起動

GUI 版を使用する場合は、`vtune-gui` コマンドを実行します。

```
[username@wa01 work]$ vtune-gui
```

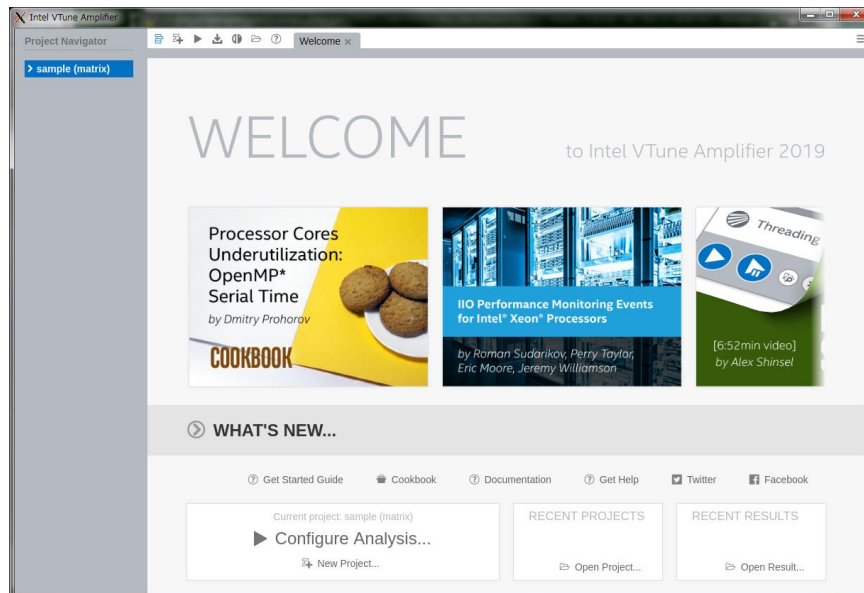


図 6-4 インテル VTune プロファイラー起動画面

CUI 版を使用する場合は、`vtune` コマンドを実行します。

```
[username@wa01 work]$ vtune <-action> [-action-option] ¥  
[-global-option] [--] target [target options]
```

MPI プログラムに対してインテル VTune プロファイラーを実行する場合は、`mpiexec` コマンドで起動するプログラムとして `vtune` を指定します。

```
[username@wa01 work]$ mpiexec -n ${PJM_MPI_PROC} ¥  
vtune -collect hpc-performance -data-limit=0 -r my_result_dir ./a.out
```

6.3.2. Arm DDT

Aquarius 対応のデバッガとして、Arm DDT が利用できます。詳細な利用方法については、利用支援ポータル のドキュメントをご参照ください。

6.3.2.1. コンパイル/リンク

コンパイル、リンクの際に、-g オプションを追加してください。

```
[username@wisteria01 work]$ ifort -g [option] file ...      # Fortran
[username@wisteria01 work]$ icc -g [option] file ...       # C
[username@wisteria01 work]$ icpc -g [option] file ...      # C++
[username@wisteria01 work]$ mpiifort -g [option] file...   # MPI Fortran
[username@wisteria01 work]$ mpiicc -g [option] file ...    # MPI C
[username@wisteria01 work]$ mpiicpc -g [option] file ...   # MPI C++
```

6.3.2.2. インタラクティブジョブの実行

Wisteria/BDEC-01 システムに X11 Forwarding を有効にしログインし、更にインタラクティブジョブ投入時に --X11 オプションを指定してください。

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-a,node=1 --mpi proc=4 --X11
```

6.3.2.3. 環境設定

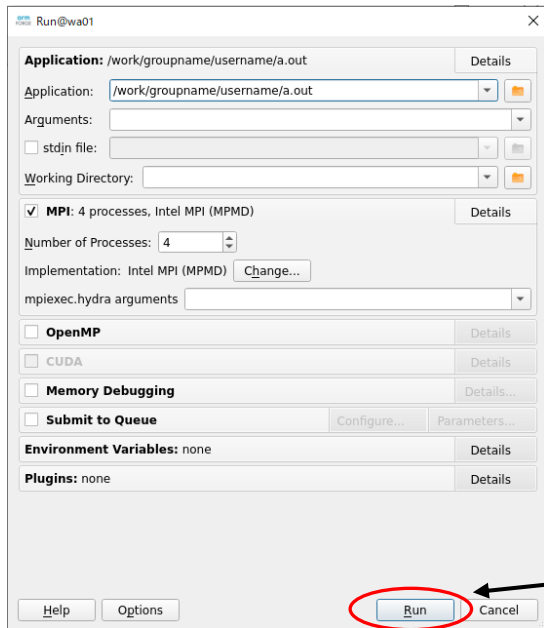
インタラクティブが開始された後、以下のコマンドを実行し環境設定を行います。

```
[username@wa01 work]$ module load forge
```

6.3.2.4. Arm DDT の起動

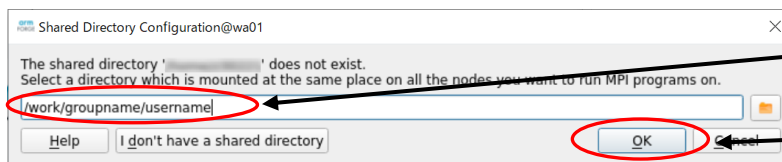
ddt コマンドを実行します。

```
[username@wa01 work]$ ddt -n ${PJM_MPI_PROC} ./a.out
```



1. Run をクリック

図 6-5 DDT 起動画面その 1



1. 共有ファイルシステムを
入力

2. OK をクリック

図 6-6 DDT 起動画面その 2

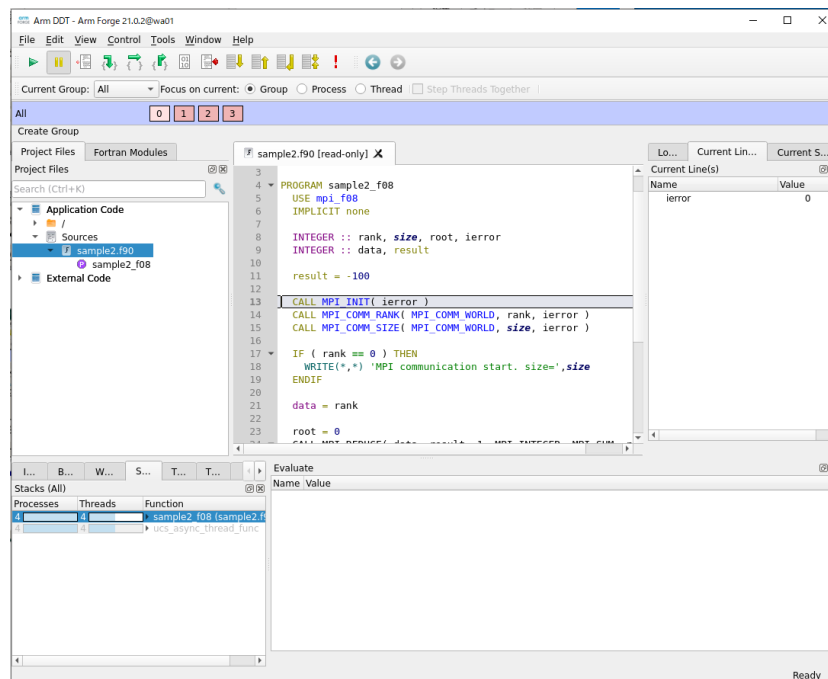


図 6-7 DDT 画面

6.3.3. Arm MAP

Aquarius 用のプロファイリングツールとして、Arm MAP が利用できます。詳細な利用方法については、利用支援ポータル のドキュメント をご参照ください。

6.3.3.1. コンパイル/リンク

コンパイル、リンクの際に、-g オプションを追加してください。

```
[username@wisteria01 work]$ ifort -g [option] file ...      # Fortran
[username@wisteria01 work]$ icc -g [option] file ...       # C
[username@wisteria01 work]$ icpc -g [option] file ...      # C++
[username@wisteria01 work]$ mpiifort -g [option] file...    # MPI Fortran
[username@wisteria01 work]$ mpiicc -g [option] file ...     # MPI C
[username@wisteria01 work]$ mpiicpc -g [option] file ...   # MPI C++
```

6.3.3.2. インタラクティブジョブの実行

Wisteria/BDEC-01 システムに X11 Forwarding を有効にしログインし、更にインタラクティブジョブ投入時に --X11 オプションを指定してください。

```
[username@wisteria01 work]$ pjsub --interact -g group1 ¥
-L rscgrp=interactive-a,node=1 --X11
```

6.3.3.3. 環境設定

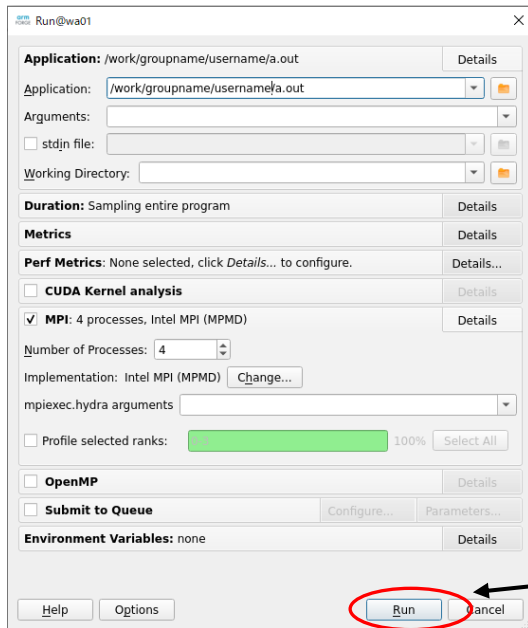
インタラクティブが開始された後、以下のコマンドを実行し環境設定を行います。

```
[username@wa01 work]$ module load forge
```

6.3.3.4. Arm MAP の起動

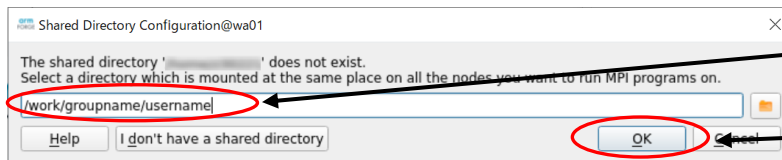
map コマンドを実行します。

```
[username@wa01 work]$ map ./a.out
```



2. Run をクリック

図 6-8 MAP 起動画面その 1



3. 共有ファイルシステムを
入力

4. OK をクリック

図 6-9 MAP 起動画面その 2

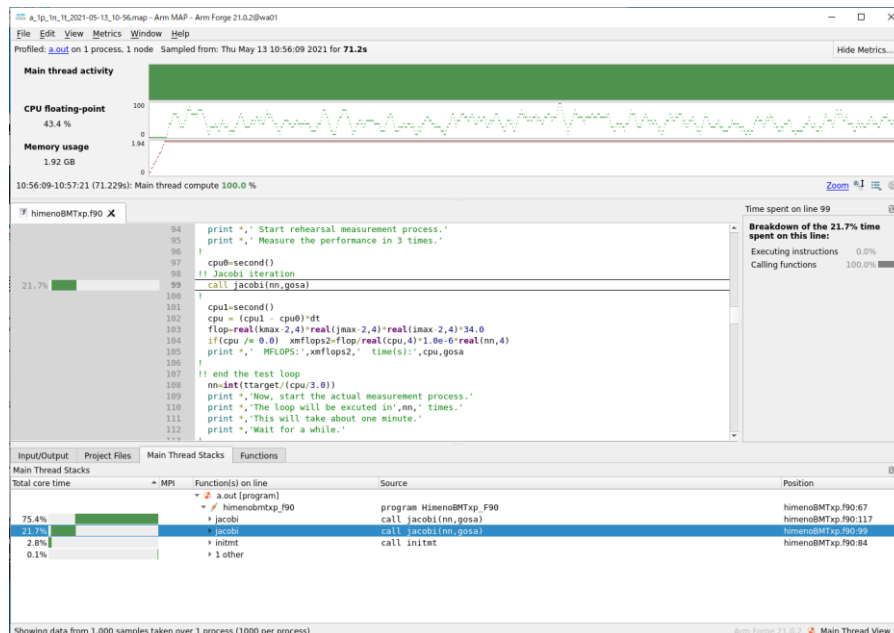


図 6-10 MAP 画面

7. マニュアル参照

コンパイラやデバッガ、各種ソフトウェアのマニュアルは、利用支援ポータルで公開しています。

(1) ブラウザで下記 URL にアクセスします。

Wisteria 利用支援ポータル: <https://wisteria-www.cc.u-tokyo.ac.jp/>

(2) 利用支援ポータルにログインします。

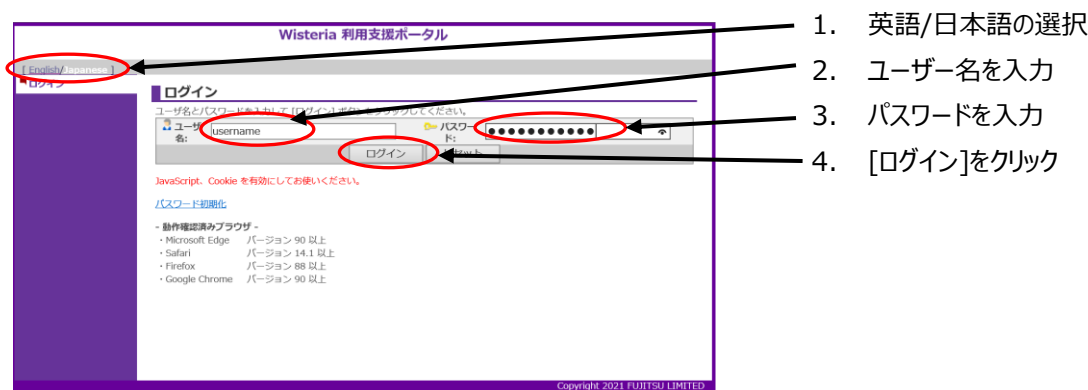


図 7-1 利用支援ポータルログイン画面

(3) [ドキュメント閲覧]メニューへアクセスします。

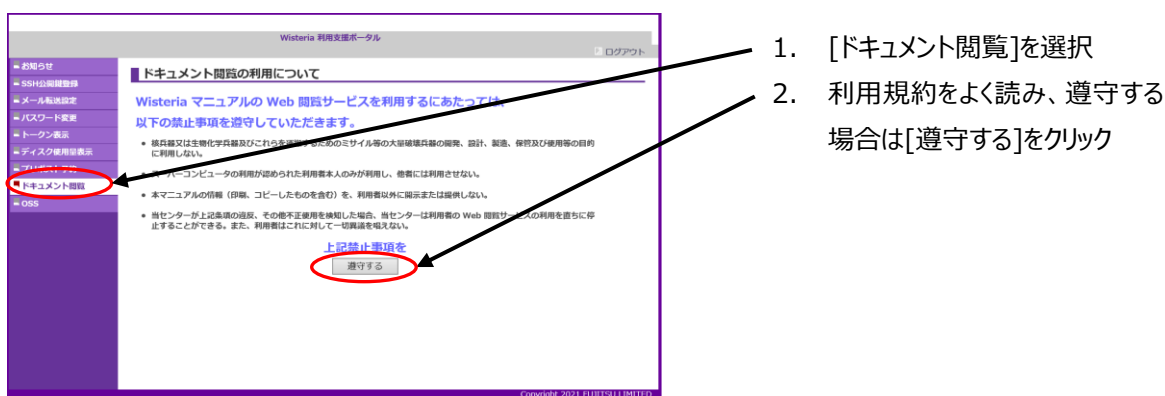


図 7-2 利用規約の確認

(4) 必要なマニュアルを表示します。

8. アプリケーション

Wisteria/BDEC-01 システムで利用可能なアプリケーションを表 8-1 に示します。利用の際には module コマンドを用いて必要な環境変数を設定してください。

表 8-1 アプリケーション一覧

アプリケーション名	公開元サイト
OpenFOAM	http://www.openfoam.org/
ABINIT-MP	http://www.ciss.iis.u-tokyo.ac.jp/dl/index.php
PHASE	
FrontFlow/blue	
FrontISTR	
REVOCAP_Coupler	
REVOCAP_Refiner	
OpenMX	http://www.openmx-square.org/
MODYLAS	http://www.modylas.org/
GROMACS	http://www.gromacs.org/
BLAST	https://blast.ncbi.nlm.nih.gov/
bioconductor	https://www.bioconductor.org/
BioPerl	http://bioperl.org/
BioRuby	http://bioruby.org/
BWA	http://bio-bwa.sourceforge.net/
GATK	https://software.broadinstitute.org/gatk/
SAMtools	http://samtools.sourceforge.net/
Quantum ESPRESSO	https://www.quantum-espresso.org/
Xcrypt	https://github.com/xcrypt-job/xcrypt
LAMMPS	https://lammps.sandia.gov
CP2K	https://www.cp2k.org
NWChem	https://www.nwchem-sw.org
Paraview	https://www.paraview.org/
VisIt	https://wci.llnl.gov/simulation/computer-codes/visit
POV-Ray	http://www.povray.org/

Wisteria/BDEC-01 システムで利用可能なアプリケーションの最新の情報については 2.6.5 利用可能なアプリケーション/ライブラリの一覧表示をご参照ください。

構築手順等は、利用支援ポータルで随時公開します。構築手順等は、利用支援ポータルの「OSS」より入手可能です。

Wisteria/BDEC-01 利用支援ポータル: https://wisteria-www.cc.u-tokyo.ac.jp/
--

Wisteria/BDEC-01 システムで利用可能な RIST(高度情報科学技術研究機構)から提供されたアプリケーションを表 8-2 に示します。利用方法につきましては、下記公開元サイトをご参照ください。

表 8-2 RIST から提供されたアプリケーション一覧

アプリケーション名	公開元サイト
AkaiKKR	https://www.hpci-office.jp/pages/akaikkr_jcahpc_2022-2
ALAMODE	https://www.hpci-office.jp/pages/alamode_jcahpc_2022-2
FrontISTR	https://www.hpci-office.jp/pages/frontistr_jcahpc_2022-2
GENESIS	https://www.hpci-office.jp/pages/genesis_jcahpc-2
Hφ	https://www.hpci-office.jp/pages/hphi_jcahpc_2022-2
MODYLAS	https://www.hpci-office.jp/pages/modylas_jcahpc_2022-2
mVMC	https://www.hpci-office.jp/pages/mvmc_jcahpc_2022-2
NTChem	https://www.hpci-office.jp/pages/ntchem_jcahpc_2022-2
OpenMX	https://www.hpci-office.jp/pages/openmx_jcahpc_2022-2
PHASE/0	https://www.hpci-office.jp/pages/phase0_jcahpc_2022-2
Phonopy	https://www.hpci-office.jp/pages/phonopy_jcahpc_2022-2
SALMON	https://www.hpci-office.jp/pages/salmon_jcahpc_2022-2
SMASH	https://www.hpci-office.jp/pages/smash_jcahpc_2022-2

9. 機械学習ライブラリ

9.1. PyTorch の利用方法

PyTorch 及び PyTorch + Horovod の利用方法について記載します。

9.1.1. PyTorch

module コマンドで実行環境を設定し、仮想環境を activate することで PyTorch を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM -g group1
#PJM -j

module load cuda/11.1
module load pytorch/1.8.1

source $PYTORCH_DIR/bin/activate # ← 仮想環境を activate
python3 train.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.1.2. PyTorch + Horovod

module コマンドで実行環境を設定し、仮想環境を activate することで PyTorch + Horovod を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=short-a
#PJM -L node=2
#PJM -L elapse=0:15:00
#PJM --mpi proc=16
#PJM -g group1
#PJM -j

module load cuda/11.1
module load nccl/2.8.4
module load gcc/8.3.1
module load omp/4.1.1
module load pytorch-horovod/1.8.1-0.21.3

GPUS_PER_NODE=`nvidia-smi -L | wc -l`
source $PYTORCH_HOROVOD_DIR/bin/activate # ← 仮想環境を activate
mpirun -np ${PJM_MPI_PROC} -machinefile ${PJM_O_NODEINF} ¥
-map-by ppr:${GPUS_PER_NODE}:node -mca pml ob1 ¥
-mca btl_tcp_if_include ib0,ib1,ib2,ib3 ¥
python3 pytorch_mnist.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.2. TensorFlow の利用方法

9.2.1. TensorFlow

module コマンドで実行環境を設定し、仮想環境を activate することで TensorFlow を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM -g group1
#PJM -j

module load cuda/11.0
module load cudnn/8.1.0
module load tensorflow/2.4.1

source $TENSORFLOW_DIR/bin/activate # ← 仮想環境を activate
python3 train.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.2.2. TensorFlow + Horovod

module コマンドで実行環境を設定し、仮想環境を activate することで TensorFlow + Horovod を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM --mpi proc=8
#PJM -g group1
#PJM -j

module load cuda/11.0
module load cudnn/8.1.0
module load nccl/2.8.4
module load gcc/8.3.1
module load omp/4.1.1
module load tensorflow-horovod/2.4.1-0.21.3

GPUS_PER_NODE=`nvidia-smi -L | wc -l`
source $TENSORFLOW_HOROVOD_DIR/bin/activate # ← 仮想環境を activate
mpirun -np ${PJM_MPI_PROC} -machinefile ${PJM_O_NODEINF} ¥
-map-by ppr:${GPUS_PER_NODE}:node -mca btl tcp -mca btl_tcp_if_include ib0,ib1,ib2,ib3 ¥
python3 mnist.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.2.3. TensorFlow + Keras

module コマンドで実行環境を設定し、仮想環境を activate することで TensorFlow + Keras を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM -g group1
#PJM -j

module load cuda/11.0
module load cudnn/8.1.0
module load tensorflow-keras/2.4.1-2.4.3

source $TENSORFLOW_KERAS_DIR/bin/activate # ← 仮想環境を activate
python3 mnist.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.2.4. TensorFlow + Keras + Horovod

module コマンドで実行環境を設定し、仮想環境を activate することで TensorFlow + Keras + Horovod を利用可能です。

以下に Aquarius でバッチジョブを使用して利用する方法を記載します。

(1) バッチジョブスクリプトを作成する。

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM --mpi proc=8
#PJM -g group1
#PJM -j

module load cuda/11.0
module load cudnn/8.1.0
module load nccl/2.8.4
module load gcc/8.3.1
module load ompi/4.1.1
module load tensorflow-keras-horovod/2.4.1-2.4.3-0.21.3

GPUS_PER_NODE=`nvidia-smi -L | wc -l`
source $TENSORFLOW_KERAS_HOROVOD_DIR/bin/activate # ← 仮想環境を activate
mpirun -np ${PJM_MPI_PROC} -machinefile ${PJM_O_NODEINF} ¥
-map-by ppr:${GPUS_PER_NODE}:node python3 mnist.py
```

(2) ジョブを投入する。

```
[username@wisteria01 work]$ pjsub run.sh
[INFO] PJM 0000 pjsub Job 12345 submitted.
```

9.2.5. 追加パッケージのインストール方法

Environment Modules を使用してシステムにインストール済の環境を使用する場合、システム環境には追加パッケージをインストールすることはできません。そのため、共有ファイルシステム領域 (/work/groupname/username) にご自身で python 環境を構築し、必要なパッケージをインストールしてください。

以下に追加パッケージとして Pillow 8.2.0 をインストールする例を記載しています。Aquarius にてインタラクティブジョブを実行し、python(バージョン 3.6.8)の仮想環境(test)を作成後、仮想環境に Tensorflow 2.4.1、Horovod 0.22.0、Pillow8.2.0 をインストールする手順となります。

例) Aquarius にて Pillow 8.2.0 のインストール

```
[username@wisteria01 work]$ pjsub --interact -L rscgrp=interactive-a,node=1 -g group1
[username@wa01 work]$ module load cuda/11.0
[username@wa01 work]$ module load cudnn/8.1.0
[username@wa01 work]$ module load nccl/2.8.4
[username@wa01 work]$ module load gcc/8.3.1
[username@wa01 work]$ module load ompi/4.1.1
[username@wa01 work]$ python3 -mvenv test
[username@wa01 work]$ source test/bin/activate
(test)[username@wa01 work]$ pip3 install --upgrade pip setuptools
(test)[username@wa01 work]$ pip3 install tensorflow==2.4.1
(test)[username@wa01 work]$ HOROVOD_WITH_TENSORFLOW=1 ¥
HOROVOD_GPU_OPERATIONS=NCCL HOROVOD_NCCL_HOME=$NCCL_HOME ¥
pip3 install --no-cache-dir horovod==0.22.0
(test)[username@wa01 work]$ pip install Pillow
(test)[username@wa01 work]$ exit
```

10. コンテナ環境

10.1. Singularity の利用方法

Singularity を利用するためには、pjsub コマンドの-L オプションに jobenv=singularity を指定してジョブを投入します。また、module コマンドで実行環境を設定します。

例) Aquarius で Singularity を利用する場合のジョブスクリプト

```
[username@wisteria01 work]$ vi run.sh
#!/bin/sh

#PJM -L rscgrp=debug-a
#PJM -L node=1
#PJM -L elapse=0:15:00
#PJM -L jobenv=singularity
#PJM -g group1
#PJM -j

module load singularity/3.7.3
singularity <command>
```